

**PENGEMBANGAN KONTROL AKSES
GRANULAR DAN DELEGASI *OFFLINE* PADA
SISTEM REKAM MEDIS ELEKTRONIK
DECMED MENGGUNAKAN MACAROONS**

Disusun sebagai syarat kelulusan tingkat sarjana

Oleh

Tazkia Nizami

NIM: 13522032



**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Juli 2026

**PENGEMBANGAN KONTROL AKSES
GRANULAR DAN DELEGASI *OFFLINE* PADA
SISTEM REKAM MEDIS ELEKTRONIK
DECMED MENGGUNAKAN MACAROONS**

Oleh

Tazkia Nizami

NIM: 13522032

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Telah disetujui dan disahkan sebagai Tugas Akhir
di Bandung, pada tanggal 2 Juli 2026

Pembimbing,

Dr. Phil. Eng. Hari Purnama, S.Si., M.Si.

NIP. 118110072

LEMBAR PERNYATAAN

Dengan ini saya menyatakan bahwa:

1. Pengerjaan dan penulisan Laporan Tugas Akhir ini dilakukan tanpa menggunakan bantuan yang tidak dibenarkan.
2. Segala bentuk kutipan dan acuan terhadap tulisan orang lain yang digunakan di dalam penyusunan laporan tugas akhir ini telah dituliskan dengan baik dan benar.
3. Laporan Tugas Akhir ini belum pernah diajukan pada program pendidikan di perguruan tinggi mana pun.

Jika terbukti melanggar hal-hal di atas, saya bersedia dikenakan sanksi sesuai dengan Peraturan Akademik dan Kemahasiswaan Institut Teknologi Bandung bagian Penegakan Norma Akademik dan Kemahasiswaan khususnya Pasal 2.1 dan Pasal 2.2.

Bandung, 2 Juli 2026



Tazkia Nizami

NIM: 13522032

ABSTRAK

PENGEMBANGAN KONTROL AKSES GRANULAR DAN DELEGASI *OFFLINE* PADA SISTEM REKAM MEDIS ELEKTRONIK DECMED MENGGUNAKAN MACAROONS

Oleh:

Tazkia Nizami
NIM: 13522032

Transformasi Rekam Medis Elektronik (RME) menuntut mekanisme kontrol akses yang menjaga kendali pasien sekaligus mendukung alur pelayanan kesehatan kolaboratif. DecMed sebagai sistem RME terdesentralisasi telah memanfaatkan IOTA, IPFS, dan Proxy Re-Encryption (PRE), tetapi mekanisme sebelumnya masih bergantung pada pasien untuk menerbitkan akses baru, belum merepresentasikan batasan token secara cukup kontekstual, dan menyimpan data RME sebagai objek besar yang sulit ditegakkan secara granular. Tugas akhir ini bertujuan merancang dan mengimplementasikan perluasan kontrol akses pada DecMed agar akses dapat dibatasi sesuai aktor, konteks layanan, segmen data, waktu berlaku, dan rantai delegasi. Pendekatan yang digunakan mengintegrasikan Macaroon sebagai token otorisasi dengan *caveats* untuk atenuasi, segmentasi data RME *off-chain* berdasarkan kategori dataset dan fungsi klinis, pencatatan metadata serta audit pada IOTA, dan PRE sebagai titik penegakan otoritas sebelum data terenkripsi diambil atau diproses ulang untuk penerima akses. Implementasi mencakup komponen segmentasi RME, token Macaroon, penegakan otoritas pada *Server PRE*, *smart contract IOTA Move*, *Patient Client*, dan *Hospital Client* untuk alur pasien, petugas rekam medis, perawat, dokter, petugas laboratorium, dan apoteker. Pengujian dilakukan melalui 12 skenario fungsional pada alur rawat jalan dan rawat inap serta pengujian unit atau integrasi PU-TA-01 sampai PU-TA-09 untuk kebutuhan nonfungsional. Evaluasi menunjukkan seluruh kebutuhan fungsional dan nonfungsional yang dirumuskan dinyatakan terpenuhi. Hasil penelitian ini menunjukkan bahwa kontrol akses granular, segmentasi RME, delegasi terbatas, revokasi, audit, dan verifikasi identitas aktor aktif dapat dioperasikan dalam DecMed pada batas implementasi prototipe yang diuji.

Kata kunci: *Rekam Medis Elektronik, kontrol akses granular, Macaroon, IOTA, Proxy Re-Encryption*

ABSTRACT

PENGEMBANGAN KONTROL AKSES GRANULAR DAN DELEGASI *OFFLINE* PADA SISTEM REKAM MEDIS ELEKTRONIK DECMED MENGGUNAKAN MACAROONS

By:

Tazkia Nizami
NIM: 13522032

The transformation of electronic medical records (EMRs) requires access-control mechanisms that preserve patient control while supporting collaborative healthcare workflows. DecMed, as a decentralized EMR system, already uses IOTA, IPFS, and Proxy Re-Encryption (PRE); however, its previous mechanism still depended on patients to issue new access, did not represent token constraints with sufficient context, and stored EMR data as large objects that were difficult to enforce granularly. This thesis aims to design and implement an access-control extension for DecMed so that access can be constrained by actor, service context, data segment, validity period, and delegation chain. The proposed approach integrates Macaroon as an authorization token with caveats for attenuation, off-chain EMR segmentation based on dataset categories and clinical functions, metadata and audit recording on IOTA, and PRE as the authority enforcement point before encrypted data is retrieved or reprocessed for an access recipient. The implementation includes EMR segmentation components, Macaroon-based tokens, authority enforcement on the PRE Server, IOTA Move smart contracts, the Patient Client, and the Hospital Client for workflows involving patients, medical record officers, nurses, physicians, laboratory officers, and pharmacists. Testing was conducted through 12 functional scenarios for outpatient and inpatient workflows, as well as unit or integration tests from PU-TA-01 to PU-TA-09 for nonfunctional requirements. The evaluation states that all formulated functional and nonfunctional requirements were fulfilled. The results show that granular access control, EMR segmentation, limited delegation, revocation, auditing, and active-actor identity verification can be operationalized in DecMed within the evaluated prototype implementation boundaries.

Keywords: electronic medical records, granular access control, Macaroon, IOTA, Proxy Re-Encryption

KATA PENGANTAR

Puji dan syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas berkat dan rahmatnya, laporan tugas akhir yang berjudul "Pengembangan Kontrol Akses Granular dan Delegasi *Offline* pada Sistem Rekam Medis Elektronik DecMed Menggunakan Macaroons" dapat diselesaikan dalam rangka memenuhi syarat kelulusan tingkat sarjana. Perlu diakui pengerjaan tugas akhir ini didukung oleh banyak pihak. Khususnya, penulis ingin mengucapkan terima kasih kepada:

1. Ani Rochani Rochmawati dan Widodo selaku orang tua penulis dan Fathiya Alisa Zahrandika selaku kakak penulis atas dukungan materi dan emosi selama ini hingga penulis berhasil menyelesaikan pengerjaan tugas akhir.
2. Bapak Dr. Phil. Eng. Hari Purnama, S.Si., M.Si., selaku dosen pembimbing atas segala bentuk dukungan yang telah diberikan dan kesabarannya dalam membimbing penulis serta memberikan saran dalam pengerjaan tugas akhir.
3. Seluruh pihak lain yang tidak disebutkan disini yang telah membantu dan mendukung dalam proses pengerjaan tugas akhir.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah terlibat dalam pengerjaan tugas akhir ini. Penulis juga ingin menyampaikan mohon maaf apabila terdapat kesalahan maupun kekurangan dalam laporan tugas akhir ini. Penulis berharap semoga tugas akhir ini dapat bermanfaat bagi pembaca dan riset-riset kedepannya.

Bandung, 2 Juli 2026

A handwritten signature in black ink, appearing to be 'Tazkia Nizami', written in a cursive style.

Tazkia Nizami

DAFTAR ISI

ABSTRAK	iv
ABSTRACT	v
KATA PENGANTAR.....	vi
DAFTAR ISI.....	viii
DAFTAR LAMPIRAN.....	xiii
DAFTAR GAMBAR.....	xiv
DAFTAR TABEL	xvi
DAFTAR KODE PROGRAM	xviii
BAB I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	3
I.4 Batasan Masalah	4
I.5 Metodologi	4
I.6 Sistematika Pembahasan.....	5
BAB II STUDI LITERATUR	6
II.1 Rekam Medis Elektronik	6
II.1.1 Definisi RME.....	6
II.1.2 Isi RME.....	6
II.1.3 RME dalam Alur Pelayanan Kesehatan Kolaboratif	7
II.1.4 Hak Akses RME	8
II.2 IOTA.....	9
II.2.1 Distributed Ledger Technology	9
II.2.2 IOTA	10
II.3 <i>InterPlanetary File System</i> (IPFS)	10
II.4 <i>Proxy Re-Encryption</i> (PRE)	11
II.4.1 Umbral PRE	11
II.5 Kontrol Akses	13
II.5.1 Definisi Kontrol Akses.....	13
II.5.2 Kontrol Akses Granular.....	13

II.5.3	Model CapBAC	14
II.6	Token Otorisasi	15
II.6.1	JSON Web Token	15
II.6.2	PASETO	16
II.6.3	Macaroons	16
II.6.4	Biscuit Token	17
II.7	DecMed	18
II.7.1	Arsitektur DecMed	19
II.7.2	Alur Pengelolaan dan Akses RME pada DecMed	20
II.8	Pemodelan Ancaman STRIDE	21
II.9	Penelitian Terkait	22
II.9.1	Hybrid RBAC dan ABAC pada Sistem RME Berbasis Blockchain.....	22
II.9.2	Model Kontrol Akses Empat Lapis untuk RME Berbasis Blockchain.....	23
II.9.3	Kontrol Akses Granular Menggunakan <i>Composable NFT</i>	23
II.9.4	Implementasi Macaroons untuk Delegasi Akses pada Sistem Nyata	24
BAB III	ANALISIS DAN RANCANGAN SOLUSI	27
III.1	Analisis Masalah	27
III.1.1	Analisis Sistem DecMed.....	27
III.1.1.1	Keterbatasan Token Kapabilitas DecMed	28
III.1.1.2	Keterbatasan Segmentasi Data pada DecMed	29
III.1.1.3	Analisis Skenario Layanan Kesehatan Kolaboratif	31
III.1.1.4	Analisis STRIDE pada Sistem DecMed	33
III.1.1.5	Analisis Mekanisme Penggunaan dan Revokasi Token	34
III.1.2	Identifikasi Masalah.....	35
III.1.3	Kebutuhan Sistem	37
III.1.3.1	Kebutuhan Fungsional Tambahan.....	37
III.1.3.2	Kebutuhan Nonfungsional.....	38
III.1.3.3	Pemetaan Masalah terhadap Kebutuhan Sistem....	39
III.2	Analisis Solusi	39
III.2.1	Analisis Delegasi Akses dan Atenuasi <i>Offline</i>	40

III.2.2	Analisis Pemilihan Token Otorisasi	42
III.2.3	Analisis Segmentasi Data RME	44
III.2.4	Analisis Penegakan Otoritas pada <i>Server</i> PRE	46
III.2.4.1	Analisis Verifikasi Aktor dan Operasi Tulis	47
III.2.4.2	Analisis Verifikasi Bukti Identitas Aktor Aktif.....	47
III.2.4.3	Analisis Revokasi dan Audit Delegasi	48
III.2.5	Pemetaan Masalah terhadap Solusi	48
III.3	Rancangan Solusi	49
III.3.1	Rancangan Arsitektur Solusi	49
III.3.2	Rancangan Segmentasi Data RME.....	50
III.3.3	Rancangan Token Otorisasi Berbasis Macaroons	52
III.3.4	Rancangan Penegakan Otoritas.....	56
III.3.4.1	Verifikasi <i>Caveats</i> Macaroons.....	57
III.3.4.2	Verifikasi <i>Role</i> dan <i>Subrole</i> Tenaga Medis untuk Akses <i>Dataset</i>	59
III.3.4.3	Verifikasi <i>Wallet</i> Aktor Aktif	60
III.3.4.4	Pencabutan Hak Akses Macaroon Delegasi	61
III.3.5	Rancangan Alur Akses dan Delegasi Macaroon.....	62
III.3.5.1	Penerbitan Token Awal oleh Pasien kepada Petugas Rekam Medis	63
III.3.5.2	Delegasi dari Petugas Rekam Medis kepada Perawat dan Dokter	66
III.3.5.3	Delegasi dari Dokter kepada Laboratorium dan Apoteker	66
III.3.5.4	Proses Pemberian dan Penggunaan Token Macaroon Delegasi	67
III.3.6	Pemetaan Rancangan terhadap Solusi.....	71
BAB IV	IMPLEMENTASI DAN PENGUJIAN.....	72
IV.1	Lingkungan Implementasi	72
IV.2	Implementasi Sistem.....	73
IV.2.1	Batasan Implementasi.....	74
IV.2.2	Implementasi Arsitektur Sistem	75
IV.2.3	Implementasi Segmentasi Data RME	76

IV.2.3.1	Implementasi Segmentasi Metadata RME pada Penyimpanan <i>On-Chain</i>	77
IV.2.3.2	Implementasi Segmentasi Data RME pada Penyimpanan <i>Off-Chain</i>	79
IV.2.4	Implementasi Token Otorisasi Berbasis Macaroons.....	80
IV.2.4.1	Komponen Implementasi Macaroons	80
IV.2.4.2	Struktur <i>Caveats</i> Macaroons	81
IV.2.4.3	Penerbitan Token Macaroon	81
IV.2.4.4	Atenuasi Token Macaroon	82
IV.2.4.5	Revokasi Token Macaroon	82
IV.2.5	Delegasi Akses dan Atenuasi <i>Offline</i>	83
IV.2.5.1	Delegasi Akses dari Pasien kepada Petugas Rekam Medis	83
IV.2.5.2	Delegasi Akses antar Tenaga Kesehatan.....	86
IV.2.6	Implementasi Penegakan Otoritas	89
IV.2.6.1	Autentikasi Token Macaroon.....	89
IV.2.6.2	Pemeriksaan Status Revokasi Token.....	91
IV.2.6.3	Verifikasi <i>Role</i> dan <i>Subrole</i> melalui <i>Move Smart Contract</i>	92
IV.2.6.4	Verifikasi Konteks Segmen.....	92
IV.2.6.5	Verifikasi Bukti Kepemilikan <i>Wallet</i>	93
IV.2.6.6	Eksekusi Operasi <i>Re-encryption</i> , IPFS, dan IOTA .	93
IV.2.7	Implementasi <i>Smart Contract</i> IOTA Move	95
IV.2.8	Implementasi <i>Patient Client</i>	96
IV.2.8.1	Halaman <i>Home</i> (<i>List RME</i>)	96
IV.2.8.2	Halaman Detail RME.....	96
IV.2.8.3	Halaman Audit Delegasi	97
IV.2.8.4	Halaman Detail <i>Scan QR</i>	97
IV.2.9	Implementasi <i>Hospital Client</i>	98
IV.2.9.1	Halaman <i>Dashboard</i> (<i>List RME</i>)	98
IV.2.9.2	Halaman Delegasi Akses	98
IV.2.9.3	Halaman Daftar RME Pasien	99
IV.2.9.4	Halaman Baca Detail RME	99
IV.2.9.5	Halaman Tulis Segmen RME	100

IV.3 Pengujian Sistem	100
IV.3.1 Strategi Pengujian	100
IV.3.2 Pengujian Kebutuhan Fungsional	101
IV.3.3 Pengujian Kebutuhan Nonfungsional	102
IV.3.3.1 Pengujian KNF-TA-01	103
IV.3.3.2 Pengujian KNF-TA-02	104
IV.3.3.3 Pengujian KNF-TA-03	106
IV.3.3.4 Pengujian KNF-TA-04	108
IV.3.3.5 Pengujian KNF-TA-05	108
IV.3.3.6 Pengujian KNF-TA-06	109
IV.4 Evaluasi.....	110
IV.4.1 Evaluasi terhadap Kebutuhan Sistem	111
BAB V PENUTUP.....	113
V.1 Kesimpulan.....	113
V.2 Saran.....	113
DAFTAR REFERENSI	119

DAFTAR LAMPIRAN

Lampiran A.	Metadata Rekam Medis Elektronik	120
Lampiran B.	Alur Pemberian Akses Medis pada DecMed	123
Lampiran C.	Kebutuhan Fungsional DecMed	124
Lampiran D.	DFD DecMed	126
Lampiran E.	Analisis STRIDE pada Sistem DecMed	130
Lampiran F.	Legenda Diagram Arsitektur Solusi	132
Lampiran G.	Skema Penyimpanan Data RME	133
Lampiran H.	Contoh <i>Caveat</i> Token Macaroon untuk Setiap Tahapan	136
Lampiran I.	Daftar Dependensi DecMed	140
Lampiran J.	Legenda Package Diagram	143
Lampiran K.	Modul dan Implementasi <i>Crate</i> <code>decmed-rme-segment</code>	144
Lampiran L.	Modul dan Implementasi <i>Crate</i> <code>decmed-macaroon-auth</code>	145
Lampiran M.	Alur Implementasi Penegakan Otoritas pada Server PRE	146
Lampiran N.	Implementasi REST API Server PRE	149
Lampiran O.	Implementasi Smart Contract Sistem DecMed	150
Lampiran P.	Tangkapan Layar Implementasi <i>Patient Client</i>	154
Lampiran Q.	Tangkapan Layar Implementasi <i>Hospital Client</i>	157

DAFTAR GAMBAR

Gambar II.1. Gambaran simulasi dari PRE (Nuñez, 2017).....	12
Gambar II.2. <i>Sequence diagram</i> dari model CapBAC. (Dokumentasi penulis).....	15
Gambar II.3. Contoh delegasi dan atenuasi kewenangan pada Macaroon. C memegang Macaroon yang diturunkan dari <i>service</i> FS untuk mengakses data pada <i>service</i> TS dengan batasan waktu, lokasi IP, dan target data yang spesifik. (Birgisson dkk., 2014).....	18
Gambar II.4. Arsitektur DecMed (Sadewa, 2025).....	19
Gambar II.5. Ilustrasi struktur data DMR menggunakan composable NFT. (Amaracitra & Alamsyah, 2025).....	24
Gambar III.1. Arsitektur Solusi (Dokumentasi Penulis).....	51
Gambar III.2. Alur penegakan otoritas pada <i>Server</i> PRE.....	58
Gambar III.3. <i>Sequence Diagram</i> Rancangan Validasi <i>subrole</i> Tenaga Medis terhadap <i>Dataset Category</i>	60
Gambar III.4. <i>Sequence Diagram</i> Rancangan Alur Pencabutan Hak Akses Macaroon Delegasi.....	62
Gambar III.5. <i>Flow Diagram</i> Alur Akses dan Delegasi pada Proses Pelayanan Pasien.....	64
Gambar III.6. <i>Sequence diagram</i> pembentukan dan pencatatan token Macaroon delegasi	68
Gambar III.7. <i>Sequence diagram</i> pengambilan dan penggunaan token Macaroon delegasi	69
Gambar IV.1. Package Diagram Implementasi	76
Gambar B.1. <i>Sequence Diagram</i> Alur Pemberian Akses Medis dari Pasien kepada Tenaga Medis (Sadewa, 2025)	123
Gambar D.1. Legenda <i>Data Flow Diagram</i> (Dokumentasi Penulis)	126
Gambar D.2. DFD DecMed Level 0 (Dokumentasi Penulis).....	126
Gambar D.3. DFD DecMed Level 1 (Dokumentasi Penulis).....	127

Gambar F.1. Legenda Diagram Arsitektur Solusi (Dokumentasi Penulis)	132
Gambar J.1. Legenda Package Diagram (Dokumentasi Penulis)	143
Gambar M.1. Legenda bentuk pada <i>Flow Diagram</i>	146
Gambar M.2. Alur implementasi penegakan otoritas pada <i>Server</i> PRE bagian autentikasi token, <i>effective capability</i> , <i>delegation chain</i> , dan revokasi...	147
Gambar M.3. Alur implementasi penegakan otoritas pada <i>Server</i> PRE bagian validasi <i>role</i> dan <i>subrole</i> , validasi segmen, <i>wallet proof</i> , serta eksekusi PRE, IPFS, dan IOTA.....	148
Gambar P.1. Halaman <i>Home</i> daftar RME.	154
Gambar P.2. Halaman detail RME.....	155
Gambar P.3. Halaman audit delegasi RME.....	155
Gambar P.4. Halaman <i>scan</i> QR.	156
Gambar P.5. Halaman konfirmasi pemberian akses awal.	156
Gambar Q.1. Halaman <i>dashboard MedicalPersonnel</i>	157
Gambar Q.2. Halaman delegasi akses bagian 1.....	157
Gambar Q.3. Halaman delegasi akses bagian 2.....	158
Gambar Q.4. Halaman daftar RME pasien pada <i>Hospital Client</i>	158
Gambar Q.5. Halaman detail RME pada <i>Hospital Client</i>	159
Gambar Q.6. Halaman alur akses tulis pada <i>Hospital Client</i>	159

DAFTAR TABEL

Tabel II.1.	Komponen dan fungsi dalam arsitektur DecMed	20
Tabel II.2.	Aktor, <i>client</i> , dan peran dalam DecMed	20
Tabel II.3.	Kategori ancaman pada STRIDE.....	21
Tabel III.1.	Pemetaan Aktor dan Kebutuhan Akses pada Skenario Layanan Kesehatan Kolaboratif	32
Tabel III.2.	Identifikasi Masalah DecMed	36
Tabel III.3.	Kebutuhan Fungsional Tambahan DecMed	38
Tabel III.4.	Kebutuhan Nonfungsional Tambahan DecMed	39
Tabel III.5.	Pemetaan Masalah terhadap Kebutuhan Sistem.....	39
Tabel III.6.	Perbandingan Pendekatan Token Otorisasi	43
Tabel III.7.	Daftar Solusi yang Diusulkan Berdasarkan Hasil Analisis.....	49
Tabel III.8.	Tabel Pemetaan <i>Dataset Category</i>	51
Tabel III.9.	Tabel Pemetaan <i>Function Category</i>	52
Tabel III.10.	Rancangan <i>Caveats</i> pada Token Macaroon	56
Tabel III.11.	Pemetaan Hak Akses Update Berdasarkan subrole Tenaga Medis.....	59
Tabel III.12.	Pemetaan Delegasi Token dari Petugas Rekam Medis.....	66
Tabel III.13.	Pemetaan Delegasi Lanjutan dari Dokter	67
Tabel III.14.	Pemetaan Rancangan terhadap Solusi	71
Tabel IV.1.	Daftar Implementasi Sistem	74
Tabel IV.2.	<i>Caveats</i> yang Diimplementasikan pada Token Macaroon.....	81
Tabel IV.3.	Status Implementasi Penegakan Otoritas pada <i>Server PRE</i>	90
Tabel IV.4.	Pengujian Kebutuhan Fungsional pada Alur Rawat Jalan dan Rawat Inap.....	101
Tabel IV.5.	Evaluasi Kebutuhan Sistem terhadap Implementasi dan Pengujian	111
Tabel A.1.	Metadata Variabel RME Rawat Jalan.....	120
Tabel A.2.	Metadata Variabel RME Rawat Inap.....	121

Tabel A.3.	Metadata Variabel RME Laboratorium	122
Tabel A.4.	Metadata Variabel RME Apotek	122
Tabel C.1.	Kebutuhan Fungsional DecMed (Sadewa, 2025)	124
Tabel D.1.	Daftar <i>External Entities</i> pada DFD DecMed.....	128
Tabel D.2.	Daftar <i>Process</i> DFD DecMed	128
Tabel D.3.	Daftar <i>Data Store</i> pada DFD DecMed.....	128
Tabel D.4.	Daftar Data Flow pada DFD DecMed (Level 0)	129
Tabel D.5.	Daftar Data Flow pada DFD DecMed (Level 1)	129
Tabel E.1.	Analisis STRIDE pada Sistem DecMed	130
Tabel G.1.	Skema Metadata Segmen RME <i>On-Chain</i>	133
Tabel G.2.	Skema Segmen Data RME <i>Off-Chain</i>	134
Tabel I.1.	Dependensi Umum yang Digunakan pada Beberapa Komponen....	140
Tabel I.2.	Dependensi PRE.....	141
Tabel I.3.	Dependensi <i>Client</i>	141
Tabel I.4.	Dependensi <i>decmed-rme-segment</i>	142
Tabel I.5.	Dependensi <i>decmed-macaroon-auth</i>	142
Tabel K.1.	Daftar Modul dan Implementasi <i>Crate</i> <i>decmed-rme-segment</i>	144
Tabel L.1.	Daftar Modul dan Implementasi <i>Crate</i> <i>decmed-macaroon-auth</i> .	145
Tabel N.1.	Daftar Endpoint REST API pada Server PRE	149
Tabel O.1.	Daftar Module Smart Contract Sistem DecMed	150
Tabel O.2.	Daftar Method pada Module <i>decmed::admin</i>	150
Tabel O.3.	Daftar Method pada Module <i>decmed::hospital_personnel</i>	151
Tabel O.4.	Daftar Method pada Module <i>decmed::patient</i>	152
Tabel O.5.	Daftar Method pada Module <i>decmed::proxy</i>	153

DAFTAR KODE PROGRAM

III.1	Struktur klaim JWT pada DecMed.....	28
III.2	Struktur metadata medis pasien pada DecMed	29
III.3	Struktur serialisasi metadata medis DecMed	30
III.4	Potongan struktur data medis <i>off-chain</i> pada DecMed	30
IV.1	Implementasi DatasetCategory pada <i>crate</i> decmed-rme-segment	76
IV.2	Implementasi FunctionCategory pada <i>crate</i> decmed-rme-segment	76
IV.3	Implementasi struktur RmeSegmentMetadata pada <i>crate</i> decmed-rme-segment	78
IV.4	Implementasi struktur RmeSegmentData pada <i>crate</i> decmed-rme-segment	79
IV.5	Definisi DelegationProofContext	87
IV.6	Definisi EffectiveCapability	90
IV.7	Definisi DelegationStep dan DelegationChain.....	91
IV.8	Struktur WalletProofContext sebagai konteks verifikasi <i>wallet</i> <i>signature</i>	93
IV.9	Pengujian PU-TA-01 untuk KNF-TA-01	103
IV.10	Pengujian PU-TA-02 untuk KNF-TA-02	104
IV.11	Pengujian PU-TA-03 untuk KNF-TA-02	105
IV.12	Pengujian PU-TA-04 untuk KNF-TA-02	105
IV.13	Pengujian PU-TA-05 untuk KNF-TA-03	106
IV.14	Pengujian PU-TA-06 untuk KNF-TA-03	107
IV.15	Pengujian PU-TA-07 untuk KNF-TA-03 dan KNF-TA-06	107
IV.16	Pengujian PU-TA-08 untuk KNF-TA-05	108
IV.17	Pengujian PU-TA-09 untuk KNF-TA-06	109
H.1	Contoh <i>Caveat</i> Token Read Awal untuk Petugas Rekam Medis	136
H.2	Contoh <i>Caveat</i> Token Update Awal untuk Petugas Rekam Medis....	136
H.3	Contoh <i>Caveat</i> Delegasi Petugas Rekam Medis → Dokter.....	137
H.4	Contoh <i>Caveat</i> Delegasi Petugas Rekam Medis → Perawat	138
H.5	Contoh <i>Caveat</i> Delegasi Dokter → Petugas Laboratorium	138
H.6	Contoh <i>Caveat</i> Delegasi Dokter → Apoteker	139

BAB I

PENDAHULUAN

I.1 Latar Belakang

Transformasi sistem rekam medis di Indonesia semakin didorong oleh regulasi pemerintah. Peraturan Menteri Kesehatan Republik Indonesia (Permenkes) Nomor 24 Tahun 2022 mewajibkan fasilitas pelayanan kesehatan (fasyankes) untuk mengimplementasikan Rekam Medis Elektronik (RME) yang terintegrasi (Kementerian Kesehatan Republik Indonesia, 2022b). Integrasi RME diperlukan untuk mendukung interoperabilitas data antar fasyankes, sesuai amanat Undang-Undang Nomor 29 Tahun 2004 (Republik Indonesia, 2004). Namun, integrasi tersebut juga menimbulkan kebutuhan perlindungan data yang lebih kuat karena RME merupakan informasi pribadi spesifik yang sensitif (UU 27/2022 tentang PDP, 2022).

Salah satu tantangan penting dalam pengembangan sistem RME adalah aspek kepemilikan data (Liu & Chen, 2025). Kepemilikan data berkaitan dengan kendali pasien terhadap informasi kesehatan miliknya, termasuk siapa yang dapat mengakses data tersebut dan dalam kondisi apa akses dapat diberikan. Sejumlah penelitian menekankan pentingnya pergeseran paradigma kontrol akses menuju sistem yang bersifat *patient-centric*, yaitu sistem yang memberikan kendali lebih besar kepada pasien sebagai pemilik data kesehatan (Patil, 2024; Peng dkk., 2023).

Meskipun kontrol pasien merupakan aspek penting, perawatan pasien di rumah sakit bersifat kolaboratif dan melibatkan banyak tenaga kesehatan lintas unit, seperti dokter, perawat, petugas laboratorium, apoteker, dan tenaga kesehatan lain yang bekerja sesuai kewenangannya (Kementerian Kesehatan Republik Indonesia, 2022a; Ramaswamy dkk., 2023). Dalam konteks tersebut, informasi klinis yang relevan perlu tersedia bagi tenaga kesehatan yang terlibat agar proses pelayanan dapat berjalan efektif (Ramaswamy dkk., 2023). NHS England menyatakan bahwa informasi yang relevan dapat dibagikan antar tenaga kesehatan yang terlibat dalam perawatan pasien berdasarkan *implied consent*, dengan tetap memperhatikan prinsip

need-to-know (NHS England, 2023, Maret 28/2025). Prinsip serupa juga tercermin pada konsep *minimum necessary* dalam HIPAA, yaitu bahwa penggunaan dan pengungkapan informasi kesehatan perlu dibatasi pada informasi minimum yang diperlukan untuk tujuan tertentu (of Health & Services, 2009).

Kondisi tersebut menunjukkan adanya kebutuhan untuk menyeimbangkan kendali pasien dengan kebutuhan operasional klinis. Studi terkait juga menunjukkan bahwa pasien pada umumnya mendukung pembagian data klinis untuk tujuan perawatan langsung tanpa persetujuan eksplisit pada setiap akses, selama pasien memperoleh informasi yang jelas dan tetap memiliki pilihan untuk *opt-out* (L. A. Jones dkk., 2022).

Kebutuhan tersebut sejalan dengan ISO 22600 yang menempatkan pengelolaan *privilege* dan kontrol akses yang granular sebagai bagian penting dalam pertukaran informasi kesehatan, khususnya pada lingkungan yang melibatkan banyak aktor, banyak sistem, dan beberapa domain kebijakan (for Standardization, 2014). Standar tersebut menunjukkan bahwa akses pada sistem kesehatan perlu dikelola berbasis *policy*, sejalan dengan prinsip *privilege management* pada sistem informasi kesehatan (for Standardization, 2014).

Salah satu solusi sistem RME terdesentralisasi adalah DecMed (Sadewa, 2025). Sistem ini memanfaatkan DLT IOTA, *InterPlanetary File System* (IPFS), dan *Proxy Re-Encryption* (PRE) dengan menerapkan *capability-based access control*. Namun, mekanisme pemberian akses pada DecMed masih bertumpu pada pasien sebagai *resource owner* untuk menerbitkan token hak akses baru. Dalam konteks layanan klinis yang kolaboratif, asumsi tersebut dapat menjadi hambatan karena kebutuhan akses sering kali muncul secara dinamis sesuai peran tenaga kesehatan dalam episode perawatan. Selain itu, kontrol akses granular tidak cukup hanya direpresentasikan pada token hak akses. Data RME yang dilindungi juga masih disimpan sebagai satu *entry* besar dan perlu dipisahkan ke dalam unit yang lebih kecil agar data dapat diakses sesuai dengan kebutuhan layanan. Dengan demikian, terdapat dua keterbatasan utama yang perlu diperbaiki, yaitu mekanisme delegasi akses yang masih bergantung pada keterlibatan langsung pasien dan struktur data RME yang perlu mendukung pemisahan informasi medis sesuai prinsip *need-to-know*.

Untuk menjawab keterbatasan tersebut, Tugas Akhir ini memperbarui mekanisme kontrol akses granular pada DecMed melalui integrasi Macaroons, segmentasi data medis, dan fitur *offline attenuation*. Fitur *offline attenuation* memungkinkan pasien menerbitkan token awal pada proses administrasi, lalu pemegang token menurunkan hak akses kepada tenaga kesehatan yang terlibat tanpa intervensi ulang dengan pasien. Integrasi ini diarahkan untuk memperbaiki mekanisme delegasi akses pada DecMed sebelumnya dan kontrol akses yang granular dengan segmentasi data medis untuk mendukung penerapan prinsip *need-to-know*.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan pada subbab I.1, rumusan masalah yang dibahas dalam tugas akhir ini adalah sebagai berikut:

1. Bagaimana rancangan dan implementasi mekanisme kontrol akses granular pada kerangka kerja DecMed dengan memanfaatkan Macaroons?
2. Bagaimana rancangan dan implementasi segmentasi data RME yang mendukung penegakan kontrol akses granular pada DecMed?
3. Bagaimana rancangan dan implementasi fitur *offline attenuation* pada token otorisasi untuk memfasilitasi pendelegasian hak akses antar tenaga kesehatan yang terlibat dalam layanan klinis tanpa memerlukan intervensi langsung dari pasien?

I.3 Tujuan

Tujuan dari Tugas Akhir ini adalah:

1. Merancang dan menerapkan kontrol akses granular pada kerangka kerja DecMed dengan memanfaatkan teknologi Macaroons.
2. Merancang dan menerapkan segmentasi data yang mendukung penegakan kontrol akses granular pada DecMed.
3. Merancang dan menerapkan fitur atenuasi *offline* untuk memfasilitasi pendelegasian hak akses antar tenaga medis tanpa memerlukan intervensi langsung dari pasien.

I.4 Batasan Masalah

Terdapat beberapa batasan masalah dari Tugas Akhir ini. Batasan masalah tersebut antara lain:

1. Kasus skenario delegasi layanan klinis terbatas hanya pada aktor dokter, perawat, petugas laboratorium, dan apoteker pada alur Rawat Jalan dan Rawat Inap.
2. Data RME pada dataset Instalasi Gawat Darurat (IGD) tidak termasuk dalam kajian rancangan dan implementasi.
3. Segmentasi data RME hanya dilakukan untuk dokumentasi klinis. Dokumentasi administratif tidak termasuk ke dalam fokus Tugas Akhir ini dan hanya diimplementasikan sesuai dengan implementasi DecMed sebelumnya.

I.5 Metodologi

Terdapat beberapa tahapan untuk melaksanakan tugas akhir secara sistematis, berikut merupakan tahapan-tahapan yang dilakukan:

1. Studi Literatur

Tahap awal berfokus pada pengumpulan fondasi teoretis dan kontekstual melalui studi literatur.

2. Analisis Kebutuhan

Tahap ini membahas analisis untuk mendefinisikan kekurangan pada kerangka kerja DecMed. Kekurangan tersebut diinterpretasikan sebagai masalah dan didefinisikan menjadi kebutuhan. Tahap ini diakhiri dengan perumusan kebutuhan fungsional dan nonfungsional sistem.

3. Analisis dan Rancangan Solusi

Tahap ini membahas alternatif solusi yang dapat digunakan untuk mengatasi masalah pada kerangka kerja DecMed. Solusi yang terpilih kemudian direncanakan menjadi rancangan solusi.

4. Implementasi Prototipe

Tahap ini merupakan realisasi dari rancangan ke dalam bentuk pengembangan dari prototipe DecMed yang sudah ada.

5. Pengujian

Tahap ini merupakan tahapan untuk menguji prototipe dapat memenuhi kebutuhan sistem sesuai dengan skenario pengujian yang ditentukan.

6. Evaluasi

Tahap ini merupakan tahapan untuk mengevaluasi prototipe berdasarkan hasil pengujian, yang menjadi dasar dari kesimpulan dan memberikan saran untuk penelitian selanjutnya.

I.6 Sistematika Pembahasan

Berikut merupakan sistematika pembahasan dari Tugas Akhir ini:

Bab I - Pendahuluan Bab ini menjelaskan gagasan utama dari tugas akhir, yang meliputi latar belakang, rumusan masalah, tujuan, batasan, metodologi, hingga sistematika pembahasan mengenai proses pengembangan solusi.

Bab II - Studi Literatur Bab ini memaparkan studi literatur yang telah dilakukan terkait sistem yang dikembangkan. Pembahasan difokuskan pada uraian teoritis berbagai komponen yang digunakan, termasuk regulasi dan teknologi yang digunakan dalam sistem.

Bab III - Analisis dan Rancangan Solusi Bab ini dijelaskan analisis terhadap masalah yang ditemukan serta berbagai alternatif solusi yang dapat digunakan untuk mengatasinya. Selain itu, bab ini juga membahas rancangan solusi yang dikembangkan, meliputi pendefinisian kebutuhan fungsional dan nonfungsional, serta rancangan setiap komponen sistem.

Bab IV - Implementasi dan Pengujian Sistem Bab ini menjelaskan hasil implementasi dari rancangan solusi yang telah dipaparkan pada Bab III. Selain itu, strategi pengujian serta hasil pengujian yang dilakukan terhadap solusi juga dibahas pada bab ini.

Bab V - Penutup Bab ini berisi kesimpulan dari hasil penelitian serta saran untuk pengembangan lebih lanjut.

BAB II

STUDI LITERATUR

II.1 Rekam Medis Elektronik

II.1.1 Definisi RME

Di Indonesia, definisi dari Rekam Medis dan Rekam Medis Elektronik didefinisikan pada Peraturan Menteri Kesehatan Republik Indonesia Nomor 24 Tahun 2022 (Permenkes 24/2022). "Rekam Medis adalah dokumen yang berisikan identitas pasien, pemeriksaan, pengobatan, tindakan, dan pelayanan lain yang telah diberikan pada pasien" sedangkan "Rekam Medis Elektronik adalah Rekam Medis yang dibuat dengan menggunakan sistem elektronik yang diperuntukkan bagi penyelenggaraan Rekam Medis." (Kementerian Kesehatan Republik Indonesia, 2022b).

Berdasarkan definisi tersebut, rekam medis adalah salah satu komponen vital dalam sistem kesehatan di Indonesia. Rekam medis bukanlah sebuah catatan biasa yang dibuat lalu disimpan, melainkan sebuah catatan yang terus diperbarui. Keberadaan Rekam Medis bertujuan untuk meningkatkan mutu pelayanan kesehatan dan memberikan kepastian hukum dalam peristiwa medis.

II.1.2 Isi RME

Berdasarkan Permenkes 24/2022 Pasal 27 Ayat 1 (Kementerian Kesehatan Republik Indonesia, 2022b), isi dari Rekam Medis Elektronik terbagi menjadi dua bagian, yaitu:

1. Dokumentasi Administratif
2. Dokumentasi Klinis

Panduan lebih rinci terkait isi dari Rekam Medis Elektronik diatur pada Keputusan Menteri Kesehatan (KMK) No. HK.01.07/MENKES/1423/2022 tentang Pedoman Variabel dan metadata pada Penyelenggaraan Rekam Medis Elektronik (Menteri

Kesehatan Republik Indonesia, 2022). Pada setiap data RME, terdapat dokumentasi administratif dan dokumentasi klinis. Dokumentasi Klinis kemudian dibagi berdasarkan kategori fungsi klinis. KMK ini mengatur variabel dan metadata yang terdapat pada RME dengan dataset yang terdiri dari:

1. IGD (Instalasi Gawat Darurat).
2. Rawat Jalan.
3. Rawat Inap.
4. Laboratorium.
5. Apotek.

Detail metadata dataset RME untuk dataset Rawat Jalan, Rawat Inap, Laboratorium, dan Apotek dapat dilihat pada Lampiran A.

II.1.3 RME dalam Alur Pelayanan Kesehatan Kolaboratif

Penyelenggaraan layanan kesehatan pada praktiknya tidak selalu dilakukan oleh satu aktor tunggal, melainkan melibatkan beberapa tenaga kesehatan dan tenaga lain yang bekerja sesuai kewenangan masing-masing (World Health Organization, 2010). Dalam konteks layanan kolaboratif, pasien dapat berinteraksi dengan petugas rekam medis pada tahap pendaftaran, perawat pada tahap pemeriksaan awal, dokter pada tahap pemeriksaan dan pengambilan keputusan klinis, petugas laboratorium pada tahap pemeriksaan penunjang, serta apoteker atau tenaga kefarmasian pada tahap pelayanan resep dan obat. Pola ini sejalan dengan konsep praktik kolaboratif antarprofesi, yaitu ketika tenaga kesehatan dari latar belakang profesi berbeda bekerja bersama pasien untuk memberikan pelayanan yang bermutu (Ramaswamy dkk., 2023).

Struktur RME di Indonesia juga menunjukkan bahwa data kesehatan pasien tidak bersifat tunggal, melainkan terdiri dari dokumentasi administratif dan dokumentasi klinis (Kementerian Kesehatan Republik Indonesia, 2022b). Pembagian ini menunjukkan bahwa kebutuhan akses terhadap RME dapat berbeda antara aktor administratif dan aktor klinis.

Standar Akreditasi Rumah Sakit memberikan dasar bahwa pelayanan pasien dilakukan melalui kerja kolaboratif antarprofesi. Dalam standar tersebut, DPJP

(Dokter Penanggung Jawab Pasien) ditempatkan sebagai *clinical leader*, sedangkan PPA (Profesional Pemberi Asuhan) bekerja sebagai tim interdisiplin dalam proses asuhan pasien (Kementerian Kesehatan Republik Indonesia, 2022a). Standar yang sama juga menunjukkan keterlibatan perawat dalam pengkajian dan asuhan keperawatan, laboratorium sebagai layanan penunjang untuk memenuhi kebutuhan pasien dan PPA, serta apoteker dalam proses penggunaan obat yang terkoordinasi.

Alur operasional layanan tersebut diperjelas oleh panduan interoperabilitas SATUSEHAT untuk layanan Rawat Jalan dan Rawat Inap. Panduan tersebut memuat tahapan layanan seperti pendaftaran pasien, pendaftaran kunjungan, anamnesis, pemeriksaan, rencana rawat, instruksi medik dan keperawatan, pemeriksaan penunjang, diagnosis, tindakan, peresepan, pengkajian resep, pemberian obat, tindak lanjut, dan resume medis (SATUSEHAT Platform, nodate, 2023). Sementara itu, Pedoman Variabel dan Metadata RME dari Kementerian Kesehatan memetakan dokumentasi RME ke dalam dataset seperti Rawat Jalan, Rawat Inap, Laboratorium, dan Apotek (Menteri Kesehatan Republik Indonesia, 2022). Kombinasi antara alur pelayanan, keterlibatan aktor, dan struktur dataset tersebut menunjukkan bahwa kebutuhan akses terhadap RME berbeda menurut tahap layanan, peran aktor, serta jenis data yang digunakan.

II.1.4 Hak Akses RME

Karena RME memuat data administratif dan klinis yang termasuk data pribadi spesifik, akses terhadap RME harus dibatasi berdasarkan kewenangan, tindakan, dan kebutuhan pelayanan. Tenaga kesehatan dapat melakukan beberapa aksi terhadap data RME pasien, yaitu (Kementerian Kesehatan Republik Indonesia, 2022b):

1. Menambahkan data baru.
2. Melakukan perbaikan data.
3. Melihat atau membaca data.

Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi (UU PDP) menjadi landasan yuridis utama yang menetapkan standar keamanan bagi sistem data rekam medis. Pasal 4 ayat 2 huruf a UU PDP mengklasifikasikan data dan informasi kesehatan sebagai data pribadi spesifik (UU 27/2022 tentang

PDP, 2022). Pengendali data pribadi diwajibkan untuk melindungi dan memastikan keamanan data yang diprosesnya (Pasal 35), mewajibkan pengendali data pribadi untuk menyediakan *audit trail* pemrosesan data pribadi (Pasal 32), wajib melindungi data dari pemrosesan yang tidak sah (Pasal 38) dan mencegah akses yang tidak sah melalui sistem keamanan yang andal (Pasal 39) (UU 27/2022 tentang PDP, 2022).

Berdasarkan Permenkes No. 73 Tahun 2016 tentang Standar Pelayanan Kefarmasian di Apotek, pelayanan farmasi klinik mencakup pengkajian resep dan *dispensing* (Indonesia, 2016). Hal ini menunjukkan bahwa apoteker memerlukan akses terhadap informasi yang relevan dengan resep, terapi, dan penggunaan obat pasien untuk memastikan kesesuaian, keamanan, dan ketepatan pelayanan obat.

Kebutuhan akses pada petugas laboratorium dapat diturunkan dari struktur metadata RME pada KMK 1423/2022 (Menteri Kesehatan Republik Indonesia, 2022). Pada pedoman tersebut, permintaan pemeriksaan laboratorium yang diminta oleh dokter untuk pasien dicantumkan sebagai bagian dari data pemeriksaan penunjang. Hal ini menunjukkan bahwa dalam alur pelayanan, petugas laboratorium tidak perlu memperoleh akses terhadap seluruh data klinis pasien, tetapi cukup terhadap informasi yang relevan untuk memproses pemeriksaan penunjang terkait laboratorium.

II.2 IOTA

II.2.1 Distributed Ledger Technology

Distributed Ledger Technology (DLT) merupakan teknologi pencatatan data terdistribusi yang memungkinkan data direkam, disimpan, dan diverifikasi oleh banyak *node* tanpa bergantung pada satu otoritas terpusat. DLT mencakup berbagai bentuk implementasi, termasuk *blockchain* dan struktur alternatif seperti *Directed Acyclic Graph* (DAG), yang masing-masing memiliki pendekatan berbeda dalam mencapai konsensus, keamanan, dan skalabilitas pada jaringan terdesentralisasi (Savadatti dkk., 2025).

Karakteristik utama DLT meliputi replikasi data pada banyak *node*, ketahanan terhadap perubahan data melalui mekanisme kriptografi, serta kemampuan

verifikasi oleh partisipan jaringan. Karakteristik tersebut membuat DLT banyak digunakan sebagai infrastruktur untuk pencatatan transaksi, penyimpanan referensi data, dan eksekusi aturan terprogram melalui *smart contract* (Savadatti dkk., 2025). Dalam konteks sistem rekam medis elektronik terdesentralisasi, DLT dapat digunakan untuk mendukung pencatatan data yang dapat diverifikasi tanpa menempatkan seluruh kendali pada satu pihak terpusat.

II.2.2 IOTA

IOTA merupakan salah satu implementasi DLT yang dirancang untuk mendukung transaksi pada lingkungan terdesentralisasi, khususnya pada skenario yang membutuhkan skalabilitas tinggi dan biaya transaksi rendah. Berbeda dari *blockchain* konvensional, IOTA menggunakan struktur DAG yang dikenal sebagai *Tangle* sebagai dasar buku besar terdistribusinya (Sedi Nzakuna dkk., 2025). Pada perkembangannya, IOTA mengalami perubahan arsitektur melalui IOTA Rebased. Perubahan ini bertujuan meningkatkan skalabilitas, desentralisasi, dan kemampuan ekosistem IOTA untuk mendukung aplikasi terdesentralisasi, termasuk penggunaan *smart contract* (Sedi Nzakuna dkk., 2025). IOTA menggunakan *Move smart contract* untuk berinteraksi dengan data dan melakukan transaksi.

II.3 InterPlanetary File System (IPFS)

IPFS adalah sebuah protokol *peer-to-peer* dan sistem *file* terdistribusi yang dirancang untuk menyimpan dan membagikan data secara desentralisasi (Benet, 2014). Berbeda dengan sistem penyimpanan tradisional yang bergantung pada *server* pusat, IPFS membangun jaringan global dengan setiap pengguna (*node*) dapat berperan sebagai penyimpan dan penyedia konten. Teknologi ini menggunakan *content addressing* daripada *location addressing*, sehingga setiap *file* diidentifikasi dengan unik berdasarkan hash kriptografi dari konten tersebut. Dengan pendekatan desentralisasi ini, IPFS menawarkan solusi yang lebih tahan terhadap sensor, lebih andal, dan tidak bergantung pada satu titik kegagalan pusat.

IPFS beroperasi melalui mekanisme yang mengintegrasikan beberapa komponen kunci. Pertama, menggunakan *Distributed Hash Table* (DHT) yang memungkinkan setiap *file* diberi identitas unik berdasarkan hash konten (*Content Identifier/CID*),

memungkinkan *file* diakses dari *node* terdekat yang menyimpannya daripada harus mengakses *server* pusat yang jauh. Kedua, IPFS menerapkan penyimpanan terdistribusi dengan membagi *file* ke dalam blok-blok kecil yang tersebar di berbagai *node* dalam jaringan, sehingga meningkatkan kecepatan akses dan ketahanan terhadap kegagalan. Ketiga, teknologi ini menggunakan sistem *caching* lokal pada setiap *node* yang menyimpan salinan konten yang sering diakses, memungkinkan akses lebih cepat tanpa perlu mengunduh ulang dari internet. Keempat, IPFS menggunakan protokol komunikasi *peer-to-peer* yang memungkinkan setiap *node* bertindak sebagai klien, *server*, atau *node* perantara, sehingga memfasilitasi komunikasi langsung antar *node* tanpa perantara *server* pusat (Benet, 2014).

II.4 *Proxy Re-Encryption* (PRE)

Proxy Re-Encryption adalah skema kriptografi yang memungkinkan pihak ketiga (disebut proxy) mengubah suatu *ciphertext* yang awalnya dienkripsi untuk satu pengguna menjadi *ciphertext* baru yang bisa didekripsi oleh pengguna lain, tanpa pernah membuka *plaintext* dan tanpa memegang kunci privat kedua pihak. Teknik ini dipakai untuk mendelegasikan hak dekripsi secara aman, misalnya pada skenario berbagi data terenkripsi di *cloud*. PRE pertama kali diperkenalkan oleh Blaze et al. (Blaze dkk., 1998) sebagai solusi untuk mendelegasikan hak dekripsi secara aman.

Gambaran simulasi dari PRE dapat dilihat pada Gambar II.1. Dalam PRE, pemilik data (misalnya Alice) mengenkripsi datanya dengan kunci publik miliknya dan menyimpannya di server atau cloud. Ketika Alice ingin memberi akses ke Bob, ia membuat *re-encryption key* dari kunci miliknya dan kunci Bob, lalu memberikannya ke proxy agar proxy bisa mengubah *ciphertext* milik Alice menjadi *ciphertext* yang dapat didekripsi Bob, tanpa pernah melihat isi datanya.

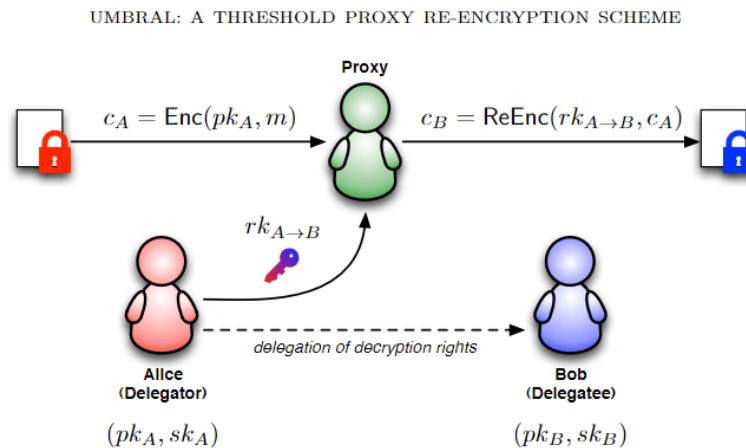
II.4.1 Umbral PRE

Umbral adalah skema *threshold proxy re-encryption* yang memungkinkan pemilik data mendelegasikan hak dekripsi kepada penerima akses melalui satu atau beberapa proxy *semi-trusted* (Nuñez, 2017). Dalam Umbral, pihak yang memberikan akses disebut *delegator*, pihak yang menerima akses disebut *delegatee*,

sedangkan proxy bertugas melakukan re-enkripsi tanpa mengetahui isi data. Berbeda dengan PRE sederhana yang dapat menggunakan satu *re-encryption key*, Umbral membagi kemampuan re-enkripsi ke dalam beberapa *key fragments* atau kfrag. Dengan pendekatan ini, proses re-enkripsi dapat dibuat berbasis ambang batas, misalnya minimal t dari total n fragmen harus digunakan agar akses dekripsi dapat berhasil.

Pada tahap awal, data dienkripsi menggunakan kunci publik milik *delegator*. Hasil enkripsi pada Umbral terdiri atas *ciphertext* dan *capsule*. *Ciphertext* berisi data terenkripsi, sedangkan *capsule* memuat informasi kriptografis yang diperlukan untuk membuka kunci enkripsi data. Ketika *delegator* ingin memberi akses kepada *delegatee*, *delegator* membuat sejumlah kfrag berdasarkan kunci privat miliknya, kunci publik milik *delegatee*, dan parameter ambang batas yang ditentukan. Fragmen tersebut kemudian diberikan kepada proxy agar proxy dapat melakukan re-enkripsi terhadap *capsule*.

Ketika *delegatee* ingin mengakses data, *delegatee* mengambil *ciphertext* dan *capsule*, lalu meminta proxy menghasilkan *capsule fragment* atau cfrag dari kfrag yang dimiliki proxy. Proxy hanya melakukan transformasi kriptografis terhadap *capsule*; proxy tidak mendekripsi *ciphertext*, tidak mengetahui *plaintext*, dan tidak membutuhkan kunci privat *delegator* maupun *delegatee*. Setelah *delegatee* memperoleh jumlah cfrag valid yang memenuhi ambang batas t , *delegatee* dapat membuka *capsule* menggunakan kunci privatnya, memperoleh kunci simetris, lalu mendekripsi *ciphertext*. Dengan demikian, Umbral memungkinkan delegasi hak



Gambar II.1: Gambaran simulasi dari PRE (Nuñez, 2017)

dekripsi tanpa membagikan kunci privat dan tanpa memberikan akses langsung terhadap isi data kepada proxy.

II.5 Kontrol Akses

II.5.1 Definisi Kontrol Akses

Kontrol Akses adalah proses pengaturan akses ke sumber daya (data, sistem informasi, perangkat, atau area tertentu) (Li dkk., 2022; Sandhu & Samarati, 1994). Proses ini memutuskan apakah seorang subjek harus diberikan atau ditolak aksesnya berdasarkan aturan tertentu. Akses dapat berarti memakai, memasuki, atau menggunakan. Kontrol Akses bertujuan untuk mencegah aktivitas yang dapat menimbulkan pelanggaran keamanan seperti kebocoran data atau modifikasi tidak sah.

II.5.2 Kontrol Akses Granular

Kontrol akses granular atau *fine-grained access control* adalah pendekatan kontrol akses yang memungkinkan keputusan otorisasi dilakukan pada tingkat cakupan yang lebih spesifik dibandingkan pemberian izin secara umum atau menyeluruh. Jika kontrol akses kasar (*coarse-grained access control*) hanya membedakan akses berdasarkan kategori besar, seperti *role* pengguna atau jenis akses, maka kontrol akses granular mempertimbangkan detail yang lebih banyak, seperti akses berdasarkan pengguna, per baris/kolom data, konteks ketika melakukan akses, waktu berlaku, dan batasan tambahan lain yang relevan dengan kebijakan sistem (Hu dkk., 2014; Odugu & Rajesh, 2021).

Sebuah mekanisme kontrol akses dapat disebut granular apabila sistem mampu membedakan hak akses berdasarkan beberapa dimensi otorisasi secara lebih spesifik. Dimensi tersebut meliputi: (1) subjek yang meminta akses, misalnya identitas pengguna, peran, atau atribut; (2) objek atau sumber daya yang diakses, misalnya pengguna tertentu, sumber data tertentu, *dataset* tertentu, atau bagian tertentu dari data; (3) operasi yang dilakukan, misalnya membaca, menulis, memperbarui, mendelegasikan, dan menghapus; dan (4) kondisi atau konteks akses, misalnya waktu berlaku, tujuan akses, lokasi, atau kondisi layanan tertentu (Hu

dkk., 2014).

Granularitas dalam kontrol akses juga berkaitan erat dengan prinsip *least privilege*, yaitu pemberian hak akses sebatas yang diperlukan untuk menjalankan tugas tertentu (Joint Task Force Transformation Initiative, 2020). Dengan pendekatan ini, pengguna tidak langsung memperoleh akses penuh terhadap seluruh sumber daya hanya karena memiliki peran tertentu. Sebagai contoh, tenaga medis tidak cukup hanya diberi akses berdasarkan peran umum sebagai “dokter”, melainkan aksesnya dapat dibatasi hanya pada pasien tertentu, episode layanan tertentu, data tertentu, serta operasi tertentu yang sesuai dengan kebutuhan tugasnya. Dengan demikian, kontrol akses granular dapat mengurangi risiko akses berlebihan dan membatasi dampak apabila kredensial atau token akses disalahgunakan.

Pada Tugas Akhir ini, kontrol akses granular dimaknai sebagai kemampuan sistem untuk membatasi hak akses berdasarkan kombinasi antara subjek, data yang diakses, jenis operasi, waktu berlaku, serta batasan delegasi.

II.5.3 Model CapBAC

Capability-Based Access Control (CapBAC) adalah model kontrol akses yang hak aksesnya direpresentasikan sebagai “*capability token*” yang tidak dapat dipalsukan (Anagnostopoulou dkk., 2025; Li dkk., 2022). Token tersebut menyatakan otoritas untuk melakukan operasi tertentu terhadap objek atau sumber daya tertentu. Dalam implementasi tertentu, capability token juga dapat dibatasi dengan identitas subjek, waktu berlaku, cakupan operasi, atau kondisi tambahan lain untuk mempersempit hak akses yang diberikan.

Secara garis besar, model CapBAC bekerja dalam 2 tahapan, (1) Pengguna meminta capability token dari sebuah *Authorization Server*, kemudian (2) Pengguna menggunakan capability token yang diperoleh dari tahap pertama untuk melakukan akses ke *Resource Server* (Li dkk., 2022). *Sequence diagram* dari model CapBAC dapat dilihat pada Gambar II.2. CapBAC memungkinkan otorisasi terdesentralisasi karena mengandalkan token kapabilitas yang tidak dapat dipalsukan sebagai bukti.

II.6 Token Otorisasi

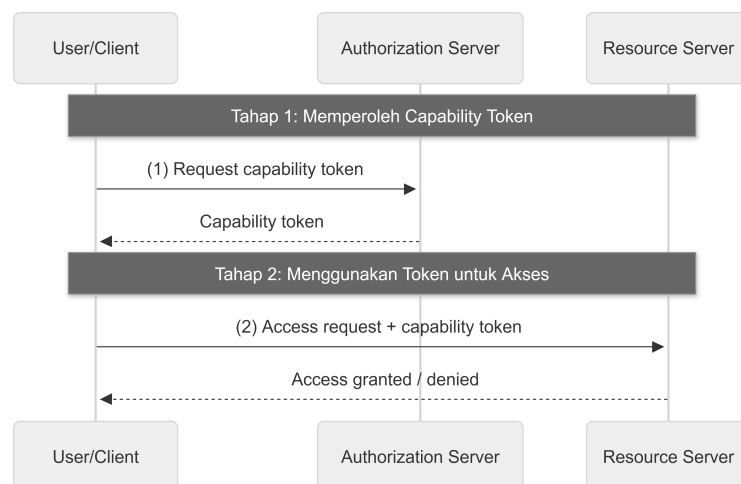
II.6.1 JSON Web Token

JSON Web Token (JWT) adalah format token yang ringkas dan *URL-safe* untuk merepresentasikan klaim yang dipertukarkan antara dua pihak (M. B. Jones dkk., 2015). Klaim pada JWT direpresentasikan dalam bentuk objek JSON dan dapat diamankan menggunakan JSON Web Signature (JWS) atau JSON Web Encryption (JWE). Dengan demikian, JWT dapat digunakan untuk membawa informasi seperti identitas pengguna, peran, cakupan akses, waktu penerbitan, dan waktu kedaluwarsa token.

Secara umum, JWT terdiri dari tiga bagian utama, yaitu *header*, *payload*, dan *signature*. Ketiga bagian tersebut dikodekan menggunakan Base64URL dan dipisahkan dengan tanda titik. Bentuk umum JWT dapat dituliskan sebagai berikut:

$$JWT = header.payload.signature$$

Bagian *header* berisi informasi mengenai tipe token dan algoritma kriptografi yang digunakan. Bagian *payload* berisi klaim yang menyatakan informasi mengenai subjek atau hak akses. Bagian *signature* digunakan untuk memastikan integritas token sehingga perubahan pada *header* atau *payload* dapat terdeteksi oleh pihak yang memverifikasi token.



Gambar II.2: *Sequence diagram* dari model CapBAC. (Dokumentasi penulis)

II.6.2 PASETO

PASETO atau *Platform-Agnostic Security Tokens* adalah format token yang dirancang sebagai alternatif terhadap JWT dengan pendekatan kriptografi yang lebih terarah (Arciszewski & Haussmann, 2022). Seperti JWT, PASETO digunakan untuk merepresentasikan klaim dalam bentuk token yang ringkas dan dapat dipertukarkan antar pihak. Namun, PASETO membatasi pilihan algoritma kriptografi berdasarkan versi dan tujuan penggunaan token, sehingga mengurangi risiko kesalahan konfigurasi algoritma yang dapat terjadi pada token berbasis JWT.

Secara umum, token PASETO P dapat direpresentasikan sebagai:

$$P = version.purpose.message[.footer]$$

version menunjukkan versi PASETO, *purpose* menunjukkan tujuan penggunaan token, *message* adalah *payload* yang telah diproses secara kriptografis, dan *footer* adalah *footer* opsional. Nilai *purpose* dapat berupa *local* atau *public*. Nilai *local* digunakan ketika isi token perlu dienkripsi menggunakan kunci simetris, sedangkan nilai *public* digunakan ketika token perlu ditandatangani menggunakan kriptografi kunci publik.

Keunggulan PASETO terletak pada desainnya yang lebih *opinionated* dibandingkan JWT, terutama dalam pemilihan algoritma kriptografi. Dengan membatasi pilihan algoritma, PASETO berupaya mengurangi fleksibilitas yang berpotensi menimbulkan kesalahan implementasi. Namun, dari sisi model otorisasi, PASETO tetap berperan sebagai token pembawa klaim. Artinya, pembatasan akses seperti peran, cakupan, dan waktu berlaku tetap perlu didefinisikan pada saat token diterbitkan.

II.6.3 Macaroons

Macaroon adalah *credential* otorisasi berbasis *bearer* yang mendukung delegasi terdesentralisasi dalam sistem terdistribusi (Birgisson dkk., 2014). Keunggulan utama Macaroon terletak pada penggunaan *chained-HMAC*, yang memungkinkan proses delegasi bersifat terdesentralisasi secara efisien serta mendukung penambahan syarat *caveat* yang sesuai dengan konteks penggunaan.

Sebuah Macaroon M didefinisikan dengan struktur:

$$M = (id, location, \vec{c}, sig)$$

id adalah pengidentifikasi unik, $location$ adalah alamat layanan target, $\vec{c} = (c_1, c_2, \dots, c_n)$ adalah daftar *caveat*, dan sig adalah tanda tangan (Birgisson dkk., 2014). Tanda tangan (sig) dihitung secara rekursif untuk menjamin integritas setiap penambahan batasan (*caveat*):

$$sig_0 = \text{HMAC}(\text{RootKey}, id)$$

$$sig_n = \text{HMAC}(sig_{n-1}, c_n)$$

RootKey adalah kunci rahasia yang digunakan untuk membangkitkan token Macaroon awal. Struktur ini memungkinkan pemegang Macaroon menambahkan *caveat* baru c_{new} (misalnya: $\text{time} < 24\text{h}$) dan menghasilkan sig_{new} untuk diberikan ke pihak lain, tanpa mengetahui *RootKey* yang asli. Kemampuan untuk menambahkan batasan baru pada token secara lokal tanpa melibatkan penerbit awal ini disebut dengan *offline attenuation*. Mekanisme ini memungkinkan delegasi terdesentralisasi yang aman dan fleksibel. Gambaran mengenai delegasi dan atenuasi kewenangan melalui *caveats* dapat dilihat pada Gambar II.3.

II.6.4 Biscuit Token

Biscuit adalah token otorisasi yang dirancang untuk mendukung verifikasi terdesentralisasi, atenuasi *offline*, dan penegakan kebijakan akses berbasis *logic language* (Eclipse Biscuit Project, nodate). Berbeda dari JWT dan PASETO yang berperan sebagai pembawa klaim, Biscuit dirancang sebagai token otorisasi berbasis *capability*. Token ini memungkinkan suatu layanan memverifikasi hak akses menggunakan informasi publik, sehingga cocok untuk lingkungan terdistribusi.

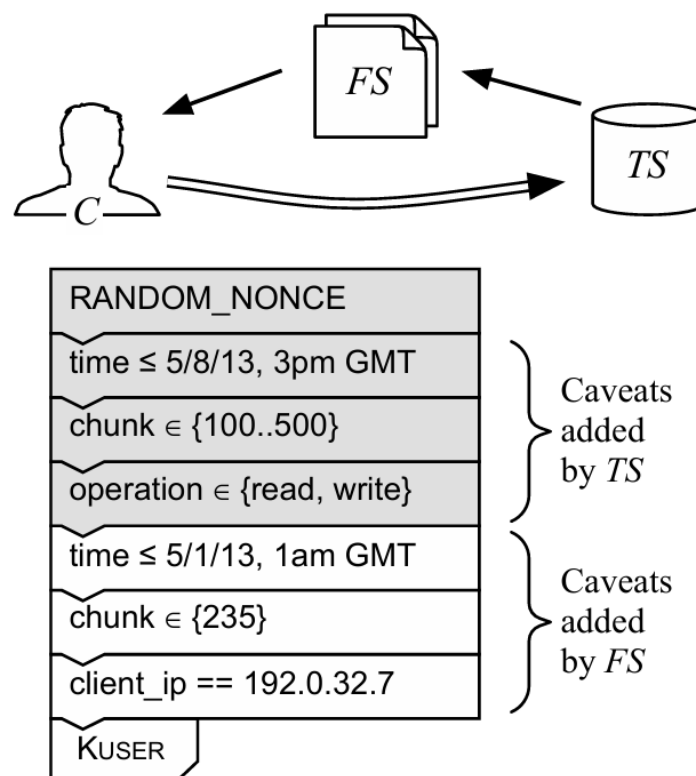
Biscuit menggunakan kriptografi kunci publik untuk memungkinkan verifikasi token oleh pihak yang memiliki kunci publik dari penerbit token. Biscuit

mendukung atenuasi hak akses melalui penambahan blok baru pada token. Pemegang token dapat menambahkan batasan baru untuk mempersempit hak akses sebelum token diberikan ke pihak lain.

Karakteristik utama Biscuit adalah penggunaan *logic language* untuk mengekspresikan *rule*. Informasi otorisasi dapat direpresentasikan dalam bentuk *fact*, *rule*, dan *check*. Pada saat token digunakan, pihak *verifier* dapat mengevaluasi token bersama dengan kebijakan lokal untuk menentukan apakah suatu permintaan akses diperbolehkan. Pendekatan ini membuat Biscuit lebih ekspresif dalam memodelkan kebijakan akses yang kompleks, misalnya kebijakan berbasis relasi antar aktor, sumber daya, aksi, dan konteks permintaan.

II.7 DecMed

Sistem DecMed adalah sebuah kerangka kerja manajemen RME terdesentralisasi yang dibangun di atas DLT IOTA (Sadewa, 2025). Pengembangan sistem ini



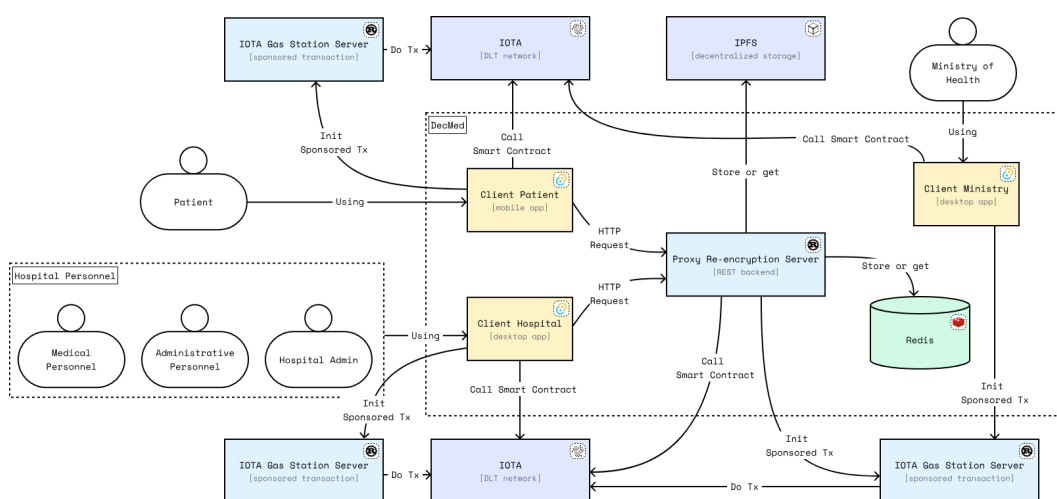
Gambar II.3: Contoh delegasi dan atenuasi kewenangan pada Macaroon. C memegang Macaroon yang diturunkan dari *service* FS untuk mengakses data pada *service* TS dengan batasan waktu, lokasi IP, dan target data yang spesifik. (Birgisson dkk., 2014)

bertujuan untuk mengatasi tantangan fragmentasi data antar institusi kesehatan dan risiko keamanan dari penyimpanan terpusat yang ada di Indonesia.

II.7.1 Arsitektur DecMed

Arsitektur DecMed membagi penyimpanan menjadi dua lapisan, yaitu penyimpanan *on-chain* dan *off-chain*. Penyimpanan *off-chain* memanfaatkan IPFS untuk menyimpan data klinis pasien yang terenkripsi. Sementara itu, penyimpanan *on-chain* memanfaatkan DLT IOTA untuk menyimpan data administratif dan metadata klinis, termasuk CID dari IPFS, guna menjamin integritas data. Untuk memastikan kedaulatan data pasien, DecMed mengimplementasikan CapBAC yang diintegrasikan dengan PRE. Skema PRE yang diterapkan pada DecMed adalah Umbral PRE yang dijelaskan pada Subbab II.4.1. Komponen utama pada arsitektur DecMed dan fungsinya dijelaskan pada Tabel II.1. *Client* DecMed terdiri dari tiga jenis. Masing-masing *client* digunakan oleh aktor yang berbeda, seperti pada Tabel II.2.

DecMed memiliki mekanisme untuk tidak membebankan biaya transaksi / *gas* kepada pengguna. Kerangka kerja DecMed memanfaatkan sebuah komponen tambahan bernama *IOTA Gas Station*. Komponen ini bekerja dengan mekanisme *sponsorship* untuk membayar biaya transaksi *on-chain* pada DecMed. Pada Tugas Akhir ini, komponen *Gas Station IOTA* akan diasumsikan sebagai bagian dari IOTA.



Gambar II.4: Arsitektur DecMed (Sadewa, 2025).

Tabel II.1: Komponen dan fungsi dalam arsitektur DecMed

Komponen	Fungsi
IOTA DLT	Menyimpan metadata, referensi data, dan status akses
IPFS	Menyimpan <i>encrypted blob</i> data RME
PRE	Melakukan verifikasi token akses dan <i>re-encryption</i> agar penerima akses dapat membuka data tanpa kunci privat pasien
Redis	Mengelola <i>cache ephemeral</i> , yang terdiri dari data Nonce, Re-Encryption Key, dan Revoked Key
<i>Patient Client</i>	Mengambil, mendekripsi, dan memberi akses
<i>Hospital Client</i>	Mengambil, menenkripsi, dan mendekripsi data RME sesuai hak akses
<i>Ministry Client</i>	Mengelola fasyankes dan adminnya
<i>IOTA Gas Station</i>	Mensponsori transaksi <i>on-chain</i> agar pengguna tidak perlu membayar <i>gas</i> secara langsung

Tabel II.2: Aktor, *client*, dan peran dalam DecMed

Aktor	<i>Client</i>	Peran dalam DecMed
Pasien	<i>Patient Client</i>	Mengelola identitas, menyimpan atau memperbarui data, dan memberikan akses kepada tenaga medis
Tenaga kesehatan / fasyankes	<i>Hospital Client</i>	Mengakses atau memperbarui RME sesuai akses yang diberikan pasien
Pemerintah	<i>Ministry Client</i>	Mengelola registrasi atau verifikasi fasyankes dalam sistem

II.7.2 Alur Pengelolaan dan Akses RME pada DecMed

Alur pengelolaan RME pada DecMed dimulai ketika data medis dibuat melalui aplikasi *hospital client*. Data tersebut dienkripsi terlebih dahulu di sisi *client* sebelum disimpan pada IPFS. Setelah data terenkripsi berhasil diunggah, IPFS menghasilkan CID sebagai referensi lokasi data. CID, *capsule key*, dan metadata pendukung kemudian dicatat pada DLT IOTA. Dengan demikian, IOTA tidak menyimpan isi rekam medis secara langsung, melainkan menyimpan metadata yang diperlukan untuk menemukan, memverifikasi, dan membuka data terenkripsi melalui mekanisme PRE.

Ketika pasien memberikan akses kepada tenaga medis, PRE menerbitkan token kapabilitas yang digunakan sebagai bukti otorisasi. Tenaga medis membawa token tersebut ketika meminta akses data melalui *Hospital Client*. *Server* PRE kemudian memverifikasi token, melakukan *re-encryption* terhadap *capsule key*, dan

Tabel II.3: Kategori ancaman pada STRIDE

Kategori	Fokus Keamanan	Deskripsi Singkat
<i>Spoofing</i>	Otentikasi	Pemalsuan identitas entitas untuk memperoleh akses tidak sah.
<i>Tampering</i>	Integritas	Modifikasi data, pesan, atau kode secara tidak sah.
<i>Repudiation</i>	<i>Non-repudiation</i>	Penyangkalan terhadap tindakan atau transaksi karena kurangnya bukti audit.
<i>Information Disclosure</i>	Kerahasiaan	Kebocoran informasi sensitif kepada pihak yang tidak berhak.
<i>Denial of Service</i>	Ketersediaan	Gangguan terhadap ketersediaan layanan atau sumber daya sistem.
<i>Elevation of Privilege</i>	Otorisasi	Perolehan hak akses yang lebih tinggi dari yang seharusnya.

mengembalikan material kriptografis yang diperlukan agar *Hospital Client* dapat membuka data terenkripsi sesuai hak akses yang diberikan. *Sequence Diagram* yang menjelaskan alur pemberian akses dapat dilihat pada Lampiran B

II.8 Pemodelan Ancaman STRIDE

STRIDE merupakan metode *threat modeling* yang dikembangkan oleh Microsoft untuk membantu mengidentifikasi ancaman keamanan pada sistem perangkat lunak sejak tahap perancangan. Metode ini mengelompokkan ancaman ke dalam enam kategori utama, yaitu *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service*, dan *Elevation of Privilege* (Microsoft, 2006; Shostack, 2014).

Dalam praktiknya, STRIDE biasanya digunakan bersama *Data Flow Diagram* (DFD) untuk memetakan kemungkinan ancaman terhadap komponen sistem, aliran data, penyimpanan data, dan batas kepercayaan (*trust boundary*) (Shostack, 2014). Pendekatan ini membantu perancang sistem mengidentifikasi kebutuhan kontrol keamanan seperti otentikasi, otorisasi, enkripsi, integritas data, dan *audit logging* sejak tahap desain (Shostack, 2014).

Tabel II.3 merangkum enam kategori ancaman pada STRIDE beserta fokus keamanan yang terkait.

Melalui STRIDE, pengembang dapat mengidentifikasi ancaman keamanan secara sistematis serta menentukan kontrol mitigasi yang sesuai pada tahap perancangan

sistem (Shostack, 2014).

II.9 Penelitian Terkait

Bagian ini meninjau beberapa penelitian yang relevan dengan perancangan kontrol akses granular pada sistem RME berbasis DLT dan implementasi sistem menggunakan Macaroons.

II.9.1 Hybrid RBAC dan ABAC pada Sistem RME Berbasis Blockchain

Singh dkk. mengusulkan model kontrol akses hibrida untuk sistem RME berbasis Blockchain dengan menggabungkan RBAC dan ABAC (Singh dkk., 2025). Penelitian ini berangkat dari keterbatasan dua model tersebut ketika digunakan secara terpisah. RBAC dinilai terlalu kaku karena keputusan akses hanya bergantung pada peran pengguna. Pendekatan ini menyulitkan penerapan kontrol akses yang *fine-grained* dan berpotensi menimbulkan *role explosion* ketika kebutuhan akses semakin beragam. Di sisi lain, ABAC menawarkan fleksibilitas yang lebih tinggi karena keputusan akses mempertimbangkan atribut pengguna, objek, dan konteks, namun kompleksitas perancangan dan pengelolaan kebijakannya juga meningkat.

Penelitian tersebut menunjukkan bahwa semakin banyak atribut yang digunakan dalam ABAC, semakin besar pula kemungkinan konflik kebijakan dan beban evaluasi aturan (Singh dkk., 2025). Oleh karena itu, penulis mengusulkan pendekatan *hybrid*, yaitu RBAC digunakan sebagai lapisan awal untuk menyaring akses berdasarkan peran utama, sedangkan ABAC digunakan untuk menambahkan batasan kontekstual yang lebih rinci. Hasil evaluasi pada *smart contract* menunjukkan bahwa pendekatan *hybrid* ini cukup efisien dari sisi konsumsi *gas* dan tetap relatif sederhana untuk diimplementasikan dibandingkan penggunaan ABAC murni.

II.9.2 Model Kontrol Akses Empat Lapis untuk RME Berbasis Blockchain

Liu dan Chen meneliti model kontrol akses untuk RME berbasis Blockchain dengan menempatkan RME sebagai kasus *big data* (Liu & Chen, 2025). Peran DLT dalam mengatasi masalah tersebut relevan untuk menjelaskan mengapa pendekatan DLT layak dipertimbangkan sebagai alternatif terhadap sistem RME terpusat, karena DLT menawarkan integritas data yang lebih kuat, jejak audit yang transparan, dan distribusi kendali yang lebih baik.

Untuk menjawab permasalahan tersebut, penelitian ini mengusulkan model otorisasi empat lapis yang terdiri dari:

1. *Role Check* menggunakan RBAC,
2. *Identity Verification* menggunakan *Identity-Based Encryption* (IBE),
3. *Policy Execution* menggunakan *smart contract*, dan
4. *Data Volume Check* menggunakan pendekatan *entropy*.

II.9.3 Kontrol Akses Granular Menggunakan *Composable NFT*

Amaracitra mengusulkan model *Decentralized Medical Record* (DMR) yang menggunakan *composable NFT* untuk mencapai keamanan data, interoperabilitas, dan privasi melalui akses data yang granular (Amaracitra & Alamsyah, 2025). Nilai utama dari penelitian ini terletak pada cara data RME dipecah menjadi unit-unit yang lebih kecil, sehingga tidak seluruh data pasien harus selalu dibuka dalam satu paket akses.

Dalam model tersebut, struktur data DMR disusun secara hierarkis ke dalam beberapa token, yaitu:

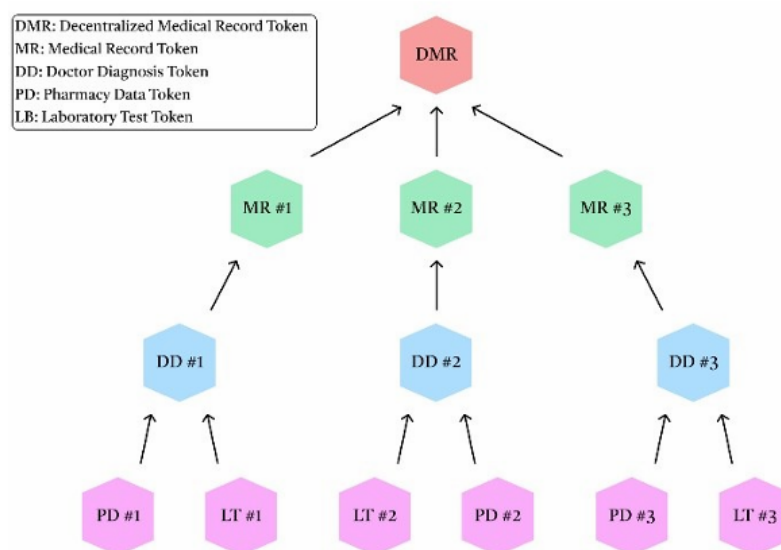
1. *DMR Token* sebagai token induk (*root/parent*),
2. *Medical Record Token*,
3. *Doctor Diagnoses Token*,
4. *Laboratory Test Token*, dan
5. *Pharmacy Data Token*.

Ilustrasi struktur data dari implementasi DMR menggunakan composable NFT ini dapat dilihat pada Gambar II.5.

Pemisahan ini penting karena menunjukkan bahwa granularitas akses tidak hanya bergantung pada mekanisme otorisasi, tetapi juga pada cara data medis dimodelkan sejak awal (Amaracitra & Alamsyah, 2025). Dengan struktur seperti ini, akses dapat diberikan hanya pada bagian data yang relevan dengan kebutuhan tenaga kesehatan tertentu. Misalnya, tenaga laboratorium dapat berinteraksi dengan data pemeriksaan laboratorium tanpa harus memperoleh akses ke keseluruhan rekam medis pasien.

II.9.4 Implementasi Macaroons untuk Delegasi Akses pada Sistem Nyata

Selain diperkenalkan secara teoretis sebagai mekanisme otorisasi berbasis *caveats*, Macaroons juga telah digunakan pada beberapa sistem penyimpanan terdistribusi. Salah satu contohnya adalah Storj, yaitu platform penyimpanan objek terdesentralisasi yang menggunakan Macaroons sebagai dasar pengelolaan akses pada jaringan Tardigrade (Leffew, 2019). Dalam implementasi tersebut, Macaroons diposisikan sebagai *API key* yang berisi aturan akses yang melekat pada token tersebut. Aturan tersebut dapat membatasi operasi yang diperbolehkan, seperti *read*, *write*, *delete*, dan *list*, serta membatasi ruang lingkup akses berdasarkan *bucket*, *path*, *path prefix*, dan jendela waktu akses. Dengan demikian, pengguna dapat



Gambar II.5: Ilustrasi struktur data DMR menggunakan composable NFT. (Amaracitra & Alamsyah, 2025)

membagikan akses terhadap objek data tertentu tanpa memberikan akses penuh terhadap seluruh akun atau seluruh ruang penyimpanan.

Penerapan Macaroons pada Storj menunjukkan bahwa token otorisasi tersebut dapat digunakan pada lingkungan penyimpanan terdesentralisasi. Otoritas akses dibawa oleh token yang secara eksplisit memuat cakupan hak aksesnya. Token tersebut kemudian dapat diturunkan menjadi token dengan cakupan yang lebih sempit melalui penambahan *caveats*.

Contoh implementasi lain dapat ditemukan pada dCache, yaitu sistem penyimpanan data ilmiah berskala besar yang mendukung Macaroons sebagai *bearer token* (dCache, nodate). Dalam dCache, Macaroons dirancang untuk mempermudah proses berbagi akses terhadap file atau direktori tertentu. Pengguna yang memiliki akses terhadap suatu *resource* dapat meminta Macaroon, kemudian token tersebut dapat digunakan oleh pihak lain untuk mengakses sumber daya yang sama sesuai batasan yang tertanam di dalam token. dCache mendefinisikan beberapa *caveats* yang merepresentasikan batasan akses, seperti *activity* untuk menentukan aktivitas yang diizinkan, *before* untuk membatasi masa berlaku token, *ip* untuk membatasi alamat jaringan pengguna, serta *path* dan *root* untuk membatasi ruang lingkup akses pada struktur direktori.

Implementasi dCache juga menunjukkan bahwa beberapa *caveats* dapat dikombinasikan untuk menghasilkan hak akses yang lebih terbatas. Sebagai contoh, beberapa *activity* caveat digabungkan dengan mengambil irisan dari aktivitas yang diizinkan, sehingga penambahan *caveat* baru tidak memperluas hak akses token, melainkan mempersempitnya. Selain itu, pengguna yang telah memiliki Macaroon dapat membuat Macaroon baru dengan tambahan *caveats*, baik secara mandiri menggunakan pustaka Macaroons maupun melalui permintaan ke dCache. Mekanisme ini memperlihatkan pola delegasi akses yang sesuai dengan prinsip atenuasi, yaitu token turunan tidak dapat memiliki hak akses yang lebih besar daripada token asal.

Dari kedua implementasi tersebut, dapat disimpulkan bahwa Macaroons telah digunakan untuk membatasi dan mendelegasikan akses pada sistem penyimpanan data terdistribusi. Storj menekankan penggunaan Macaroons untuk pengelolaan akses objek data pada jaringan penyimpanan terdesentralisasi, sedangkan dCache

menunjukkan bagaimana *caveats* dapat digunakan untuk membatasi aktivitas, lokasi sumber daya, alamat jaringan, dan masa berlaku akses.

BAB III

ANALISIS DAN RANCANGAN SOLUSI

III.1 Analisis Masalah

Dalam mengembangkan mekanisme kontrol akses granular dan delegasi pada kerangka kerja DecMed, diperlukan analisis terhadap implementasi dan arsitektur sistem yang sudah ada. Hasil analisis tersebut digunakan sebagai dasar untuk mengidentifikasi permasalahan serta merumuskan kebutuhan sistem yang diusulkan pada Tugas Akhir ini.

Analisis masalah pada Tugas Akhir ini diawali dengan analisis terhadap implementasi dan arsitektur DecMed pada Subbab III.1.1, termasuk analisis STRIDE dan skenario layanan kesehatan kolaboratif. Hasil analisis tersebut kemudian digunakan untuk mengidentifikasi permasalahan pada Subbab III.1.2 dan merumuskan kebutuhan sistem pada Subbab III.1.3.

III.1.1 Analisis Sistem DecMed

Sebagaimana dijelaskan pada Bab II.7, DecMed adalah kerangka kerja manajemen RME terdesentralisasi dengan menggunakan DLT IOTA sebagai penyimpanan *on-chain*, IPFS sebagai penyimpanan *off-chain*, serta PRE untuk mendukung distribusi akses data yang aman. Arsitektur ini sudah memberikan fondasi untuk menjaga integritas data dan mengurangi ketergantungan pada penyimpanan terpusat. Terdapat tiga aktor utama pada DecMed, yaitu:

1. Pasien
2. Staf fasyankes, terdiri dari tenaga kesehatan, petugas rekam medis, dan admin fasyankes
3. Pemerintah, yaitu Kementerian Kesehatan

Pada DecMed, masing-masing aktor memiliki kepentingan dan aplikasi *client* yang berbeda. *Patient Client* digunakan oleh pasien, *Hospital Client* digunakan

oleh tenaga kesehatan di sebuah fasyankes, dan *Ministry Client* digunakan oleh pemerintah. Detail kebutuhan fungsional DecMed untuk memenuhi kebutuhan masing-masing aktor dapat dilihat pada Lampiran C.

III.1.1.1 Keterbatasan Token Kapabilitas DecMed

Model kontrol akses yang digunakan pada sistem DecMed adalah model CapBAC. DecMed memanfaatkan token otorisasi berupa JWT sebagai token otorisasi yang merepresentasikan kapabilitas akses. Pada kerangka kerja DecMed, ketika pasien memberikan akses terhadap RME mereka kepada seorang tenaga medis, maka PRE akan menerbitkan token JWT sesuai dengan *request* dari pasien. Token yang diterbitkan oleh pasien membawa klaim seperti pada Kode III.1. Dalam implementasi DecMed sebelumnya, token JWT masih bersifat *coarse-grained*. Penentuan hak akses pengguna dilakukan hanya berdasarkan *role* dan *purpose* yang ada pada token yang disertakan saat melakukan *request* ke PRE. Token tersebut belum dirancang untuk membawa batasan kontekstual, misalnya jenis data yang boleh diakses. Padahal, pada praktik pelayanan kesehatan, setiap tenaga medis tidak selalu membutuhkan keseluruhan isi RME pasien sesuai dengan prinsip *need-to-know*. Kondisi ini menunjukkan bahwa desain *claims* token otorisasi pada DecMed belum cukup untuk menjawab kebutuhan kontrol akses granular.

```
1  #[derive(Debug, Deserialize, Serialize)]
2  pub struct JwtClaims {
3      pub address: String, // address IOTA pemegang token
4      pub role: AuthRole, // AdministrativePersonnel
5                          // atau MedicalPersonnel
6      pub purpose: ReencryptionPurposeType, // Read atau Update
7  }
```

Kode III.1: Struktur klaim JWT pada DecMed

Rancangan *claims* pada JWT ini menimbulkan keterbatasan karena token JWT pada DecMed masih bersifat *coarse-grained*. Penentuan hak akses pengguna dilakukan hanya berdasarkan *role* dan *purpose* yang ada pada token yang disertakan saat melakukan *request* ke PRE. Token tersebut belum dirancang untuk membawa batasan kontekstual, misalnya jenis data yang boleh diakses. Padahal, pada praktik pelayanan kesehatan, setiap tenaga medis tidak selalu membutuhkan keseluruhan

isi RME pasien sesuai dengan prinsip *need-to-know*. Kondisi ini menunjukkan bahwa desain *claims* token otorisasi pada DecMed belum cukup untuk menjawab kebutuhan kontrol akses granular.

III.1.1.2 Keterbatasan Segmentasi Data pada DecMed

Pada kerangka kerja DecMed, data RME pasien disimpan dalam dua sistem penyimpanan yang berbeda, yaitu penyimpanan *on-chain* (IOTA) dan *off-chain* (IPFS). Penyimpanan *on-chain* digunakan untuk menyimpan metadata dan CID dari data yang disimpan di *off-chain*. Penyimpanan *off-chain* digunakan untuk menyimpan data administratif dan klinis yang terenkripsi. Pemisahan ini sesuai dengan kebutuhan yang dibahas pada II.1.2 terkait dokumentasi administratif dan klinis.

Pada penyimpanan *on-chain*, data RME pasien direpresentasikan sebagai metadata dalam sebuah *struct* `MedicalMetadata` untuk data klinis. Metadata tersebut menyimpan CID serta informasi kunci terenkripsi yang digunakan dalam proses dekripsi data medis terkait. Struktur penyimpanannya terikat dengan *struct* `PatientAccount` dan `PatientMedicalMetadata` pada Kode III.2.

```
1 public struct PatientAccount has store {
2     ...
3     medical_metadata: TableVec<PatientMedicalMetadata>,
4 }
5 public struct PatientMedicalMetadata has copy, drop, store {
6     index: u64,
7     metadata: String,
8 }
```

Kode III.2: Struktur metadata medis pasien pada DecMed

Karena *field* `medical_metadata` pada *struct* `PatientAccount` menyimpan daftar metadata klinis pasien, akses terhadap struktur ini memungkinkan sistem memperoleh referensi menuju objek data klinis terenkripsi, seperti CID dan *capsule* untuk PRE. CID dan *capsule* tidak secara langsung membuka isi RME karena data klinis tetap disimpan dalam bentuk terenkripsi. Namun, struktur metadata tersebut menunjukkan bahwa unit akses pada DecMed masih mengarah pada satu objek data klinis yang relatif besar, bukan pada segmen data yang spesifik.

Kondisi tersebut menyulitkan penegakan kontrol akses granular. Ketika proses *re-encryption* diberikan terhadap objek data klinis tersebut, penerima akses berpotensi memperoleh cakupan *plaintext* yang lebih luas daripada kebutuhan aktualnya. Hal ini terjadi karena struktur metadata belum menyediakan atribut segmentasi yang cukup untuk membedakan cakupan akses sesuai kebutuhan pelayanan. Variabel metadata pada *struct* PatientMedicalMetadata merupakan serialisasi dari *struct* yang ditunjukkan pada Kode III.3.

```
1  #[derive(Debug, Deserialize, Serialize)]
2  pub struct MedicalMetadata {
3      pub capsule: String, // capsule untuk PRE re-encryption
4      pub cid: String, // pointer ke data di IPFS
5      pub created_at: String, // timestamp
6      pub enc_key_and_nonce: String, // kunci dan nonce yang
7      // terenkripsi
8  }
```

Kode III.3: Struktur serialisasi metadata medis DecMed

cid pada *struct* MedicalMetadata merupakan *pointer* ke data klinis yang sudah terenkripsi disimpan di penyimpanan *off-chain* sebagai sebuah *Encrypted Blob*. Potongan field relevan pada *struct* MedicalData dapat dilihat pada Kode III.4.

```
1  pub struct MedicalData {
2      pub anamnesis: String,
3      pub physical_check: String,
4      pub psychological_check: String,
5      pub diagnose: String,
6      pub therapy: String,
7  }
```

Kode III.4: Potongan struktur data medis *off-chain* pada DecMed

Penyimpanan data klinis pada DecMed hanya berfokus pada dataset rawat jalan. Pada *struct* MedicalData, data klinis masih disimpan dengan format yang sederhana, tanpa kategorisasi yang jelas. DecMed memang telah memisahkan penyimpanan *on-chain* dan *off-chain*, tetapi pendekatan tersebut belum secara eksplisit menunjukkan segmentasi data klinis yang cukup rinci untuk menegakkan

kebijakan akses granular. Jika data klinis masih dikelola sebagai objek terenkripsi yang terlalu besar, pemberian akses kepada tenaga medis berisiko membuka data di luar kebutuhan klinisnya. Sebagai contoh, seorang perawat tidak membutuhkan akses tulis ke data therapy pada *struct* MedicalData karena yang berhak menentukan terapi adalah dokter.

III.1.1.3 Analisis Skenario Layanan Kesehatan Kolaboratif

Kebutuhan akan kontrol akses granular dan delegasi berantai menjadi lebih jelas ketika arsitektur DecMed ditempatkan pada skenario layanan kesehatan yang nyata. Dalam alur pelayanan rumah sakit, pasien tidak hanya berinteraksi dengan satu pihak, tetapi dengan beberapa aktor yang memiliki kebutuhan akses berbeda pada tahap yang berbeda pula sebagaimana dibahas pada Subbab II.1.3. Oleh karena itu, subbab ini memodelkan satu skenario layanan kesehatan kolaboratif untuk menunjukkan secara konkret mengapa mekanisme akses pada DecMed saat ini belum memadai.

Skenario yang dianalisis adalah pelayanan pasien rawat jalan dan rawat inap yang melibatkan petugas rekam medis, perawat, dokter penanggung jawab pasien, petugas laboratorium, dan apoteker. Alur tersebut dipilih karena mewakili pola kolaborasi yang umum terjadi di sebuah fasyankes.

1. **Tahap pendaftaran pasien.** Saat pasien datang ke rumah sakit, petugas rekam medis hanya membutuhkan akses terhadap data administratif pasien.
2. **Tahap pemeriksaan awal oleh perawat.** Setelah proses pendaftaran selesai, perawat memerlukan hak untuk menulis data klinis untuk anamnesis awal, pemeriksaan fisik, dan pemeriksaan psikologis.
3. **Tahap pemeriksaan oleh dokter.** Pada tahap ini, dokter penanggung jawab pasien memerlukan hak baca terhadap keseluruhan data RME pasien dan hak tulis untuk pelayanan saat ini.
4. **Tahap rujukan internal ke laboratorium.** Apabila dokter membutuhkan pemeriksaan laboratorium, maka pasien perlu diarahkan ke petugas laboratorium. Dalam tahap ini, petugas laboratorium hanya membutuhkan akses terhadap data administratif pasien dan permintaan pemeriksaan laboratorium.

Tabel III.1: Pemetaan Aktor dan Kebutuhan Akses pada Skenario Layanan Kesehatan Kolaboratif

Aktor	Jenis Akses	Cakupan Data yang Dibutuhkan
Petugas Rekam Medis	Baca data administratif	Data administratif pasien.
Perawat	Baca dan tulis data klinis (hanya pemeriksaan awal)	Baca data administratif pasien. Tulis data anamnesis awal, pemeriksaan fisik, dan pemeriksaan psikologis.
Dokter	Baca dan tulis data klinis	Baca keseluruhan RME pasien. Tulis data sesuai konteks pelayanan saat ini.
Petugas Laboratorium	Baca dan tulis terbatas (hanya data laboratorium)	Baca data administratif pasien dan permintaan pemeriksaan penunjang. Tulis data laboratorium.
Apoteker	Baca dan tulis terbatas (hanya data apotek)	Baca data administratif pasien dan data terapi. Tulis data peresepan dan <i>dispensing</i> .

5. **Tahap tindak lanjut hasil pemeriksaan.** Setelah hasil laboratorium tersedia, dokter kembali menggunakan data tersebut sebagai bahan pengambilan keputusan klinis.

6. **Tahap pemberian resep oleh apotek.** Setelah dokter memberikan resep, apotek memerlukan hak untuk membaca data resep dan mengunggah data obat yang diresepkan.

Berdasarkan skenario tersebut, kebutuhan akses setiap aktor dapat diringkas pada Tabel III.1.

Tabel III.1 juga menunjukkan bahwa kategori tenaga medis tidak dapat diperlakukan sebagai satu kelompok akses tenaga medis. Dokter, perawat, petugas laboratorium, dan apoteker sama-sama terlibat dalam pelayanan klinis, tetapi memiliki kewenangan baca dan tulis yang berbeda. Oleh karena itu, penggunaan role umum seperti *MedicalPersonnel* yang digunakan pada DecMed saat ini belum cukup untuk menentukan hak akses tulis secara aman, karena sistem tetap perlu membedakan fungsi klinis masing-masing aktor.

Alur pemberian akses terhadap data RME pasien oleh pasien kepada tenaga medis pada DecMed sebelumnya dapat dilihat pada Lampiran B. Pada alur tersebut, pasien harus memilih satu tenaga medis lalu menerbitkan akses untuk tenaga medis tersebut. Implementasi DecMed sebelumnya tidak memiliki mekanisme delegasi antar tenaga kesehatan. Sehingga ketika akses perlu diberikan kepada lebih dari satu tenaga medis, proses yang sama perlu diulang dari awal untuk masing-masing

penerima akses.

Dalam skenario tersebut, keterbatasan DecMed tampak secara operasional. Model token kapabilitas yang statis menyulitkan proses pembatasan akses secara bertahap sesuai perpindahan peran di lapangan. Selain itu, pemberian akses hanya dapat dilakukan secara individual dari pasien kepada satu tenaga medis pada satu waktu, sehingga proses otorisasi menjadi tidak efektif ketika layanan melibatkan beberapa tenaga medis sekaligus. Pasien perlu memberikan akses kepada dokter utama dan petugas laboratorium secara terpisah, padahal proses tersebut dapat dibuat lebih efisien apabila dokter utama dapat mendelegasikan sebagian hak aksesnya kepada petugas laboratorium tanpa perlu intervensi langsung dari pasien. Dokter perlu dapat mendelegasikan sebagian hak akses saja, misalnya akses terhadap permintaan pemeriksaan dan hak tulis hasil laboratorium tanpa ikut membuka seluruh riwayat klinis pasien.

Dengan demikian, skenario layanan kesehatan kolaboratif ini memperlihatkan bahwa tiga persoalan penelitian saling berkaitan: sistem memerlukan kontrol akses yang granular agar setiap aktor hanya memperoleh data yang relevan, segmentasi data *off-chain* agar pembatasan akses dapat ditegakkan secara teknis, dan delegasi berantai dengan atenuasi *offline* agar perpindahan hak akses dapat berlangsung efisien tanpa intervensi ulang dari pasien.

III.1.1.4 Analisis STRIDE pada Sistem DecMed

Analisis STRIDE pada subbab ini digunakan untuk mengevaluasi implikasi keamanan dari arsitektur DecMed saat ini ketika ditempatkan pada skenario layanan kesehatan kolaboratif yang menjadi fokus Tugas Akhir ini. Berdasarkan pembahasan pada Subbab II.8, tahapan awal dalam melakukan pemodelan ancaman menggunakan STRIDE adalah mengidentifikasi komponen dan alur kerja sistem yang menjadi target ancaman. Hal ini dapat dilakukan dengan menggambarkan DFD yang menunjukkan aliran data dan komunikasi antar komponen sistem. Pada analisis ini, dilakukan pemodelan DFD Level 0 dan Level 1. Pemodelan ke level lebih lanjut tidak dilakukan karena pemodelan hingga level 1 sudah cukup untuk mengidentifikasi komponen dari DecMed dan alur data antar komponen. Hasil pemodelan DFD dapat dilihat pada Lampiran D.

Berdasarkan arsitektur DecMed pada Gambar II.4 dan DFD pada Lampiran D, *attack surface* utama sistem mencakup *Patient Client*, *Hospital Client*, *Ministry Client*, data pada lapisan *on-chain*, data terenkripsi pada lapisan *off-chain*, Redis PRE, jaringan IOTA, *IOTA Gas Station*, serta token kapabilitas yang mengatur otorisasi akses. Hasil identifikasi ancaman berdasarkan metode STRIDE terhadap komponen utama pada DecMed dapat dilihat pada Lampiran E.

Berdasarkan hasil analisis STRIDE pada Tabel E.1, ancaman yang paling relevan terhadap fokus Tugas Akhir ini adalah *spoofing* (T01), *tampering* (T02 dan T03), *information disclosure* (T05 dan T06), dan *elevation of privilege* (T08). Ancaman-ancaman tersebut secara langsung berkaitan dengan kebutuhan akan kontrol akses yang lebih granular, segmentasi data *off-chain*, serta mekanisme delegasi yang mampu membatasi hak akses secara bertahap. Ancaman *repudiation* (T04) tetap relevan terhadap kebutuhan audit delegasi, tetapi pada penelitian ini diposisikan sebagai implikasi pendukung karena fokus utama berada pada pembatasan cakupan akses dan delegasi. Ancaman *denial of service* (T07) tidak dijadikan masalah yang diselesaikan ataupun diuji pada Tugas Akhir ini, karena tidak termasuk dalam fokus Tugas Akhir terkait kontrol akses granular, segmentasi data, dan mekanisme delegasi *offline*.

Dengan demikian, hasil analisis STRIDE memperkuat bahwa keterbatasan pada model kontrol akses dan delegasi DecMed tidak hanya merupakan persoalan fungsional, tetapi juga memiliki implikasi keamanan yang nyata. Oleh karena itu, diperlukan mekanisme otorisasi yang mampu mendukung pembatasan akses yang lebih rinci dan aman pada sistem rekam medis elektronik terdesentralisasi

III.1.1.5 Analisis Mekanisme Penggunaan dan Revokasi Token

Pada implementasi DecMed sebelumnya, kapabilitas akses ditentukan oleh kepemilikan token dan klaim yang dibawa oleh token tersebut. Setiap operasi pembacaan maupun penulisan data RME dilakukan melalui *request* yang menyertakan token akses. Komponen PRE kemudian memvalidasi token dengan mengacu pada hak akses yang tercatat pada penyimpanan *on-chain* IOTA. Validasi ini digunakan untuk memastikan bahwa *address* IOTA yang tercantum pada klaim token memiliki hak akses yang sesuai pada penyimpanan *on-chain* dan hak akses tersebut masih berlaku. Struktur klaim token ditunjukkan pada Kode III.1.

Mekanisme tersebut masih memiliki keterbatasan karena validasi berfokus pada klaim yang dibawa token, bukan pada pembuktian bahwa pengirim *request* benar-benar merupakan pemilik *address* yang tercantum pada klaim. Jika token akses diperoleh oleh pihak lain, misalnya melalui kebocoran atau pencurian token, pihak tersebut dapat mengirimkan *request* dengan klaim *address* yang masih valid. Kondisi ini membuat token dapat digunakan oleh aktor yang tidak berwenang selama klaim di dalam token masih diterima oleh sistem. Oleh karena itu, diperlukan mekanisme tambahan untuk mengikat penggunaan token dengan bukti identitas dari aktor yang melakukan akses.

Selain verifikasi terhadap pengguna token, mekanisme pencabutan hak akses juga perlu diperhatikan. Pada DecMed, pasien dapat mencabut hak akses sebelum masa berlaku akses berakhir. Proses ini dilakukan melalui *patient client* dengan mengirimkan *request* revokasi ke PRE. Setelah diterima, status revokasi dicatat pada Redis dan penyimpanan *on-chain* IOTA agar akses yang telah dicabut tidak lagi diterima oleh sistem.

Apabila mekanisme delegasi akses ditambahkan, sebagaimana dibutuhkan pada skenario layanan kesehatan kolaboratif di fasilitas pelayanan kesehatan pada Subbab III.1.1.3, mekanisme revokasi tidak cukup hanya menangani token awal. Token hasil delegasi juga perlu diperhitungkan, termasuk hubungan antara token induk dan token turunannya. Dengan demikian, pencabutan akses dapat diterapkan tidak hanya terhadap akses yang diberikan langsung oleh pasien, tetapi juga terhadap akses yang muncul melalui proses delegasi.

III.1.2 Identifikasi Masalah

Berdasarkan analisis terhadap arsitektur DecMed pada Subbab III.1.1, Tugas Akhir ini mengidentifikasi sejumlah masalah yang perlu diselesaikan agar DecMed dapat mendukung kontrol akses granular dan delegasi *offline*. Permasalahan utama terdiri atas tiga aspek. Pertama, token otorisasi DecMed belum mampu menyatakan batasan akses secara granular, seperti jenis data, aksi akses, masa berlaku, dan ruang lingkup delegasi. Kedua, data RME pada penyimpanan *off-chain* belum direpresentasikan sebagai objek akses yang tersegmentasi sehingga pembatasan akses sulit ditegakkan pada unit data spesifik. Ketiga, alur pemberian akses DecMed masih bergantung pada keterlibatan langsung pasien sehingga belum mendukung

Tabel III.2: Identifikasi Masalah DecMed

ID	Masalah	Dasar Analisis
M-TA-01	Token otorisasi DecMed belum mampu mengekspresikan batasan akses secara granular.	III.1.1.1, T02, T05
M-TA-02	Data RME belum direpresentasikan sebagai objek akses yang tersegmentasi.	III.1.1.2, T05
M-TA-03	Alur pemberian akses belum mendukung delegasi antaraktor dalam layanan kolaboratif.	III.1.1.3
M-TA-04	Delegasi antaraktor memerlukan mekanisme atenuasi.	III.1.1.3, T05, T08
M-TA-05	<i>Role</i> MedicalPersonnel terlalu umum untuk membedakan kewenangan antar aktor.	III.1.1.3, T02, T05
M-TA-06	Mekanisme revokasi dan audit akses perlu diperluas agar mencakup akses turunan pada rantai delegasi.	III.1.1.5, T04, T08
M-TA-07	Token akses belum secara eksplisit membuktikan kepemilikan identitas aktor aktif pada setiap permintaan.	III.1.1.5, T01

delegasi antaraktor dalam proses layanan kesehatan yang kolaboratif.

Selain tiga masalah utama tersebut, terdapat beberapa masalah turunan yang perlu diperhatikan agar rancangan solusi dapat ditegakkan secara aman. Delegasi antaraktor harus dibatasi agar hak akses turunan tidak melebihi hak akses induknya. Perbedaan data klinis yang dapat diakses oleh masing-masing aktor juga perlu diperhitungkan karena *role* umum seperti MedicalPersonnel belum cukup untuk membedakan kewenangan dokter, perawat, petugas laboratorium, dan apoteker. Selain itu, karena delegasi dapat membentuk rantai akses, mekanisme pencabutan dan audit pada DecMed perlu diperluas agar dapat mencakup akses turunan. Penggunaan token akses juga perlu dikaitkan dengan pembuktian identitas aktor aktif pada setiap permintaan agar token tidak hanya diperlakukan sebagai *bearer token* yang dapat digunakan oleh pihak yang tidak semestinya.

Agar keterhubungan antara analisis masalah, ancaman keamanan, kebutuhan sistem, rancangan, implementasi, dan pengujian dapat ditelusuri, permasalahan tersebut dirinci menggunakan ID M-TA pada Tabel III.2.

Berdasarkan Tabel III.2, Tugas Akhir ini tidak hanya memandang keterbatasan DecMed sebagai persoalan token otorisasi, tetapi sebagai rangkaian masalah yang saling bergantung. M-TA-01, M-TA-02, dan M-TA-03 merupakan masalah inti yang masing-masing berkaitan dengan kebutuhan kontrol akses granular, segmentasi objek data RME, dan delegasi antaraktor. Sementara itu, M-TA-04,

M-TA-05, M-TA-06, dan M-TA-07 merupakan masalah turunan atau implikasi yang perlu diselesaikan agar ketiga masalah inti tersebut dapat ditangani secara aman. M-TA-04 menekankan perlunya pembatasan hak akses turunan, M-TA-05 menekankan perlunya pembedaan kewenangan berdasarkan aktor, M-TA-06 menekankan perlunya perluasan pencabutan dan audit terhadap rantai delegasi, sedangkan M-TA-07 menekankan perlunya pembuktian identitas aktor aktif pada setiap permintaan akses. ID masalah tersebut kemudian digunakan sebagai dasar pemetaan kebutuhan, rancangan, implementasi, dan pengujian pada bagian selanjutnya.

III.1.3 Kebutuhan Sistem

Berdasarkan masalah yang telah dipetakan pada Tabel III.2, kebutuhan sistem pada Tugas Akhir ini dirumuskan sebagai perluasan terhadap kebutuhan DecMed sebelumnya. Kebutuhan ini tidak dimaksudkan untuk menggantikan fungsi dasar DecMed, melainkan untuk memastikan bahwa sistem dapat mendukung kontrol akses granular, segmentasi data RME, delegasi berantai, revokasi akses turunan, audit delegasi, dan pembuktian identitas aktor aktif.

Kebutuhan sistem dibagi menjadi kebutuhan fungsional dan kebutuhan nonfungsional. Kebutuhan fungsional menjelaskan kemampuan yang harus disediakan oleh sistem dari sudut pandang aktor dan proses bisnis layanan kesehatan. Sementara itu, kebutuhan nonfungsional menjelaskan batasan kualitas dan keamanan yang harus dipenuhi dalam penerapan mekanisme kontrol akses dan delegasi.

III.1.3.1 Kebutuhan Fungsional Tambahan

Kebutuhan fungsional tambahan dirumuskan dari sudut pandang aktor, sama seperti pola pada kebutuhan fungsional DecMed pada Lampiran C. Perumusan berbasis aktor digunakan untuk memperjelas pihak yang dapat menerima akses, jenis data yang dapat diakses, operasi yang dapat dilakukan, serta batasan akses yang harus ditegakkan. Batasan tersebut mencakup pasien, episode RME, dataset, fungsi klinis, aksi akses, waktu berlaku, penerima delegasi, dan kedalaman delegasi. Rangkuman kebutuhan fungsional tambahan DecMed ditunjukkan pada Tabel III.3.

Tabel III.3: Kebutuhan Fungsional Tambahan DecMed

ID	Deskripsi
KF-TA-01	Pasien dapat memberikan akses awal kepada petugas rekam medis sesuai konteks layanan kesehatan.
KF-TA-02	Pasien dapat mencabut akses awal dan akses turunannya yang telah diberikan.
KF-TA-03	Pasien dapat melihat riwayat akses dan delegasi terhadap RME miliknya.
KF-TA-04	Petugas rekam medis dapat mendelegasikan akses terbatas kepada perawat dan dokter sesuai cakupan akses.
KF-TA-05	Dokter dapat membaca dan menulis segmen klinis sesuai cakupan akses dan konteks pelayanan.
KF-TA-06	Dokter dapat mendelegasikan akses terbatas kepada tenaga laboratorium dan apoteker sesuai cakupan akses.
KF-TA-07	Perawat dapat membaca data yang relevan dan menulis segmen klinis pemeriksaan awal sesuai cakupan akses.
KF-TA-08	Petugas laboratorium dapat membaca data yang relevan dan menulis segmen klinis terkait pemeriksaan laboratorium sesuai cakupan akses.
KF-TA-09	Apoteker dapat membaca data yang relevan dan menulis segmen klinis terkait pelayanan obat sesuai cakupan akses.

Kebutuhan fungsional dengan ID KF-TA-01 merupakan penyesuaian dari F06 dan F07 pada Lampiran C. Penyesuaian ini dilakukan karena dalam skenario layanan kesehatan kolaboratif, akses awal terhadap RME hanya diberikan langsung oleh pasien kepada petugas rekam medis. Kemudian hak akses tersebut dapat didelegasikan secara terbatas kepada aktor tenaga medis yang terlibat dalam proses pelayanan, seperti dokter, perawat, petugas laboratorium, dan apoteker.

KF-TA-02 juga merupakan penyesuaian dari F09 pada Lampiran C. Penyesuaian ini terkait dengan mekanisme revokasi akses pada skenario layanan kesehatan kolaboratif. Dalam skenario tersebut, pasien memerlukan kemampuan untuk mencabut akses awal dan akses turunannya yang telah didelegasikan.

III.1.3.2 Kebutuhan Nonfungsional

Selain kebutuhan fungsional, sistem juga memiliki kebutuhan nonfungsional untuk memastikan bahwa mekanisme kontrol akses granular dan delegasi berantai dapat diterapkan secara aman. Kebutuhan nonfungsional pada Tugas Akhir ini difokuskan pada aspek keamanan, keterlacakan, dan kompatibilitas dengan arsitektur DecMed.

Tabel III.4: Kebutuhan Nonfungsional Tambahan DecMed

ID	Deskripsi
KNF-TA-01	Sistem harus menjaga kerahasiaan data RME dengan memastikan bahwa penerima akses hanya dapat memperoleh <i>plaintext</i> dari segmen yang sesuai dengan cakupan otorisasinya.
KNF-TA-02	Sistem menjamin integritas token otorisasi dan batasan akses tidak mungkin diperluas ketika delegasi.
KNF-TA-03	Sistem menjamin penolakan permintaan akses jika token, rantai delegasi, atau bukti identitas aktor aktif tidak valid.
KNF-TA-04	Sistem menyediakan keterlacakan delegasi dengan mencatat informasi delegasi.
KNF-TA-05	Sistem menjamin penolakan akses untuk hak akses yang dicabut.
KNF-TA-06	Sistem harus memverifikasi keterikatan bukti kepemilikan identitas terhadap aktor aktif, token akses, dan konteks permintaan akses.

Tabel III.5: Pemetaan Masalah terhadap Kebutuhan Sistem

ID Masalah	Kebutuhan Fungsional	Kebutuhan Nonfungsional
M-TA-01	KF-TA-01, KF-TA-04–KF-TA-09	KNF-TA-01, KNF-TA-02, KNF-TA-03
M-TA-02	KF-TA-05, KF-TA-07, KF-TA-08, KF-TA-09	KNF-TA-01
M-TA-03	KF-TA-01, KF-TA-04, KF-TA-06	KNF-TA-04
M-TA-04	KF-TA-04, KF-TA-06	KNF-TA-02, KNF-TA-03
M-TA-05	KF-TA-05, KF-TA-07, KF-TA-08, KF-TA-09	KNF-TA-01, KNF-TA-03
M-TA-06	KF-TA-02, KF-TA-03	KNF-TA-04, KNF-TA-05
M-TA-07	<i>Tidak ada</i>	KNF-TA-03, KNF-TA-06

III.1.3.3 Pemetaan Masalah terhadap Kebutuhan Sistem

Setelah kebutuhan fungsional dan nonfungsional dirumuskan, dilakukan pemetaan antara masalah yang telah diidentifikasi dan kebutuhan sistem yang diusulkan. Pemetaan ini digunakan untuk memastikan bahwa setiap masalah memiliki kebutuhan yang menanganinya.

Kebutuhan fungsional dan nonfungsional tambahan tersebut menjadi dasar bagi perancangan solusi pada tahap berikutnya.

III.2 Analisis Solusi

DecMed sudah memiliki fondasi yang sesuai untuk sistem RME terdesentralisasi melalui IOTA, IPFS, dan PRE. Oleh karena itu, solusi yang dianalisis pada bagian ini diarahkan untuk memperkuat lapisan kontrol akses agar menjadi lebih granular

dan memfasilitasi kemampuan delegasi *offline*.

Berdasarkan kebutuhan sistem pada Subbab III.1.3, solusi yang dianalisis pada Tugas Akhir ini dikelompokkan ke dalam enam aspek utama, yaitu token otorisasi, segmentasi objek data, delegasi akses *offline*, serta penegakan otoritas. Pengelompokan ini diperlukan karena kontrol akses granular dan kemampuan delegasi *offline* tidak dapat ditegakkan hanya melalui satu komponen. Token otorisasi diperlukan untuk menyatakan batasan akses, segmentasi data diperlukan agar batasan tersebut memiliki objek penegakan yang jelas, sedangkan *Server PRE* diperlukan sebagai titik penegakan sebelum data terenkripsi diambil dan diproses *re-encryption*. Dengan demikian, analisis solusi pada bagian ini berfungsi sebagai dasar pemilihan pendekatan sebelum rancangan teknis dijelaskan pada Subbab III.3.

III.2.1 Analisis Delegasi Akses dan Atenuasi *Offline*

Delegasi akses diperlukan berdasarkan hasil analisis pada Subbab III.1.1.3. Dalam skenario DecMed sebelumnya, pasien memberikan akses kepada aktor tertentu secara langsung. Pola tersebut kurang sesuai untuk layanan klinis kolaboratif karena kebutuhan akses dapat berpindah antar aktor selama episode berlayanan berlangsung. Jika setiap perpindahan akses harus selalu diterbitkan langsung oleh pasien, maka proses pelayanan menjadi bergantung pada ketersediaan pasien.

Alternatif solusi pertama yang dipertimbangkan adalah melakukan delegasi akses melalui *client* dan mencatat hasil delegasi pada penyimpanan *on-chain*. Pada pendekatan ini, aktor yang memiliki token induk membentuk token turunan secara lokal dengan menambahkan batasan baru terhadap hak akses yang dimiliki. Token turunan tersebut kemudian dibagikan kepada penerima delegasi melalui penyimpanan *on-chain*. Pendekatan ini dapat mengurangi keterlibatan pasien karena pemegang token dapat menurunkan hak akses yang sudah diterima sebelumnya.

Namun, pendekatan tersebut menjadikan *client* sebagai komponen yang menentukan hasil delegasi. Kondisi ini kurang ideal karena *client* dapat dimanipulasi untuk melakukan delegasi sesuka hati. Akibatnya, sistem berpotensi mencatat delegasi yang tidak konsisten dengan hak akses induk, meskipun token turunan tersebut kemudian dapat ditolak ketika digunakan untuk mengakses *Server*

PRE.

Alternatif berikutnya adalah melibatkan *Server PRE* sebagai komponen yang memverifikasi permintaan delegasi sebelum token turunan dibentuk. Pada pendekatan ini, aktor delegator tetap menginisiasi delegasi melalui *client*. *Client* mengirimkan token induk yang dimiliki *delegator*, lalu token dan hak akses *delegator* diverifikasi oleh *Server PRE*.

Jika permintaan delegasi dinyatakan sah, *Server PRE* membentuk hak akses turunan dengan menambahkan dengan batasan baru yang mempersempit hak akses. *PRE* kemudian mengembalikan hak akses tersebut kepada *client*. Setelah menerima hasil atenuasi dari *PRE*, *client* kemudian dapat melakukan proses pencatatan lebih lanjut pada penyimpanan *on-chain*. Dengan demikian, *PRE* hanya menjadi verifikator dan pembentuk hak akses turunan, sedangkan pencatatan delegasi tetap dilakukan oleh *client*.

Berdasarkan pertimbangan tersebut, Tugas Akhir ini memilih mekanisme delegasi akses yang dimediasi oleh *Server PRE*. Mekanisme ini memanfaatkan konsep atenuasi sehingga token turunan hanya dapat memiliki cakupan yang sama atau lebih sempit daripada token induk. Istilah *offline* pada atenuasi delegasi dalam Tugas Akhir ini tidak dimaknai sebagai proses yang sepenuhnya dilakukan tanpa internet, melainkan sebagai proses delegasi yang tidak memerlukan keterlibatan pasien. Setelah pasien memberikan akses awal, aktor yang sah dapat mengajukan delegasi terbatas melalui *client*, sedangkan *PRE* memverifikasi dan membentuk hak akses turunan berdasarkan hak akses induk yang masih berlaku.

Pemilihan mekanisme ini memberikan batas tanggung jawab yang lebih jelas. *client* berperan sebagai inisiator delegasi dan pencatat delegasi pada penyimpanan *on-chain*. *Server PRE* berperan sebagai pembentuk hak akses turunan. Penyimpanan *on-chain* mencatat riwayat delegasi dan bukti akses. Dengan pembagian tersebut, proses pelayanan tidak perlu selalu melibatkan pasien, tetapi pembentukan hak akses turunan tetap melewati validasi *PRE* sebelum dicatat dan digunakan.

III.2.2 Analisis Pemilihan Token Otorisasi

Berdasarkan masalah yang telah diidentifikasi pada Subbab III.1.2, mekanisme otorisasi pada DecMed perlu mendukung kontrol akses granular dan memiliki mekanisme atuneasi. Selain itu, pada Subbab III.2.1 dibahas bahwa mekanisme delegasi antaraktor perlu mendukung atenuasi *offline*. Oleh karena itu, diperlukan analisis perbandingan terhadap beberapa pendekatan token otorisasi untuk menentukan teknologi token otorisasi yang paling sesuai dengan kebutuhan sistem.

Alternatif token otorisasi yang dibandingkan pada analisis ini adalah JWT, PASETO, Macaroons, dan Biscuit. JWT dipilih sebagai pembanding karena merupakan bentuk token yang digunakan pada rancangan DecMed awal dan umum digunakan untuk merepresentasikan klaim otorisasi. PASETO dipilih karena memiliki tujuan penggunaan yang serupa dengan JWT, tetapi dengan kriptografi yang *opinionated*. Macaroons dianalisis sebagai kandidat utama karena menyediakan mekanisme *caveat* yang memungkinkan atenuasi *offline*. Biscuit dianalisis sebagai pembanding karena mendukung verifikasi terdesentralisasi dan atenuasi *offline*, tetapi menggunakan pendekatan yang lebih ekspresif melalui aturan berbasis *logic language*.

Kriteria evaluasi antar token diturunkan dari kebutuhan sebagai berikut:

1. Representasi batasan akses granular

Menilai apakah token dapat membawa batasan akses yang dibutuhkan agar token benar-benar dapat mengekspresikan pembatasan akses sesuai kebutuhan.

2. Atenuasi *offline*

Menilai apakah pemegang token dapat menambahkan batasan baru tanpa harus melibatkan pasien dan menerbitkan token baru. Sesuai dengan solusi dari Subbab III.2.1.

3. Jaminan token turunan tidak memperluas hak akses

Menganalisis jenis kunci kriptografi yang digunakan dan seberapa kuat sistem delegasi token mencegah token turunan digunakan untuk mengakses lebih banyak atau lebih luas daripada token induknya.

Tabel III.6: Perbandingan Pendekatan Token Otorisasi

Kriteria	JWT	PASETO	Macaroons	Biscuit
Representasi batasan akses granular	Melalui klaim	Melalui klaim	Melalui <i>caveats</i>	Melalui <i>fact, rule</i> , dan <i>check</i>
Atenuasi <i>offline</i>	Tidak didukung secara bawaan	Tidak didukung secara bawaan	Didukung melalui penambahan <i>caveat</i>	Didukung melalui penambahan blok pembatasan
Jaminan token turunan tidak memperluas hak akses	Bergantung pada penerbitan token baru atau logika <i>server</i>	Bergantung pada penerbitan token baru atau logika <i>server</i>	Ditegakkan melalui verifikasi seluruh <i>caveat</i>	Ditegakkan melalui blok dan pemeriksaan kriptografis
Model verifikasi dan pengelolaan kunci	Dapat menggunakan kunci simetris atau asimetris	Dapat menggunakan mode lokal atau publik	Berbasis kunci simetris	Berbasis kunci asimetris
Kesesuaian ekspresivitas dengan kebutuhan DecMed	Cukup untuk klaim statis, kurang sesuai untuk delegasi lokal	Cukup untuk klaim statis, kurang sesuai untuk delegasi lokal	Sesuai untuk batasan berbasis atribut yang langsung	Lebih ekspresif, tetapi melebihi kebutuhan aturan akses yang relatif langsung
Kompleksitas integrasi	Rendah	Rendah	Sedang	Tinggi

4. Model verifikasi dan pengelolaan kunci

Mengkaji bagaimana masing-masing token diverifikasi dan bagaimana pengelolaan kuncinya.

5. Kesesuaian ekspresivitas dengan aturan akses DecMed

DecMed membutuhkan token yang dapat mengekspresikan batasan akses secara langsung sebagai atribut pada token itu sendiri.

6. Kompleksitas integrasi ke PRE

membahas tingkat kesulitan integrasi ke dalam sistem PRE sebagai penegak otoritas akses.

Tabel III.6 menunjukkan perbandingan antar token otorisasi berdasarkan kriteria evaluasi.

JWT dan PASETO dapat digunakan untuk membawa informasi otorisasi dalam bentuk klaim, seperti identitas subjek, cakupan akses, dan waktu kedaluwarsa. Namun, keduanya tidak menyediakan mekanisme bawaan untuk menambahkan batasan baru pada token yang sudah diterbitkan. Akibatnya, delegasi hak akses yang lebih sempit tetap membutuhkan penerbitan token baru oleh pihak penerbit atau mekanisme tambahan di sisi *server*. Karakteristik tersebut kurang sesuai dengan kebutuhan delegasi *offline* pada Tugas Akhir ini.

Macaroons dan Biscuit lebih dekat dengan kebutuhan karena keduanya mendukung atenuasi *offline*. Perbedaan utama keduanya terletak pada model pembatasan

dan model verifikasi. Macaroons menggunakan *caveats* untuk menambahkan batasan pada token, sedangkan Biscuit menggunakan blok berisi *fact*, *rule*, dan *check*. Model Biscuit memberikan ekspresivitas yang lebih tinggi, tetapi kebutuhan pembatasan akses pada DecMed masih dapat direpresentasikan melalui klaim pada token, seperti identitas pasien, identitas aktor, cakupan data, aksi, dan masa berlaku. Pembatasan tersebut hanya memanfaatkan *fact* saja, karena pemeriksaan otoritas dari klaim pada token sudah cukup ditangani oleh *server* PRE. Oleh karena itu, penggunaan Biscuit dinilai menambah kompleksitas integrasi yang tidak diperlukan untuk ruang lingkup Tugas Akhir ini.

Dari sisi model verifikasi, Macaroons lebih sesuai dengan rancangan DecMed karena proses verifikasi token tetap dipusatkan pada *Server* PRE sebagai titik penegakan otorisasi. Dengan model tersebut, kunci verifikasi tidak perlu disebar ke banyak layanan. Biscuit lebih sesuai untuk arsitektur yang membutuhkan verifikasi token oleh banyak komponen berbeda menggunakan kunci publik. Pada rancangan DecMed, kebutuhan tersebut tidak menjadi fokus karena seluruh akses data RME tetap melewati PRE.

Berdasarkan perbandingan tersebut, Macaroons dipilih sebagai mekanisme token otorisasi pada DecMed karena memberikan keseimbangan antara representasi batasan akses granular, dukungan delegasi terbatas melalui atenuasi *offline*, dan kompleksitas integrasi yang sesuai dengan ruang lingkup Tugas Akhir. Melalui mekanisme *caveats*, token dapat membawa batasan akses dan pemegang token dapat menambahkan batasan baru secara lokal untuk mempersempit hak akses sebelum token didelegasikan kepada aktor lain.

III.2.3 Analisis Segmentasi Data RME

Masalah granularitas tidak hanya berada pada token, tetapi juga pada struktur data yang dilindungi. Sesuai analisis pada Subbab III.1.1.2, jika data RME disimpan sebagai satu *encrypted blob*, maka pemberian akses terhadap blob tersebut akan membuka seluruh isi data meskipun aktor hanya membutuhkan sebagian informasi. Oleh karena itu, implementasi pemrosesan data pada DecMed perlu disesuaikan untuk memenuhi kebutuhan akses granular.

Alternatif pertama adalah melakukan pemfilteran setelah data klinis diambil dari

penyimpanan *off-chain*. Pada pendekatan ini, data tetap disimpan sebagai satu *encrypted blob*, kemudian sistem memilih bagian data yang boleh ditampilkan berdasarkan hak akses pengguna. Pendekatan ini kurang sesuai dengan karakteristik DecMed. IPFS tidak menyediakan mekanisme pencarian atau pemfilteran isi data secara langsung. Selain itu, DecMed menyimpan data dalam bentuk terenkripsi, sehingga isi data baru dapat diproses setelah didekripsi. Jika pemfilteran dilakukan di *client*, maka keseluruhan *plaintext* data klinis sudah terbuka terlebih dahulu sebelum batas akses diterapkan.

Alternatif yang dipilih adalah melakukan segmentasi data klinis sebelum data disimpan pada penyimpanan *off-chain*. Pada pendekatan ini, data klinis dipisahkan menjadi beberapa segmen. Setiap segmen memiliki metadata yang disimpan pada penyimpanan *on-chain*. Metadata tersebut digunakan oleh sistem untuk memilih segmen yang sesuai dengan cakupan akses sebelum data diambil dari IPFS. Dengan cara ini, proses pengambilan data menjadi lebih terarah karena sistem hanya mengambil segmen yang relevan dengan hak akses pengguna.

Penentuan kategori segmentasi tidak dilakukan secara arbitrer. Opsi pertama adalah mengikuti pembagian data pada penelitian terkait yang dibahas pada Subbab II.9.3. Namun, pembagian tersebut kurang sesuai dijadikan dasar segmentasi pada Tugas Akhir ini karena ditujukan untuk pembuktian konseptual, bukan untuk merepresentasikan struktur data pelayanan medis. Oleh karena itu, rancangan segmentasi mengacu pada KMK 1423/2022 tentang Pedoman Variabel dan Metadata pada Penyelenggaraan Rekam Medis Elektronik yang telah dibahas pada Subbab II.1.2.

Berdasarkan pedoman tersebut, data RME dapat dibedakan berdasarkan *dataset* pelayanan. Dalam cakupan Tugas Akhir ini, *dataset category* yang digunakan meliputi:

- Rawat Jalan
- Rawat Inap
- Laboratorium
- Apotek

Pembagian berdasarkan *dataset* pelayanan dipilih karena setiap layanan memiliki

jenis data dan kebutuhan akses yang berbeda. Sebagai contoh, petugas laboratorium membutuhkan akses tulis terhadap *dataset* laboratorium, sedangkan apoteker membutuhkan akses tulis terhadap *dataset* apotek. Dengan pemisahan ini, sistem dapat membedakan konteks layanan sebelum menentukan segmen data yang dapat dibaca atau ditulis oleh aktor.

Selain berdasarkan *dataset*, data klinis juga disegmentasi berdasarkan kategori fungsi klinis. *Function category* digunakan untuk membedakan fungsi data di dalam suatu *dataset*, seperti anamnesis, pemeriksaan fisik, pemeriksaan penunjang, diagnosis, persetujuan tindakan, atau terapi. Metadata fungsi klinis digunakan untuk menentukan apakah suatu segmen data relevan dengan peran dan kebutuhan akses tenaga medis.

Kombinasi *dataset category* dan *function category* dipilih agar segmentasi tidak terlalu kasar seperti pemisahan per dokumen penuh, tetapi juga tidak terlalu rinci hingga tingkat *field*. Segmentasi hingga tingkat *field* akan membuat kontrol akses bergantung pada setiap elemen data yang tersimpan. Pendekatan tersebut tidak dipilih karena kebutuhan akses pada rancangan ini dinilai berdasarkan konteks layanan dan fungsi klinis, bukan berdasarkan setiap *field* secara terpisah.

Sebagai contoh, Formulir Umum / Asesmen Awal Rawat Jalan dapat dipetakan ke fungsi klinis seperti anamnesis, pemeriksaan fisik, dan pemeriksaan psikologi. Sementara itu, Pemeriksaan Spesialistik dapat dipetakan ke fungsi klinis seperti riwayat penggunaan obat, rencana rawat, instruksi medik dan keperawatan, pemeriksaan penunjang, diagnosis, persetujuan tindakan, dan terapi. Pemetaan lengkap kategori fungsi klinis dapat dilihat pada Lampiran A.

Melalui pendekatan ini, data *off-chain* tidak lagi diperlakukan sebagai satu objek besar, tetapi sebagai kumpulan segmen klinis dengan metadata yang terdefinisi. Segmentasi tersebut menyediakan objek penegakan yang lebih jelas bagi token otorisasi karena batasan akses pada token dapat dicocokkan langsung dengan *dataset category* dan *function category* pada metadata *on-chain*.

III.2.4 Analisis Penegakan Otoritas pada *Server* PRE

Pada DecMed, PRE digunakan untuk mengubah data terenkripsi dari pasien ke penerima akses tanpa sekalipun membuka *plaintext*. Pada implementasi

DecMed sebelumnya, PRE berfokus melakukan verifikasi token dan menjalankan proses *re-encryption*. *Server* PRE dipilih sebagai titik penegakan otoritas karena komponen ini berada pada jalur kritis sebelum data terenkripsi diproses. Pengguna tidak memperoleh *plaintext* langsung dari penyimpanan *off-chain*, melainkan membutuhkan proses *re-encryption* yang dikendalikan oleh PRE. Oleh karena itu, PRE dapat berperan sebagai titik penegakan kebijakan akses yang memverifikasi token dan menolak permintaan sebelum data diambil dari IPFS apabila otorisasi tidak valid.

III.2.4.1 Analisis Verifikasi Aktor dan Operasi Tulis

Pada DecMed sebelumnya, *role* *MedicalPersonnel* digunakan untuk merepresentasikan tenaga medis secara umum, dan memberikan hak untuk mengakses data klinis pasien. Namun, *role* tersebut belum cukup untuk membedakan dokter, perawat, petugas laboratorium, dan apoteker. Padahal, keempat aktor tersebut memiliki kewenangan tulis yang berbeda. Sebagai contoh, dokter dapat menulis diagnosis dan terapi sedangkan petugas laboratorium hanya dapat menulis hasil laboratorium.

Oleh karena itu, diperlukan verifikasi tambahan terhadap *subrole* tenaga medis, terutama untuk operasi tulis. *Server* PRE tetap perlu memastikan bahwa aktor aktif memiliki *subrole* yang sesuai dengan *dataset category* dan *function category* yang akan ditulis. Verifikasi ini mencegah kondisi ketika aktor membawa token dengan izin tulis, tetapi secara kewenangan profesional tidak berhak memperbarui segmen data tertentu.

III.2.4.2 Analisis Verifikasi Bukti Identitas Aktor Aktif

Pada DecMed sebelumnya, proses otorisasi berfokus pada pemeriksaan klaim yang terdapat pada token. Namun, mekanisme tersebut belum memastikan bahwa pengirim *request* merupakan aktor yang identitasnya tercantum pada token. Kondisi ini menimbulkan kebutuhan verifikasi tambahan, yaitu verifikasi bukti identitas aktor aktif sebagaimana dirumuskan pada KNF-TA-06.

Identitas pengguna pada DecMed direpresentasikan melalui *wallet* yang dibuat saat pendaftaran akun. *Wallet address* menjadi identifikator unik yang dapat membedakan setiap pengguna pada sistem. Oleh karena itu, solusi yang diusulkan

adalah menggunakan *wallet* sebagai dasar verifikasi identitas aktor aktif.

Verifikasi dilakukan dengan meminta aktor aktif membuktikan kepemilikan *wallet* yang sesuai dengan klaim pada token. Dengan mekanisme ini, *request* tidak hanya membawa token otorisasi, tetapi juga membawa bukti bahwa pengirim *request* menguasai *wallet* dari aktor yang sedang menggunakan hak akses tersebut. Hasil verifikasi ini kemudian digunakan oleh *Server PRE* untuk menentukan apakah aktor aktif berwenang melanjutkan operasi akses terhadap data RME.

III.2.4.3 Analisis Revokasi dan Audit Delegasi

Delegasi akses menyebabkan token tidak lagi berdiri sendiri, tetapi membentuk relasi antara token induk dan token turunan. Oleh karena itu, mekanisme revokasi pada DecMed perlu diperluas agar pencabutan token induk dapat berdampak pada token turunan yang berasal dari token tersebut. Tanpa pencabutan berantai, aktor yang memperoleh token turunan masih dapat menggunakan akses meskipun akses asalnya telah dicabut oleh pasien.

Revokasi berantai diperlukan untuk mempertahankan kendali pasien sebagai pemilik data. Ketika pasien mencabut akses awal, sistem harus dapat mengenali bahwa token-token turunan yang berada dalam rantai delegasi yang sama tidak lagi memiliki dasar kewenangan yang sah. Dengan demikian, pemeriksaan revokasi tidak cukup hanya dilakukan terhadap token yang dikirim pada saat permintaan akses, tetapi juga perlu mempertimbangkan hubungan token tersebut dengan token asalnya.

Selain revokasi, delegasi juga membutuhkan audit yang mencatat hubungan antaraktor. Audit tidak cukup hanya mencatat bahwa suatu segmen diakses, tetapi juga perlu mencatat sumber kewenangan akses tersebut, termasuk aktor pemberi delegasi, aktor penerima delegasi, waktu delegasi, cakupan akses, dan token yang digunakan. Informasi ini diperlukan agar pasien dan sistem dapat menelusuri bagaimana akses terhadap data RME terbentuk selama proses pelayanan.

III.2.5 Pemetaan Masalah terhadap Solusi

Setelah pendekatan solusi dianalisis, setiap solusi pada bagian ini diberi ID S-TA untuk menjaga ketertelusuran terhadap masalah yang telah diidentifikasi pada Tabel

Tabel III.7: Daftar Solusi yang Diusulkan Berdasarkan Hasil Analisis

ID	Nama Solusi	Masalah Terkait
S-TA-01	Token otorisasi berbasis Macaroons dengan <i>caveats</i>	M-TA-01, M-TA-04
S-TA-02	Segmentasi data RME berdasarkan <i>dataset_category</i> dan <i>function_category</i>	M-TA-02
S-TA-03	Delegasi akses melalui atenuasi <i>offline</i> Macaroons	M-TA-03, M-TA-04
S-TA-04	Penegakan akses pada <i>Server PRE</i> berbasis <i>effective capability</i> , <i>wallet proof</i> , dan metadata segmen	M-TA-01, M-TA-02, M-TA-04, M-TA-05, M-TA-07
S-TA-05	Validasi subrole tenaga medis terhadap dataset untuk operasi tulis	M-TA-05
S-TA-06	Pencabutan akses berantai dan audit log akses dan delegasi	M-TA-06

III.2. Daftar solusi utama ditunjukkan pada Tabel III.7.

III.3 Rancangan Solusi

Berdasarkan hasil analisis solusi pada Subbab III.2, rancangan solusi difokuskan pada pengembangan mekanisme kontrol akses granular dan delegasi terbatas pada DecMed melalui integrasi Macaroons. Rancangan ini mempertahankan fondasi utama DecMed, yaitu IOTA sebagai penyimpanan *on-chain*, IPFS sebagai penyimpanan *off-chain*, dan PRE sebagai mekanisme distribusi akses terhadap data terenkripsi. Perancangan ini mencakup arsitektur sistem, token otorisasi berbasis Macaroons, segmentasi data *off-chain*, penegakan otoritas pada *Server PRE*, validasi subrole tenaga medis, pencabutan akses, audit, alur akses data, dan delegasi berantai dengan atenuasi *offline*.

III.3.1 Rancangan Arsitektur Solusi

Arsitektur solusi mempertahankan keseluruhan arsitektur kerangka kerja DecMed. Tiga *client* utama DecMed, yaitu *Patient Client*, *Hospital Client*, dan *Ministry Client* tetap ada namun disesuaikan sesuai kebutuhan Tugas Akhir ini. *Patient Client* digunakan pasien untuk menerbitkan token akses awal. *Hospital Client* digunakan oleh petugas rekam medis, dokter, perawat, apoteker dan petugas laboratorium untuk mengakses atau memperbarui data sesuai kewenangannya. *Ministry Client* tetap berada pada ruang lingkup manajemen fasyankes sebagaimana

rancangan DecMed awal.

Perubahan utama berada pada *server* PRE, token otorisasi berbasis Macaroons, dan implementasi *Move smart contract*. Pada rancangan ini, token akses diganti dengan Macaroon yang membawa daftar *caveats*. Setiap *request* akses data harus menyertakan Macaroon. *Server* PRE kemudian memverifikasi hak akses, lalu menjalankan proses *re-encryption* untuk operasi baca dan enkripsi data ketika operasi tulis. Implementasi *Move smart contract* disesuaikan untuk menyimpan metadata yang diperlukan oleh mekanisme segmentasi dan kontrol akses.

Pada rancangan ini, *server* PRE juga berperan sebagai komponen pembentuk token Macaroon turunan pada proses delegasi. *Client delegator* mengirimkan permintaan atenuasi kepada *server* PRE. *Server* PRE memverifikasi *request* delegasi. Apabila valid, *server* PRE mengembalikan token turunan untuk diproses lebih lanjut oleh *client delegator* untuk dicatat pada penyimpanan *on-chain*.

Skema penyimpanan *on-chain* dan *off-chain* disesuaikan agar dapat mendukung kontrol akses granular sesuai hasil analisis solusi pada Subbab III.2.3. Penyimpanan data pada sistem yang diusulkan dibagi menjadi dua lapisan. Penyimpanan pada lapisan *on-chain* disesuaikan dengan mengubah implementasi dari *Move smart contract*. Sedangkan penyimpanan pada lapisan *off-chain* disesuaikan dengan mengubah mekanisme penyimpanan data yang diproses pada *hospital client*.

Gambaran arsitektur sistem secara keseluruhan dapat dilihat pada gambar III.1. Warna merah digunakan untuk merepresentasikan komponen yang diubah dari rancangan DecMed awal. Legenda dari diagram arsitektur ini dapat dilihat pada lampiran F.

III.3.2 Rancangan Segmentasi Data RME

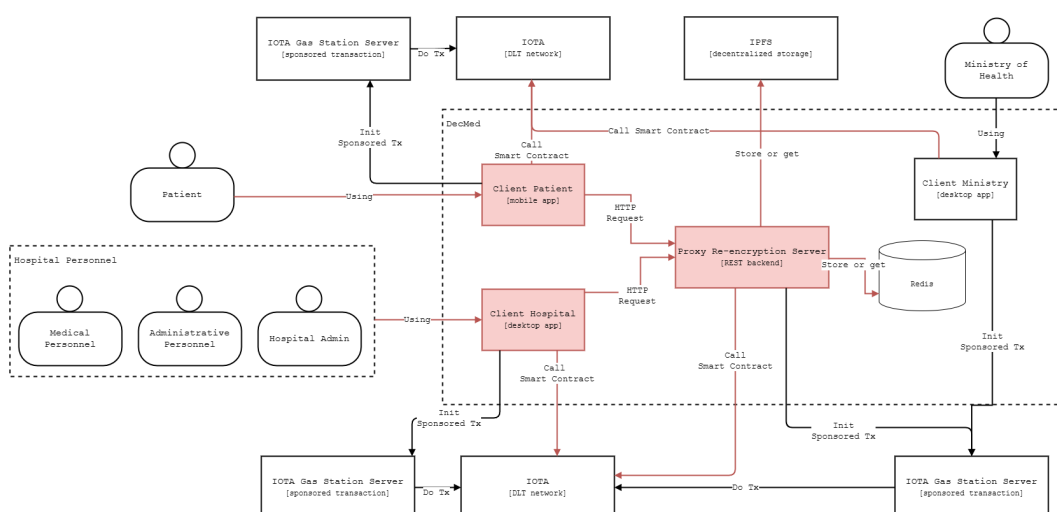
Sesuai dengan analisis solusi pada Subbab III.2.5, rancangan segmentasi data RME memecah data klinis menjadi beberapa segmen terenkripsi. Setiap segmen dibentuk berdasarkan kombinasi *dataset category* dan *function category*. *Dataset category* merepresentasikan konteks layanan, seperti Rawat Jalan, Rawat Inap, Laboratorium, dan Apotek. *Function category* merepresentasikan fungsi data di dalam *dataset*, seperti anamnesis, diagnosis, terapi, pemeriksaan penunjang, atau *dispensing*. Kombinasi kedua kategori tersebut menjadi unit data terkecil yang

<i>Dataset Category</i>	<i>Enum Value</i>
Rawat Jalan	RAWAT_JALAN
Rawat Inap	RAWAT_INAP
Laboratorium	LABORATORIUM
Apotek	APOTEK

Tabel III.8 menunjukkan *dataset category* yang digunakan pada rancangan ini.

Tabel III.9 menunjukkan *function category* yang digunakan pada implementasi.

Pada rancangan ini, *function category* tidak selalu dibentuk dari setiap subkelompok metadata secara langsung. Beberapa subkelompok metadata dapat digabung ke dalam satu *function category* selama masih memiliki fungsi pelayanan dan kebutuhan akses yang sama. Sebagai contoh, subkelompok pada *dataset*



51

Tabel III.9: Tabel Pemetaan *Function Category*

<i>Function Category</i>	<i>Enum Value</i>
Dokumentasi Administratif	ADMINISTRATIVE_GENERAL
Anamnesis	ANAMNESIS
Pemeriksaan Fisik	PEMERIKSAAN_FISIK
Pemeriksaan Psikologis, Sosial Ekonomi, Spiritual	PEMERIKSAAN_PSIKOLOGIS
Riwayat Penggunaan Obat	RIWAYAT_PENGUNAAN_OBAT
Rencana Rawat	RENCANA_RAWAT
Perencanaan Pemulangan Pasien	PERENCANAAN_PEMULANGAN
Instruksi Medik dan Keperawatan	INSTRUKSI_MEDIK_DAN_KEPERAWATAN
Pemeriksaan Penunjang	PEMERIKSAAN_PENUNJANG
Diagnosis	DIAGNOSIS
<i>Informed Consent</i>	INFORMED_CONSENT
Terapi	TERAPI
Laboratorium	LABORATORIUM
Peresepan	PERESEPAN
<i>Dispensing</i>	DISPENSING

Laboratorium, seperti permintaan pemeriksaan, spesimen klinis, hasil pemeriksaan, serta interpretasi dan validasi hasil, dikelompokkan ke dalam *function category* LABORATORIUM karena seluruhnya merepresentasikan satu rangkaian fungsi pemeriksaan laboratorium. Sementara itu, *dataset* Apotek dibedakan menjadi PERESEPAN dan DISPENSING karena keduanya merepresentasikan dua fungsi yang berbeda, yaitu pencatatan resep dan proses penyiapan serta penyerahan obat.

Secara penyimpanan, satu entri metadata *on-chain* menunjuk pada satu segmen data terenkripsi di penyimpanan *off-chain*. Rincian skema metadata *on-chain* disajikan pada Tabel G.1, sedangkan skema objek data segmen *off-chain* disajikan pada Tabel G.2.

Rancangan ini membuat *Server* PRE dapat menegakkan akses pada tingkat segmen. Ketika aktor meminta akses terhadap data RME, *Server* PRE mencocokkan *caveats* pada token Macaroon dengan metadata segmen yang diminta. Sebagai contoh, token yang hanya mengizinkan akses tulis terhadap *dataset* Laboratorium tidak dapat digunakan untuk menulis segmen pada *dataset* Rawat Jalan.

III.3.3 Rancangan Token Otorisasi Berbasis Macaroons

Macaroons digunakan sebagai pengganti token JWT DecMed sesuai dengan hasil analisis solusi pada Subbab III.2.2. Token Macaroon tetap mengikuti model CapBAC karena token berperan sebagai bukti kapabilitas yang dibawa oleh

pemegang akses.

Pada rancangan ini, *Server PRE* bertindak sebagai *issuer* token Macaroon awal dan memegang *RootKey* yang digunakan untuk proses penandatanganan token. *RootKey* disimpan pada *environment variable* *Server PRE* dan tidak pernah diberikan kepada *client*. Token awal diterbitkan oleh *Server PRE* setelah terdapat persetujuan dari pasien sebagai pemilik data. Dengan demikian, sifat *patient-centric* tetap dipertahankan karena penerbitan akses awal tetap bergantung pada persetujuan pasien, sedangkan otoritas kriptografis penerbitan token berada pada *Server PRE*. Setelah token awal diterbitkan, pemegang token dapat melakukan atenuasi melalui *client* miliknya dan mengirimkan *request* untuk mengatenuasi Macaroons ke *server PRE*.

Karena *caveat* pada Macaroons bersifat *append-only*, *caveat* pada token induk tidak dapat dihapus atau diganti pada token turunan. Oleh karena itu, rancangan *caveat* harus dibuat dalam bentuk batasan yang bersifat monoton. Setiap *caveat* baru hanya dapat mempersempit hak akses dan tidak dapat memperluas hak akses. Dengan demikian, token turunan yang dibuat pada *client* tetap aman karena seluruh batasan pada token induk tetap ikut terbawa dan diverifikasi oleh *server PRE*.

Berdasarkan implementasi DecMed sebelumnya, token memiliki *claim purpose* dan terdapat dua buah token untuk operasi baca atau tulis. Implementasi *Move smart contract*, *client*, dan *PRE* juga mencatat akses yang telah diberikan kepada tenaga medis untuk 2 *purpose*, yaitu Read dan Update. Pada pengembangan ini, pola 2 buah token masih dipertahankan dengan membedakan nilai *caveat purpose*, yaitu Read atau Update. Hal ini dilakukan sehingga rancangan token Macaroon tetap kompatibel dengan implementasi DecMed.

Pada rancangan ini, tidak ada *caveat* yang digunakan sebagai penanda pemegang token saat ini. Hal ini disebabkan karena nilai pemegang token akan berubah ketika token didelegasikan, sedangkan Macaroon tidak mendukung penghapusan atau penggantian *caveat* lama. Apabila token induk memiliki *holder_address* = 0xPETUGAS_REKAM_MEDIS dan token turunan menambahkan *holder_address* = 0xDOKTER, maka token tersebut akan memiliki dua *caveat* identitas yang saling kontra. Kondisi tersebut dapat menyebabkan token tidak dapat digunakan karena satu permintaan akses tidak mungkin berasal dari dua alamat pemegang yang

berbeda.

Sebagai gantinya, rancangan ini menggunakan pendekatan rantai delegasi dengan *caveat* *root_subject*, *delegated_by*, dan *delegated_to*. *root_subject* menyatakan aktor penerima token awal dari pasien, sedangkan *delegated_by* dan *delegated_to* digunakan untuk merepresentasikan proses delegasi pada token turunan. Dengan pendekatan ini, sistem dapat menambahkan informasi delegasi baru tanpa perlu mengubah *caveat* yang sudah ada sebelumnya. Untuk memastikan delegasi berjalan hanya pada satu fasyankes, *caveat* *hospital_cid* digunakan untuk mengikat token terhadap ID rumah sakit.

Selain informasi delegasi, token juga memuat batasan akses terhadap data RME tertentu. *Caveat* *patient_address* digunakan untuk mengikat token terhadap data RME pasien tertentu. *Caveat* *related_rme_id* digunakan untuk membatasi token terhadap satu episode pelayanan. Kedua *caveat* tersebut memastikan bahwa token tidak dapat digunakan untuk mengakses data pasien lain atau episode pelayanan lain yang tidak termasuk dalam cakupan token.

Namun *caveat* *related_rme_id* hanya berlaku untuk token Update dan tidak berlaku untuk token Read. Hal ini karena pada skenario alur klinis pada pelayanan kesehatan di sebuah fasyankes, seorang dokter pada umumnya membutuhkan akses ke semua data RME pasien untuk menentukan tindakan medis yang tepat. Namun, untuk aksi Update pada data RME, dokter hanya perlu mengakses data RME yang berkaitan dengan episode pelayanan yang sedang berlangsung.

Pada rancangan ini, *related_rme_id* menggunakan format RME-*{sequence_6_digit}*, misalnya RME-000001. Format ini mengikuti implementasi DecMed sebelumnya dan dipilih karena sudah cukup untuk kebutuhan prototipe.

Mekanisme pembatasan kategori data dilakukan dengan metode *whitelisting*. Token harus secara eksplisit menyatakan *dataset category* dan *function category* yang dapat diakses oleh pemegang token. Rancangan ini menggunakan *caveat* *read_dataset_in*, *write_dataset_in*, *read_function_in*, dan *write_function_in*. Keempat *caveat* tersebut disesuaikan dengan *metadata* segmen RME yang disimpan pada penyimpanan *on-chain*, yaitu *dataset_category* dan *function_category*. Dengan demikian, *Server* PRE

dapat memvalidasi apakah segmen yang diminta termasuk dalam cakupan akses yang diizinkan oleh token.

Pembatasan waktu akses dilakukan dengan *caveat expires_before*. *Caveat* ini menyatakan batas waktu maksimum token dapat digunakan. Pada token turunan, nilai *expires_before* harus sama dengan atau lebih awal daripada token induk. Pada proses verifikasi, *caveat expires_before* yang diambil adalah nilai paling awal dari seluruh *caveat expires_before*.

Pembatasan jumlah delegasi dilakukan dengan *caveat max_delegation_depth*. Nilai ini menyatakan jumlah maksimum delegasi yang masih diperbolehkan dari token terkait. Token awal dapat memiliki nilai *max_delegation_depth* = 2 yang menandakan bahwa token Macaroon tersebut dapat diturunkan 2 kali. Pada saat membuat token turunan, token turunan dapat ditambahkan *caveat max_delegation_depth* = 1, yang berarti kemampuan menurunkan token lebih lanjut telah berkurang 1. *Caveat* ini dapat divalidasi lebih lanjut lagi dengan memanfaatkan *caveat delegated_by* dan *delegated_to* untuk berapa banyak turunan yang telah terjadi.

Untuk membantu verifikasi token yang sudah direvokasi, *caveat parent_token_hash* digunakan untuk mengikat token turunan dengan token induk. Nilai *parent_token_hash* adalah *hash* dari token induk. Rincian lebih lanjut tentang mekanisme revokasi token Macaroon dijelaskan pada Subbab IV.2.4.5.

Rancangan *caveats* yang digunakan pada token Macaroon ditunjukkan pada Tabel III.10.

Sebagai contoh, pasien dapat memberikan token awal kepada petugas rekam medis dengan cakupan akses yang cukup untuk mengoordinasikan pelayanan pada satu episode RME tertentu. Petugas rekam medis diberikan token awal dengan cakupan yang luas karena berperan sebagai koordinator awal pelayanan. Meskipun cakupannya luas, namun aksi yang dilakukan oleh petugas rekam medis dapat dibatasi dengan penegakan otoritas berbasis *role* yang akan dijelaskan pada Subbab III.3.4.2. Contoh *caveat* token awal untuk petugas rekam medis disajikan pada Kode H.1 dan Kode H.2.

Pada saat token turunan digunakan oleh tenaga medis, *Server PRE* akan

Tabel III.10: Rancangan *Caveats* pada Token Macaroon

<i>Caveat</i>	Fungsi	Pada Token
purpose	Menyatakan tujuan penggunaan token (Read atau Update).	Keduanya
patient_address	Mengikat token terhadap data RME milik pasien tertentu.	Keduanya
related_rme_id	Menentukan episode pelayanan atau data RME yang dapat diakses.	Update
root_subject	Menyatakan aktor penerima token awal dari pasien.	Keduanya
delegated_by	Menyatakan aktor yang melakukan delegasi token.	Keduanya
delegated_to	Menyatakan aktor penerima token turunan.	Keduanya
read_dataset_in	Membatasi <i>dataset category</i> yang dapat dibaca.	Read
write_dataset_in	Membatasi <i>dataset category</i> yang dapat ditulis.	Update
read_function_in	Membatasi <i>function category</i> yang dapat dibaca.	Read
write_function_in	Membatasi <i>function category</i> yang dapat ditulis.	Update
expires_before	Membatasi masa berlaku token.	Keduanya
max_delegation_depth	Membatasi jumlah delegasi turunan yang masih diperbolehkan.	Keduanya
hospital_cid	Mengikat token dengan rumah sakit tertentu.	Keduanya
parent_token_hash	Mengikat token turunan dengan token induk untuk proses revokasi.	Keduanya

memverifikasi seluruh *caveat* yang terdapat pada token, baik *caveat* dari token awal maupun *caveat* tambahan dari token turunan. Mekanisme verifikasi token pada *Server PRE* dijelaskan pada Subbab III.3.4.1.

Rancangan ini juga mendukung prinsip *least privilege*. Dokter dapat memiliki cakupan akses yang relatif luas untuk menjalankan tugas klinisnya, tetapi token turunan yang diberikan kepada petugas laboratorium atau apoteker dapat dibatasi hanya pada segmen yang relevan. Sebagai contoh, petugas laboratorium hanya dapat membaca segmen PEMERIKSAAN_PENUNJANG pada *dataset* RAWAT_JALAN dan menulis segmen LABORATORIUM pada *dataset* LABORATORIUM. Petugas tersebut tidak dapat membaca diagnosis, anamnesis, resep, atau segmen klinis lain yang tidak relevan dengan tugasnya.

III.3.4 Rancangan Penegakan Otoritas

Penegakan otoritas pada rancangan ini merujuk pada proses memastikan *request* akses terhadap data RME hanya dilayani apabila aktor yang mengajukan

permintaan memiliki kewenangan yang sah.

Bagian ini membahas tiga aspek utama penegakan otoritas pada *Server PRE*. Pertama, Subbab III.3.4.1 menjelaskan verifikasi *caveats* Macaroon untuk memastikan token masih valid. Kedua, Subbab III.3.4.2 membahas validasi tambahan terhadap *subrole* *MedicalPersonnel* agar akses tulis hanya dapat dilakukan pada *dataset category* yang sesuai dengan kewenangannya. Ketiga, Subbab III.3.4.4 menjelaskan rancangan mekanisme revokasi token Macaroon, termasuk revokasi agar token turunannya tidak dapat digunakan.

Sebagai gambaran, Gambar III.2 menunjukkan alur penegakan otoritas pada *Server PRE*. Secara garis besar, alur akses dimulai dengan aktor melalui *client* untuk mengirimkan permintaan akses ke *Server PRE*. *Server PRE* kemudian memverifikasi token Macaroon, memverifikasi status revokasi dari token, dan menentukan aktor aktif. Jika semua verifikasi berhasil, *Server PRE* melanjutkan proses ke 3 kemungkinan skenario, yaitu mengambil daftar RME yang dapat diakses, melakukan pembacaan data RME yang melibatkan proses *re-encryption*, dan melakukan penulisan data RME.

III.3.4.1 Verifikasi *Caveats* Macaroons

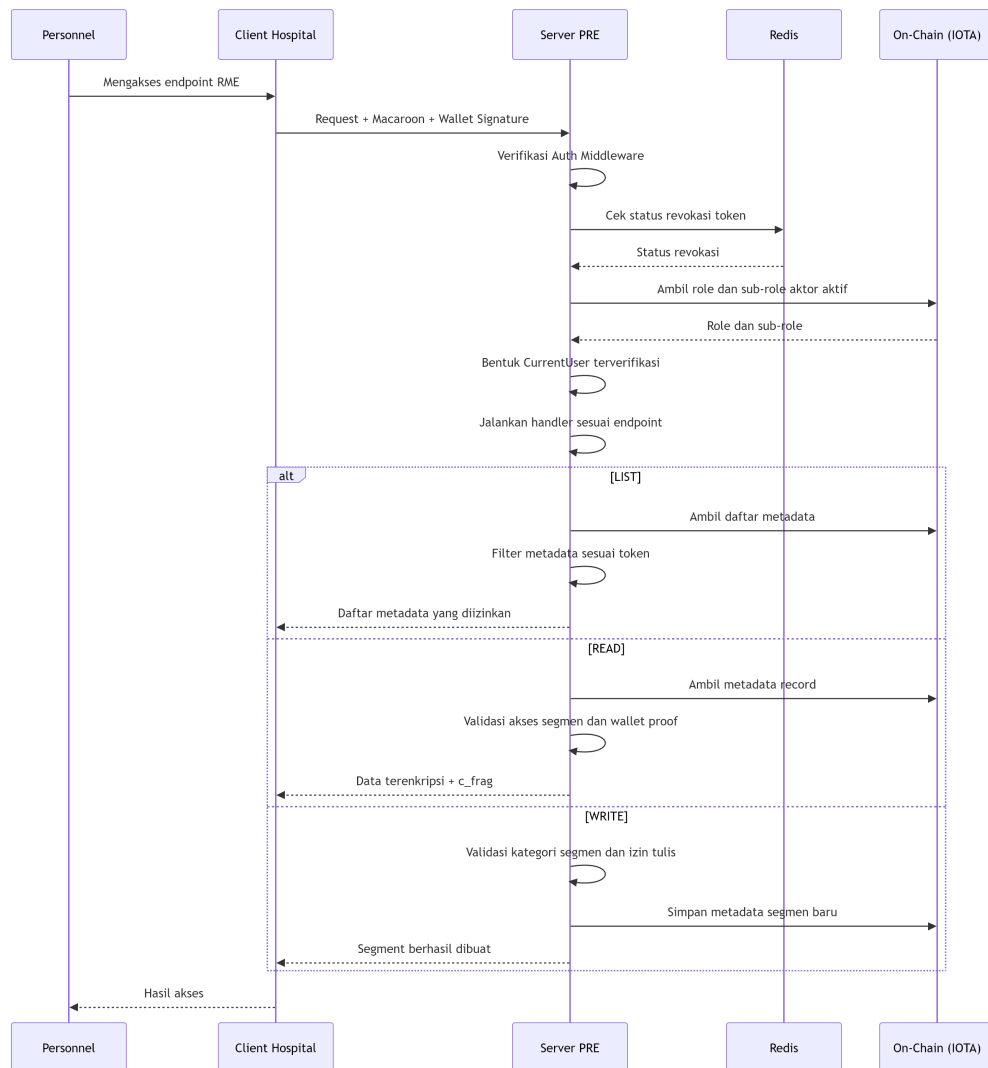
Server PRE bertindak sebagai titik penegakan kebijakan akses. *Server PRE* memverifikasi token Macaroon dan seluruh *caveats*. Jika terdapat *caveat* yang tidak dikenali atau diluar rancangan pada Tabel III.10, permintaan akses ditolak.

Rancangan langkah verifikasi token Macaroon untuk permintaan akses baca data RME adalah sebagai berikut:

1. *Server PRE* memverifikasi validitas token Macaroon melalui mekanisme *Chained-HMAC* menggunakan *RootKey*.
2. *Server PRE* memverifikasi bahwa *patient_address* dan *related_rme_id* pada token sesuai dengan data RME yang diminta.
3. *Server PRE* menghitung aktor aktif. Jika tidak terdapat *delegated_to*, maka aktor aktif adalah *root_subject*. Jika terdapat satu atau lebih *delegated_to*, maka aktor aktif adalah nilai *delegated_to* terakhir.
4. *Server PRE* memverifikasi tanda tangan *wallet* dari aktor aktif pada permintaan

akses.

5. *Server PRE* menghitung kedalaman delegasi berdasarkan jumlah `delegated_to` pada token dan memastikan nilainya tidak melebihi `max_delegation_depth`.
6. *Server PRE* memvalidasi seluruh batas waktu pada `expires_before` dan menggunakan batas waktu yang paling awal sebagai masa berlaku efektif token.
7. *Server PRE* menghitung cakupan akses efektif berdasarkan irisan seluruh `read_dataset_in`, `write_dataset_in`, `read_function_in`, dan `write_function_in`.
8. *Server PRE* mengambil metadata segmen dari IOTA dan mencocokkan `dataset_category` serta `function_category` segmen dengan cakupan akses efektif token.



Gambar III.2: Alur penegakan otoritas pada *Server PRE*

Tabel III.11: Pemetaan Hak Akses Update Berdasarkan subrole Tenaga Medis

<i>subrole</i>	<i>Dataset Category</i>	<i>Function Category</i>
Perawat	Rawat Jalan/Rawat Inap	Anamnesis, Pemeriksaan Fisik, Pemeriksaan Psikologis
Dokter	Rawat Jalan/Rawat Inap	Seluruh <i>function category</i>
Lab	Laboratorium	Laboratorium
Apoteker	Apotek	Peresepan dan Dispensing

9. Jika seluruh verifikasi berhasil, *Server PRE* melanjutkan proses pengambilan data terenkripsi dari IPFS dan melakukan *re-encryption*. Jika salah satu pemeriksaan gagal, permintaan akses ditolak.

III.3.4.2 Verifikasi *Role* dan *Subrole* Tenaga Medis untuk Akses *Dataset*

Dari hasil analisis pada Subbab III.2.4.1, diperlukan verifikasi tambahan terhadap *subrole* *MedicalPersonnel* agar akses tulis hanya dapat dilakukan pada *dataset category* yang sesuai dengan kewenangannya. Pada implementasi Tugas Akhir ini, terdapat 4 *subrole* turunan dari *role* *MedicalPersonnel*, yaitu dokter, perawat, petugas laboratorium, dan apoteker. Sebagaimana dijelaskan pada Subbab III.2.3, setiap *subrole* tersebut memiliki kebutuhan akses data yang berbeda-beda untuk melakukan akses jenis *update*. Pemetaan kebutuhan akses data untuk setiap *subrole* dapat dilihat pada Tabel III.11.

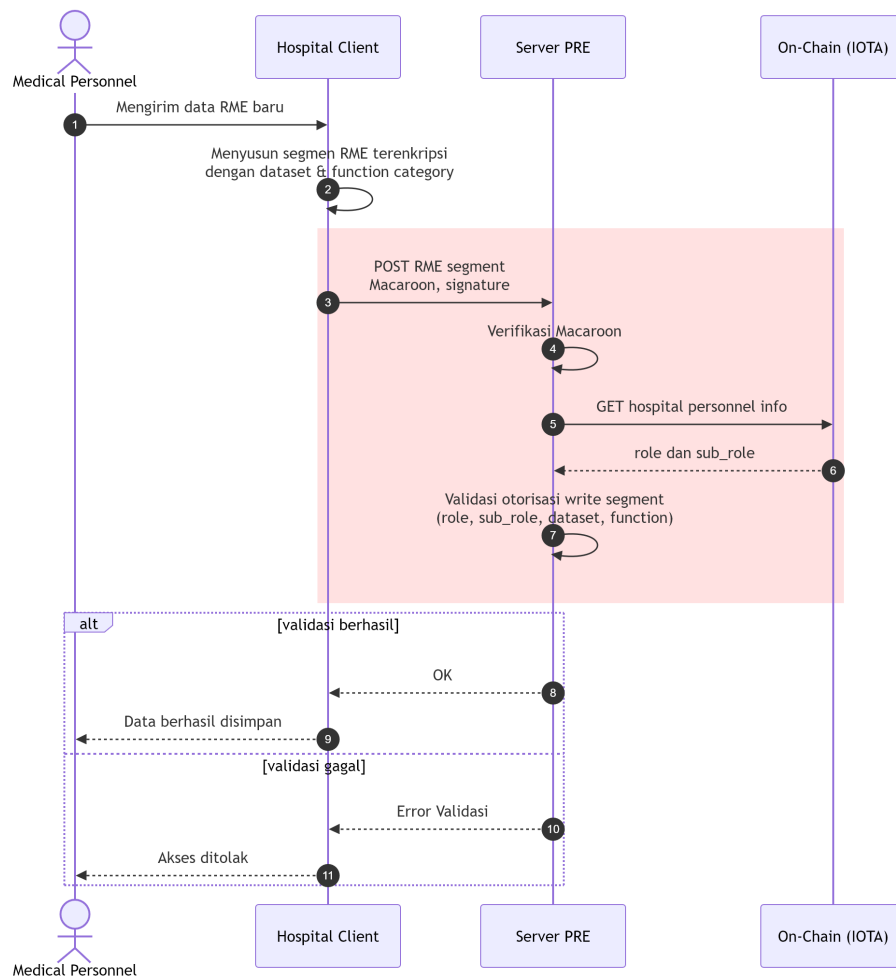
Berdasarkan pemetaan tersebut, terdapat verifikasi tambahan untuk *role* *MedicalPersonnel*, yaitu melakukan validasi apakah *subrole* pengguna yang saat ini mencoba melakukan aksi *write* memang sesuai dengan jenis *dataset* yang diakses. Alur tambahan tersebut ditunjukkan pada Gambar III.3. Pada alur ini, *Server PRE* terlebih dahulu mendapatkan informasi *subrole* tenaga medis melalui *smart contract*. Informasi tersebut kemudian digunakan untuk memvalidasi pasangan *role* dan *subrole* terhadap kategori *dataset* yang coba ditulis. Apabila *subrole* aktor tidak memiliki kewenangan untuk menulis pada kategori *dataset* tersebut, maka *request* ditolak. Bagian berlatar merah pada Gambar III.3 menunjukkan alur yang berubah atau ditambahkan pada rancangan Tugas Akhir ini dibandingkan Implementasi DecMed sebelumnya. Perubahan tersebut mencakup pengambilan informasi *subrole* dari *smart contract* serta validasi kesesuaian *role*, *subrole*, dan kategori *dataset* sebelum akses tulis diberikan.

Validasi menggunakan data *role* dan *subrole* yang terdapat pada penyimpanan *on-chain* karena pada Macaroon tidak terdapat informasi *role* dari pengguna aktif saat ini, sesuai dengan rancangan *caveats* pada Tabel III.10. Sementara itu, *registry personnel* pada penyimpanan *on-chain* merepresentasikan status kelembagaan aktor, misalnya apakah aktor masih terdaftar sebagai *personnel* rumah sakit dengan *role* atau *subrole* tertentu. Dengan demikian, *request* hanya dilanjutkan apabila aktor memiliki token yang valid sekaligus masih memiliki otoritas kelembagaan yang sesuai.

III.3.4.3 Verifikasi *Wallet* Aktor Aktif

Sesuai analisis pada Subbab IV.2.6.5, diperlukan verifikasi tambahan terhadap *wallet* aktor aktif untuk memastikan *request* dilakukan oleh aktor aktif yang sah.

Rancangan proses verifikasi dilakukan dengan mekanisme *presigned request*.



Gambar III.3: *Sequence Diagram* Rancangan Validasi *subrole* Tenaga Medis terhadap *Dataset Category*

Ketika *client* mengirimkan *request* ke *Server PRE*, *client* juga menambahkan header yang berisi konteks *request* yang telah di-*sign* menggunakan kunci privat *wallet* pengguna. Konteks *request* memuat informasi seperti *token_id*, *patient_address*, *related_rme_id*, *operation*, *segment_id*, *dataset_category*, *function_category*, dan *timestamp*. Keberadaan *timestamp* pada konteks yang ditandatangani juga membuat *signature* hanya berlaku untuk rentang waktu tertentu. Apabila *signature* tidak valid maka *request* ditolak.

III.3.4.4 Pencabutan Hak Akses Macaroon Delegasi

Sesuai dengan analisis pada Subbab III.2.4.3, diperlukan mekanisme revokasi dan audit delegasi untuk memastikan bahwa pencabutan akses dapat berdampak pada seluruh token turunan yang berasal dari token tersebut. Revokasi akses pada implementasi Tugas Akhir ini masih menjadikan pasien sebagai *resource owner* yang berwenang untuk mencabut akses terhadap RME.

Revokasi akses pada implementasi ini dilakukan melalui kombinasi pencatatan status di IOTA *on-chain* dan Redis *PRE*, dengan *server PRE* bertindak sebagai titik penegakan kebijakan akses. Lapisan *on-chain* digunakan untuk mencatat perubahan status akses dan audit trail revokasi, sedangkan Redis *PRE* digunakan untuk menyimpan *hash* dari token yang telah dicabut. Alur pencabutan akses oleh pasien ditunjukkan pada Gambar III.4. Pada alur tersebut, pasien memilih akses yang akan dicabut melalui *Patient Client*. *Patient Client* kemudian mengeksekusi fungsi *revoke_access()* pada *smart contract Move* untuk menandai akses sebagai dicabut, menghapus hak akses pada penyimpanan *on-chain*, melakukan pencabutan berantai terhadap penerima delegasi turunan, serta mencatat kejadian revokasi ke dalam *audit log*. Setelah transaksi *on-chain* berhasil, *Patient Client* mengirimkan *payload* revokasi ke *Server PRE* agar menyimpan *hash* dari token yang telah dicabut pada Redis *PRE* dan menghapus *re-encryption key* terkait dengan token Macaroon yang direvokasi.

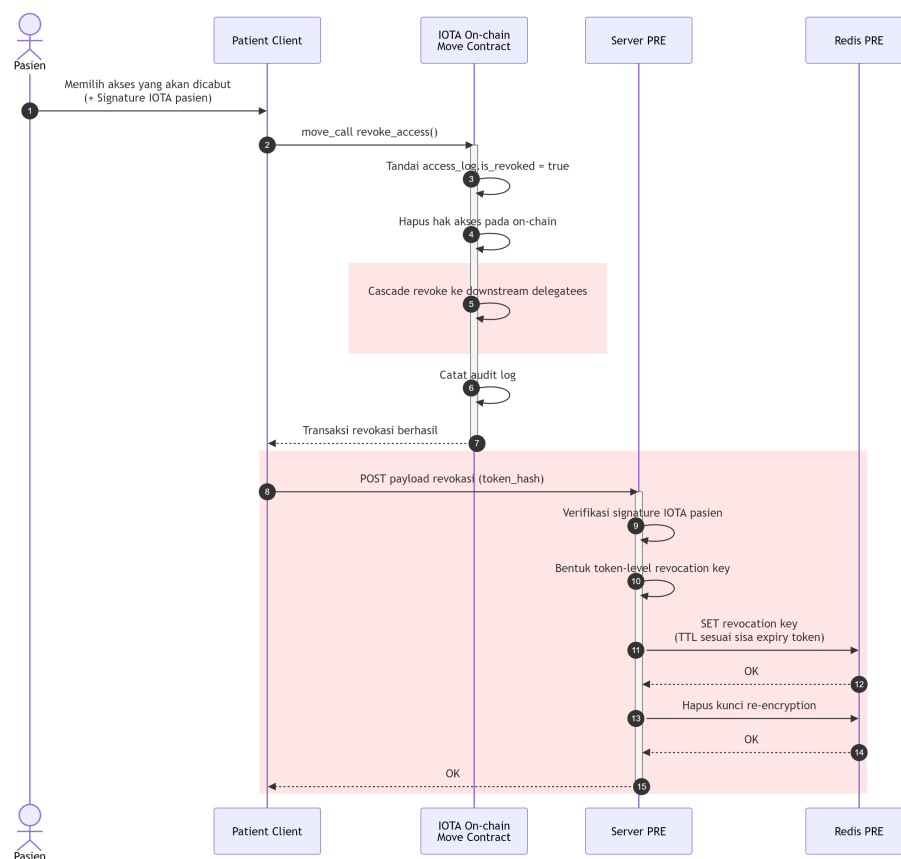
Bagian berlatar merah pada Gambar III.4 menunjukkan alur baru yang ditambahkan pada Tugas Akhir ini. Alur baru tersebut mencakup pencabutan berantai terhadap seluruh *downstream delegates*, pengiriman *payload* revokasi ke *Server PRE*, pembentukan *revocation key*, penyimpanan status revokasi pada Redis dengan

TTL sesuai masa berlaku token, serta penghapusan kunci *re-encryption*. Dengan penambahan alur ini, revokasi tidak hanya tercatat pada IOTA, tetapi juga langsung ditegakkan oleh *Server PRE* ketika token Macaroon digunakan kembali.

Pada saat verifikasi, *Server PRE* memeriksa *hash* token yang sedang digunakan dan setiap *caveat parent_token_hash* ke *revocation key* pada Redis. Dengan demikian, apabila pasien mencabut akses pada delegator awal, maka token Macaroon turunan yang dibuat dari akses tersebut juga akan dianggap tidak valid. Selain itu, pemeriksaan status hak akses juga dilakukan pada penyimpanan *on-chain* yang mencatat status hak akses.

III.3.5 Rancangan Alur Akses dan Delegasi Macaroon

Sesuai dengan analisis pada Subbab III.2.1 dan skenario pada Subbab III.1.1.3, rancangan alur akses dan delegasi pada Tugas Akhir ini dirancang untuk menjawab kebutuhan delegasi akses antar aktor layanan kesehatan tanpa mengharuskan pasien menerbitkan ulang token setiap kali terjadi perpindahan tugas dalam proses



Gambar III.4: *Sequence Diagram* Rancangan Alur Pencabutan Hak Akses Macaroon Delegasi

pelayanan.

Alur akses dan delegasi pada rancangan ini mengikuti alur pelayanan kesehatan yang umum terjadi pada proses rawat jalan dan rawat inap. Alur ini dibahas pada Subbab III.1.1.3. Pasien terlebih dahulu diterima oleh petugas rekam medis. Setelah itu, petugas rekam medis memberikan akses kepada perawat untuk melakukan pemeriksaan awal. Petugas rekam medis juga memberikan akses kepada dokter terkait agar dokter dapat memeriksa pasien dan menentukan tindakan medis berikutnya. Apabila dokter membutuhkan pemeriksaan penunjang, dokter dapat mendelegasikan akses kepada petugas laboratorium. Setelah hasil pemeriksaan tersedia dan dokter menentukan terapi atau pengobatan, dokter dapat mendelegasikan akses kepada apoteker untuk memproses resep dan penyerahan obat. Rancangan alur tersebut ditunjukkan pada Gambar III.5.

Delegasi akses pada rancangan ini terdiri atas tiga tahap utama. Tahap pertama adalah penerbitan token awal kepada petugas rekam medis sebagai aktor yang menerima pasien dan mengoordinasikan akses awal dalam proses pelayanan. Tahap kedua adalah delegasi lokal dari petugas rekam medis kepada perawat dan dokter. Tahap ketiga adalah delegasi lanjutan dari dokter kepada petugas laboratorium dan apoteker, sesuai kebutuhan tindakan medis.

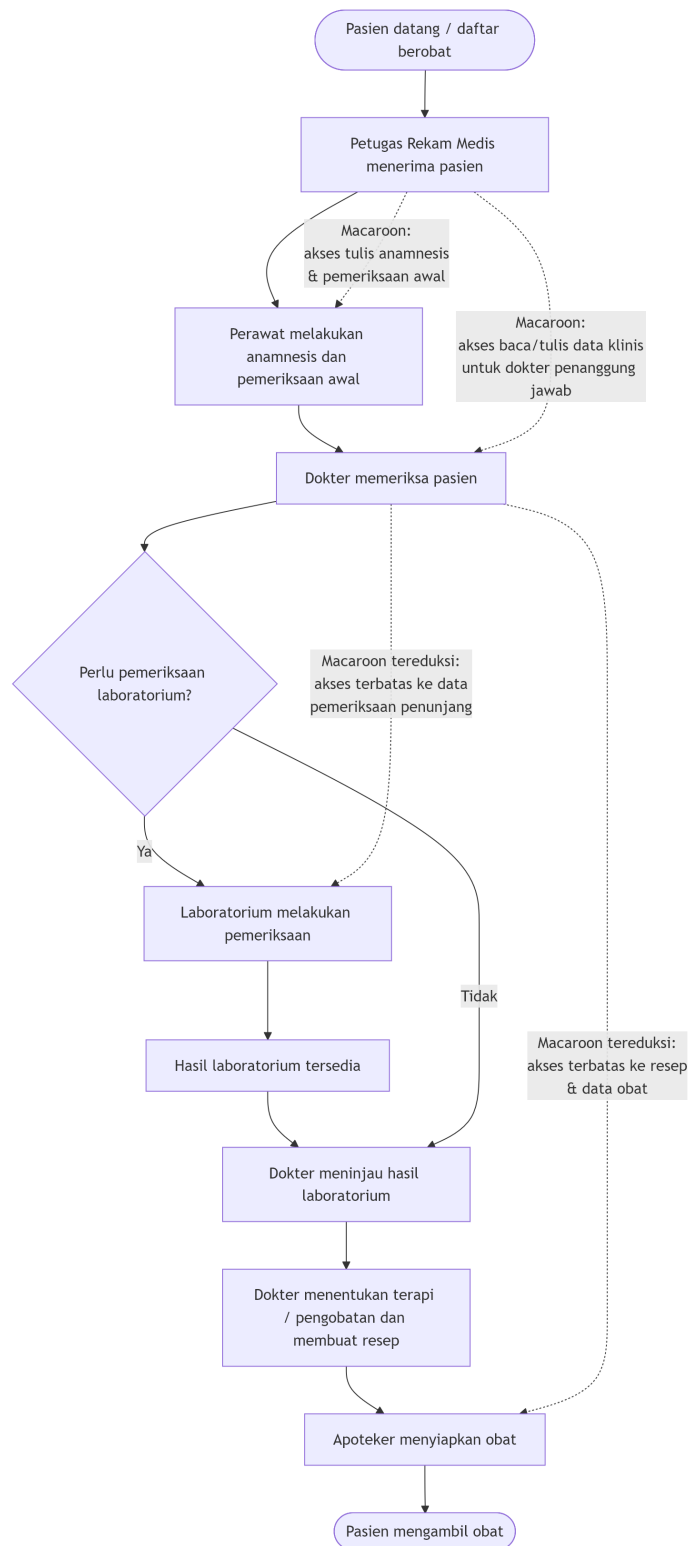
Penjelasan kemudian dilanjutkan dengan bagian berikutnya, yaitu pola teknis proses pemberian token Macaroon atau proses delegasi oleh *delegator* hingga akhirnya digunakan oleh *delegatee* untuk mengakses data RME. Pola teknis ini dibagi menjadi dua bagian, yaitu proses pemberian token Macaroon delegasi hingga tercatat pada *on-chain* dan proses pengambilan dan penggunaan token Macaroon delegasi.

III.3.5.1 Penerbitan Token Awal oleh Pasien kepada Petugas Rekam Medis

Penerbitan token awal merupakan proses pemberian otorisasi pertama pada saat pasien memulai layanan kesehatan. Pada tahap ini, pasien tetap menjadi pihak yang mengendalikan pemberian akses awal terhadap data RME miliknya. Pasien memberikan persetujuan akses melalui *Patient Client*, kemudian *Server PRE* menerbitkan token awal berdasarkan persetujuan tersebut. Pasien mengetahui identitas dari petugas rekam medis melalui QR yang dipindai, kemudian

membuktikan kepemilikan *wallet* kepada PRE dengan *nonce* sebelum token awal diterbitkan.

Pada rancangan ini, petugas rekam medis berperan sebagai aktor awal yang menerima pasien dan menginisiasi proses pelayanan. Token ini tidak dimaksudkan



Gambar III.5: *Flow Diagram* Alur Akses dan Delegasi pada Proses Pelayanan Pasien

agar petugas rekam medis dapat mengakses seluruh data klinis pasien secara bebas, melainkan sebagai token induk yang dapat digunakan untuk menurunkan akses kepada aktor layanan kesehatan yang relevan. Terdapat 2 token Macaroon yang diterbitkan pada tahapan ini, yaitu token Read dan token Update. Penggunaan langsung token tersebut tetap dibatasi oleh pemeriksaan otoritas yang dibahas pada Subbab III.3.4, sehingga cakupan token induk tidak otomatis berarti seluruh data dapat dibaca bebas oleh petugas rekam medis. Contoh *caveat* yang ditambahkan pada token awal untuk petugas rekam medis dapat dilihat pada Lampiran H.1 dan H.2.

Untuk memastikan RME tetap dapat diproses dengan mekanisme Umbral PRE, dibutuhkan sebuah *data PRE seed*. *Data PRE seed* ini digunakan untuk membangkitkan pasangan kunci PRE data yang dibutuhkan dalam proses enkripsi, dekripsi, dan *re-encryption*. *Data PRE seed* tersebut dibuat pada *patient client* ketika akses awal diberikan, lalu disimpan dalam metadata akses di *on-chain* dalam kondisi terenkripsi menggunakan kunci publik PRE penerima akses. Pada proses delegasi, *data PRE seed* yang sudah ada didekripsi oleh pemberi delegasi, kemudian dienkripsi ulang menggunakan kunci publik PRE penerima delegasi dan disimpan kembali bersama metadata delegasi di *on-chain*.

Alur penerbitan token awal kepada petugas rekam medis diawali dengan pasien bertemu dengan petugas rekam medis yang menerima permintaan pelayanan serta memperlihatkan QR untuk discan oleh pasien. Pasien memberikan persetujuan akses awal melalui *Patient Client* setelah memilih konteks pelayanan (rawat jalan atau rawat inap), batasan waktu akses, dan jenis akses yang diberikan. *Patient Client* mengirimkan permintaan penerbitan token kepada *Server PRE*. *Server PRE* memverifikasi identitas pasien, identitas petugas rekam medis, serta batasan yang diberikan. Jika lolos proses verifikasi, *Server PRE* menerbitkan token Macaroon awal dengan *caveats* yang mengikat token pada pasien, ID RME terkait, petugas rekam medis sebagai penerima awal, cakupan akses, masa berlaku, serta batas delegasi. *Server PRE* mencatat penerbitan token awal ke dalam penyimpanan *on-chain*. Token awal didistribusikan kepada petugas rekam medis melalui mekanisme yang dijelaskan pada Subbab III.3.5.4 dengan memanfaatkan penyimpanan *on-chain*.

Tabel III.12: Pemetaan Delegasi Token dari Petugas Rekam Medis

Penerima Delegasi	Tujuan Delegasi	Cakupan Akses
Perawat	Melakukan anamnesis dan pemeriksaan awal pasien.	Menulis segmen anamnesis, pemeriksaan fisik, dan pemeriksaan psikologis pada <i>dataset</i> rawat jalan.
Dokter	Memeriksa pasien, membaca hasil pemeriksaan awal, menentukan tindakan medis, diagnosis, terapi, dan kebutuhan pemeriksaan penunjang.	Membaca dan menulis segmen klinis yang relevan pada <i>dataset</i> rawat jalan atau rawat inap, serta memiliki kemampuan delegasi lanjutan untuk apoteker dan petugas laboratorium.

III.3.5.2 Delegasi dari Petugas Rekam Medis kepada Perawat dan Dokter

Setelah menerima token awal, petugas rekam medis dapat mendelegasikan akses kepada perawat dan dokter terkait sesuai kebutuhan pelayanan. Delegasi ini dilakukan dengan mengirimkan permintaan atenuasi kepada *server* PRE. *Server* PRE memverifikasi *request* delegasi. Apabila valid, *server* PRE mengembalikan token turunan untuk diproses lebih lanjut oleh *client delegator* untuk dicatat pada penyimpanan *on-chain*.

Delegasi kepada perawat diberikan agar perawat dapat membaca RME pasien untuk melakukan penanganan, dan menulis hasil pemeriksaan awal. Sementara itu, delegasi kepada dokter diberikan agar dokter dapat membaca data seluruh rekam medis pasien, menuliskan RME untuk episode layanan saat ini, serta melakukan delegasi lanjutan apabila diperlukan. Pemetaan delegasi dari petugas rekam medis ditunjukkan pada Tabel III.12. Contoh *caveats* yang ditambahkan pada token turunan untuk perawat dan dokter dapat dilihat pada Lampiran H.3 dan H.4.

III.3.5.3 Delegasi dari Dokter kepada Laboratorium dan Apoteker

Setelah dokter menerima token turunan dari petugas rekam medis, dokter dapat melakukan delegasi lanjutan apabila dibutuhkan dalam proses pelayanan. Delegasi lanjutan dilakukan kepada petugas laboratorium apabila dokter membutuhkan pemeriksaan penunjang. Setelah dokter menentukan terapi, dokter juga dapat mendelegasikan akses kepada apoteker untuk memproses resep dan penyerahan obat. Contoh *caveats* yang ditambahkan pada token turunan untuk laboratorium dan apoteker dapat dilihat pada Lampiran H.5 dan H.6. Pemetaan delegasi lanjutan dari dokter ditunjukkan pada Tabel III.13.

Tabel III.13: Pemetaan Delegasi Lanjutan dari Dokter

Penerima Delegasi	Tujuan Delegasi	Cakupan Akses
Petugas Laboratorium	Melakukan pemeriksaan penunjang berdasarkan permintaan dokter dan mengunggah hasil pemeriksaan.	Membaca segmen pemeriksaan penunjang dan menulis pada <i>dataset</i> laboratorium.
Apoteker	Memproses resep, menyiapkan obat, dan mencatat penyerahan obat kepada pasien.	Membaca segmen terapi dan menulis pada <i>dataset</i> apotek.

III.3.5.4 Proses Pemberian dan Penggunaan Token Macaroon Delegasi

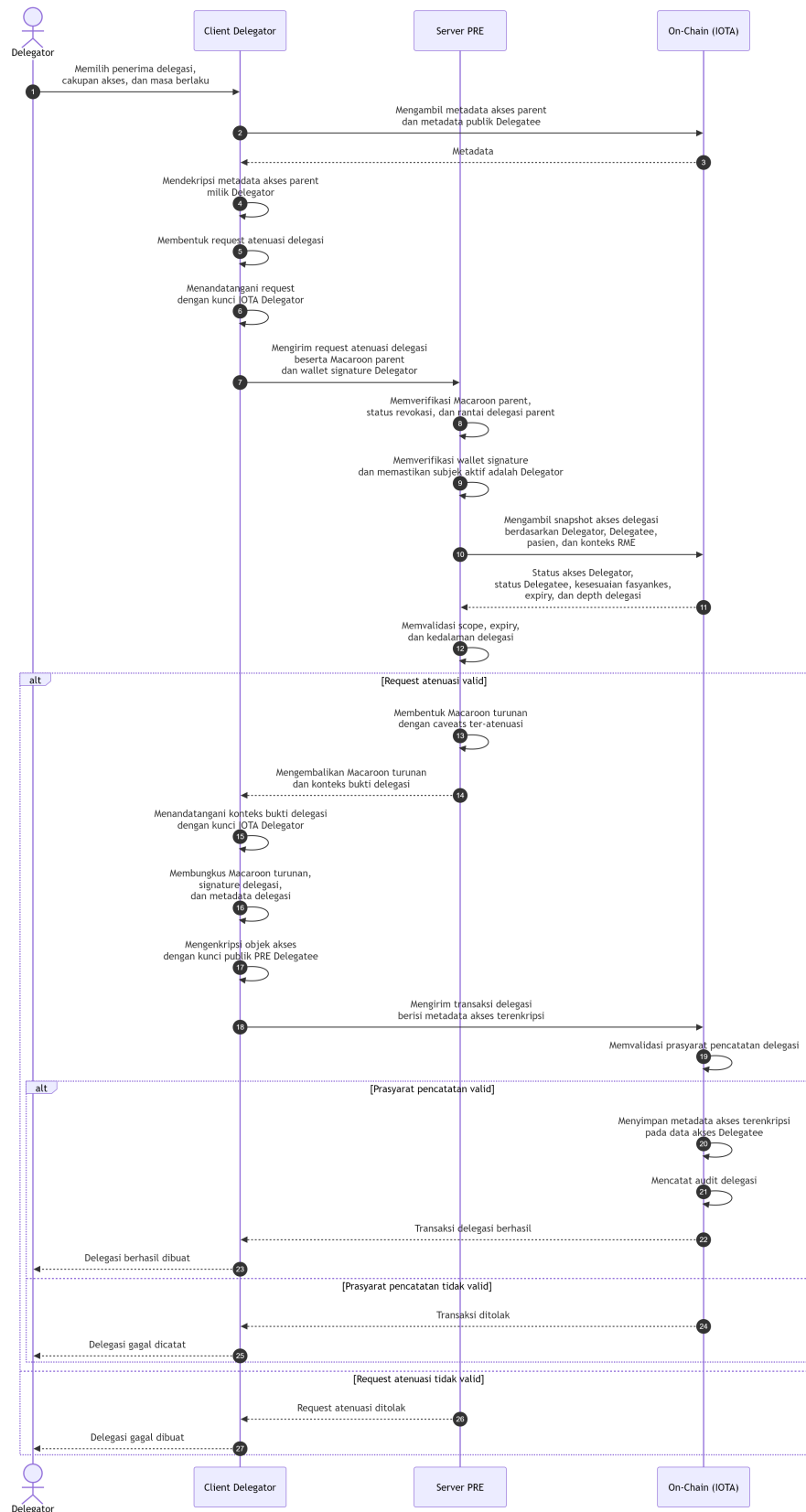
Secara umum, proses pemberian dan penggunaan token Macaroon delegasi dibagi menjadi dua bagian. Bagian pertama adalah proses pembentukan dan pencatatan delegasi, *client* yang meminta atenuasi token ke *server* PRE. Bagian ini mencakup pembentukan token Macaroon turunan, enkripsi metadata delegasi, serta pencatatan metadata akses terenkripsi pada penyimpanan *on-chain*. Alur proses tersebut ditunjukkan pada Gambar III.6.

Berdasarkan Gambar III.6, proses delegasi diawali ketika *delegator* memilih cakupan akses yang akan diberikan kepada *delegatee*. Setelah cakupan akses ditentukan, *client delegator* mengambil token Macaroon yang dimiliki oleh *delegator* dan melakukan *request* atenuasi token ke *server* PRE.

Server PRE memverifikasi *request* delegasi. Validasi dilakukan untuk memeriksa apakah cakupan akses turunan lebih luas, verifikasi *signature* dari *delegator*, status akses *delegator* pada *on-chain*, serta batasan cakupan yang diminta. Apabila valid, *server* PRE mengembalikan token turunan untuk diproses lebih lanjut oleh *client delegator* untuk dicatat pada penyimpanan *on-chain*.

Setelah token Macaroon turunan terbentuk, token tersebut tidak disimpan secara langsung dalam bentuk *plaintext* pada penyimpanan *on-chain*. Aplikasi *client* milik *delegator* terlebih dahulu mengambil kunci publik milik *delegatee* dari IOTA, termasuk kunci publik PRE milik *delegatee*. Kunci publik PRE kemudian digunakan untuk mengenkripsi objek akses yang berisi token Macaroon turunan dan metadata delegasi. Dengan pendekatan ini, metadata akses yang dicatat pada *on-chain* hanya tersedia dalam bentuk terenkripsi yang hanya bisa dibuka oleh *delegatee*.

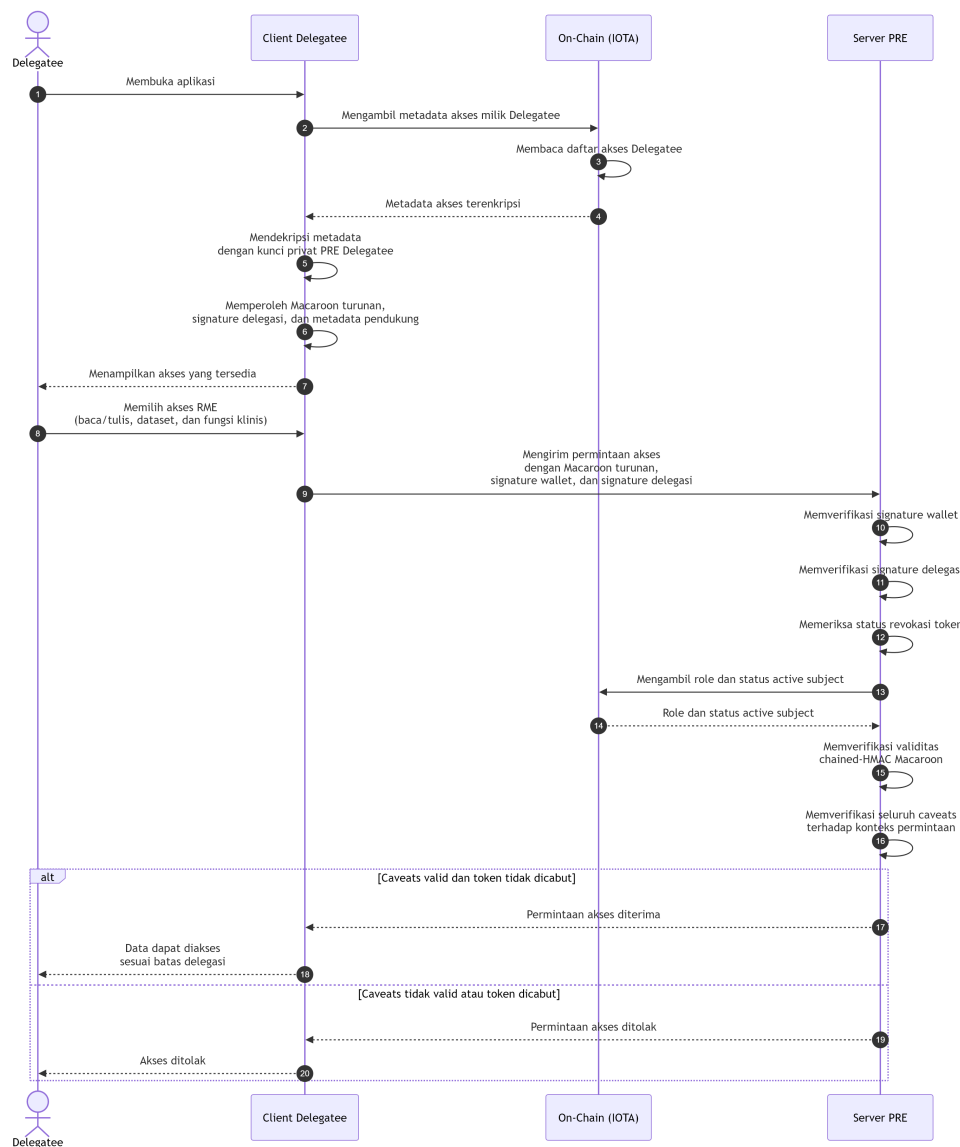
Metadata akses terenkripsi kemudian dikirimkan ke disimpan pada IOTA menggunakan *smart contract*. Pada tahap ini, *smart contract* mencatat metadata



Gambar III.6: Sequence diagram pembentukan dan pencatatan token Macaroon delegasi

terenkripsi pada data akses milik *delegatee* dan juga memvalidasi prasyarat delegasi yang dapat diperiksa dari penyimpanan *on-chain*. Prasyarat tersebut meliputi status aktif akun, kesesuaian fasyankes antara *delegator* dan *delegatee*, serta keberadaan dan masa berlaku akses *delegator* terhadap pasien terkait. Apabila prasyarat tersebut valid, metadata akses terenkripsi disimpan pada daftar akses milik *delegatee*. Apabila prasyarat tidak terpenuhi, transaksi delegasi ditolak.

Bagian kedua dari proses ini adalah pengambilan dan penggunaan token Macaroon delegasi oleh *delegatee*. Bagian ini terjadi setelah metadata akses terenkripsi berhasil dicatat pada blockchain. Alur pengambilan dan penggunaan token Macaroon delegasi ditunjukkan pada Gambar III.7.



Gambar III.7: Sequence diagram pengambilan dan penggunaan token Macaroon delegasi

Berdasarkan Gambar III.7, ketika *delegatee* membuka aplikasi, *client* mengambil metadata akses terenkripsi yang tersimpan pada blockchain. Metadata tersebut kemudian didekripsi secara lokal menggunakan kunci privat PRE milik *delegatee*. Apabila proses dekripsi berhasil, aplikasi memperoleh token Macaroon turunan beserta metadata pendukungnya. Token Macaroon turunan tersebut kemudian dapat digunakan sebagai token otorisasi ketika *delegatee* mengirim permintaan akses baca atau tulis ke *Server PRE*. Selain membentuk token turunan, aplikasi *client* juga membentuk bukti delegasi yang di-sign menggunakan *wallet* milik *delegator*. Bukti ini digunakan agar *Server PRE* dapat memastikan bahwa token turunan memang dibuat oleh pihak yang berwenang dalam rantai delegasi.

Pada tahap penggunaan token, *Server PRE* melakukan verifikasi terhadap token Macaroon yang dikirimkan oleh *delegatee*. Verifikasi ini mencakup pemeriksaan validitas token melalui mekanisme *chained-HMAC*, pemeriksaan status revokasi token, verifikasi bukti kepemilikan *wallet* serta bukti delegasi, dan pemeriksaan seluruh *caveats* yang melekat pada token. Selain itu, *Server PRE* juga dapat mengambil informasi aktor aktif dari *on-chain*, seperti *role* dan status akun, untuk memastikan permintaan akses berasal dari pihak yang sesuai. *Caveats* tersebut diverifikasi terhadap konteks permintaan akses, seperti identitas pasien, RME terkait, jenis hak akses, dataset, fungsi klinis, waktu kedaluwarsa, dan identitas *delegatee*. Apabila seluruh *caveats* terpenuhi, permintaan akses diterima sesuai batas delegasi yang diberikan. Sebaliknya, apabila terdapat *caveats* yang tidak terpenuhi, *Server PRE* menolak permintaan akses tersebut.

Dengan mekanisme ini, IOTA digunakan sebagai media pencatatan dan distribusi metadata akses yang dapat diaudit, sedangkan kerahasiaan token tetap dilindungi melalui enkripsi. Token Macaroon tidak tersedia dalam bentuk terbuka pada *state on-chain*, sehingga pihak selain *delegatee* tidak dapat langsung memanfaatkan metadata akses yang tercatat pada blockchain. Selain itu, karena pembatasan hak akses dibawa langsung oleh *caveats* pada Macaroon, *server PRE* tetap dapat menegakkan batas delegasi pada saat token digunakan.

Untuk mengurangi risiko *delegator* melakukan delegasi akses berlebihan, jumlah delegasi yang dapat dilakukan oleh seorang pemegang hak akses dapat dibatasi dengan mencatat jumlah delegasi yang telah dilakukan oleh *delegator* pada penyimpanan *on-chain*. Jumlah delegasi ini dapat diperiksa oleh *server PRE* saat

Tabel III.14: Pemetaan Rancangan terhadap Solusi

ID	Nama Rancangan	Merealisasikan Solusi
R-TA-01	Rancangan arsitektur solusi DecMed dengan Macaroons, segmentasi, dan PRE sebagai <i>policy enforcement point</i> .	S-TA-01, S-TA-02, S-TA-03, S-TA-04
R-TA-02	Rancangan segmentasi data RME berdasarkan <i>dataset</i> dan <i>function category</i> .	S-TA-02
R-TA-03	Rancangan <i>caveats</i> Macaroons.	S-TA-01
R-TA-04	Rancangan rantai delegasi Macaroons dengan <i>root_subject</i> , <i>delegated_by</i> , <i>delegated_to</i> , dan <i>max_delegation_depth</i> .	S-TA-03, S-TA-04
R-TA-05	Rancangan verifikasi <i>caveats</i> , <i>effective capability</i> , dan bukti kepemilikan <i>wallet</i> pada <i>Server PRE</i> .	S-TA-04
R-TA-06	Rancangan validasi <i>subrole</i> tenaga medis terhadap <i>dataset category</i> .	S-TA-05
R-TA-07	Rancangan revokasi delegasi token Macaroons.	S-TA-06
R-TA-08	Rancangan alur akses dan delegasi pada skenario layanan kesehatan kolaboratif.	S-TA-03, S-TA-04, S-TA-05

memverifikasi *request* delegasi. Apabila jumlah delegasi melebihi batas yang ditentukan, *server PRE* menolak *request* delegasi. Jumlah delegasi ini dapat dibatasi dengan aturan bahwa seorang *delegator* hanya dapat melakukan delegasi sebanyak 1 kali untuk setiap *role*.

III.3.6 Pemetaan Rancangan terhadap Solusi

Setelah rancangan solusi dijelaskan, setiap rancangan dipetakan kembali terhadap solusi yang telah dianalisis pada Subbab III.2.5. Pemetaan ini digunakan untuk menunjukkan bahwa setiap rancangan R-TA memiliki keterkaitan langsung dengan solusi S-TA yang diusulkan.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

IV.1 Lingkungan Implementasi

Implementasi sistem dilakukan pada lingkungan prototipe yang merepresentasikan komponen utama DecMed, yaitu *patient client*, *hospital client*, *ministry client*, *Server PRE*, penyimpanan *off-chain* IPFS, Redis, IOTA *Gas Station*, dan *smart contract* IOTA Move pada jaringan IOTA Localnet. Lingkungan ini dipilih agar rancangan pada Bab III dapat diuji tanpa bergantung pada integrasi langsung dengan sistem informasi rumah sakit yang sesungguhnya.

Pada implementasi DecMed sebelumnya, semua komponen dijalankan pada lingkungan lokal. Semua *client*, *Server PRE*, Redis PRE, kluster IPFS, IOTA *Gas Station*, dan jaringan IOTA jenis Localnet dijalankan langsung di komputer lokal. Perangkat lokal yang digunakan tidak cukup kuat untuk menjalankan semua komponen secara bersamaan. Oleh karena itu, dilakukan pembagian komponen implementasi tersebut ke dalam lingkungan lokal dan dua buah *Virtual Private Server* (VPS).

Komponen *client* dan *Server PRE* dijalankan pada lingkungan lokal. Lingkungan lokal digunakan untuk memudahkan pengembangan fitur, pengujian alur akses, serta pengamatan respons sistem. Komponen pendukung yang membutuhkan ketersediaan jaringan, yaitu kluster IPFS dan IOTA *Gas Station*, dijalankan pada VPS. *Smart contract* IOTA Move di-deploy pada jaringan IOTA Localnet yang juga berjalan di VPS. IOTA Localnet digunakan karena sudah memadai untuk menyimulasikan interaksi dengan jaringan IOTA. VPS digunakan untuk menjalankan komponen pendukung karena komponen tersebut tidak banyak berubah selama implementasi, tetapi tetap dibutuhkan selama pengembangan. Pemanfaatan VPS juga mengurangi beban perangkat lokal karena perangkat lokal tidak perlu menjalankan seluruh komponen pendukung.

Berikut adalah spesifikasi dari perangkat lokal yang digunakan untuk implementasi:

- Sistem Operasi: Ubuntu 24.04.4, berjalan di lingkungan WSL2
- CPU: Intel Core i5-11400H (3 core fisik dan 6 *logical thread* dialokasikan untuk WSL2)
- Memori RAM: 16 GB (8 GB teralokasi untuk WSL2)
- *Tools*:
 - Rust (rustc 1.93.0, cargo 1.93.0)
 - Node.js v24.13.0
 - Docker 29.5.3

Sementara itu, spesifikasi kedua VPS yang digunakan adalah sebagai berikut:

- Sistem Operasi: Ubuntu 24.04.4 LTS
- Jumlah vCPU: 8 vCPU
- Memori RAM: 8 GB
- *Tools*:
 - Docker: Docker 29.4.2

IV.2 Implementasi Sistem

Bagian ini menjelaskan implementasi dari rancangan solusi yang telah dijabarkan pada Bab III. Implementasi difokuskan pada integrasi Macaroons sebagai token otorisasi, segmentasi data RME, mekanisme atenuasi dan delegasi akses, penegakan otoritas pada *Server* PRE, perubahan *smart contract* IOTA Move, serta penyesuaian antarmuka *client*.

Daftar komponen implementasi dan keterkaitannya dengan rancangan solusi ditunjukkan pada Tabel IV.1. ID I-TA digunakan sebagai rujukan pada implementasi terkait. Rancangan ID R-TA-01 merupakan rancangan arsitektur sistem yang implementasinya merupakan keseluruhan implementasi sistem DecMed, sehingga tidak muncul pada tabel.

Tabel IV.1: Daftar Implementasi Sistem

ID	Komponen	Merealisasikan Rancangan
I-TA-01	<i>Crate decmed-rme-segment.</i>	R-TA-02
I-TA-02	<i>Crate decmed-macaroon-auth</i> dan <i>Crate macaroon.</i>	R-TA-03, R-TA-04, R-TA-05
I-TA-03	Implementasi penegakan otoritas pada <i>Server PRE.</i>	R-TA-05, R-TA-06, R-TA-07
I-TA-04	Implementasi <i>smart contract</i> IOTA Move untuk metadata segmen, akses, dan audit.	R-TA-02, R-TA-04, R-TA-07
I-TA-05	Implementasi <i>Patient Client</i> untuk pemberian akses awal, pencabutan, dan audit.	R-TA-07, R-TA-08
I-TA-06	Implementasi <i>Hospital Client</i> untuk akses baca dan tulis segmen RME dan delegasi <i>offline.</i>	R-TA-02, R-TA-04, R-TA-08

IV.2.1 Batasan Implementasi

Implementasi pada Tugas Akhir ini memiliki beberapa batasan sebagai berikut:

1. Implementasi dilakukan dalam lingkungan komputer lokal dan VPS.
 - Semua *client* dan *Server PRE* dijalankan langsung di komputer lokal.
 - VPS menjalankan kluster IPFS dan IOTA *Gas Station*.
 - *Smart contract* yang digunakan di-deploy pada IOTA Localnet yang berjalan di VPS.
2. Implementasi mempertahankan arsitektur kerangka kerja DecMed sebagaimana yang dijelaskan pada Subbab III.3.1.
3. Kluster IPFS yang digunakan dalam implementasi hanya menggunakan konfigurasi *default* sesuai implementasi DecMed sebelumnya. Konfigurasi tersebut terdiri atas tiga buah *peers*.
4. Segmentasi data sesuai dengan yang dibahas pada III.3.2 hanya dilakukan hingga tingkat kombinasi *dataset category* dan *function category*, kemudian disimpan sebagai *text*.
5. Nilai `max_delegation_depth` pada token Macaroon hanya diatur sampai tingkat 2. Hal ini untuk memastikan bahwa token turunan hanya dapat diturunkan sebanyak 2 kali sesuai dengan skenario pada Subbab III.1.1.3 dan rancangan pada Subbab III.3.5.

IV.2.2 Implementasi Arsitektur Sistem

Implementasi arsitektur sistem mengikuti rancangan pada Subbab III.3.1. Komponen dasar DecMed dipertahankan, yaitu IOTA untuk metadata *on-chain*, IPFS untuk penyimpanan *off-chain* segmen data yang terenkripsi, dan *Server PRE* sebagai penegak otoritas dan proses *re-encryption* data. Arsitektur DecMed secara keseluruhan dapat dilihat pada gambar III.1.

Selain perubahan sesuai rancangan yang sudah dijelaskan pada subbab III.3, implementasi juga memperbaiki pola pengorganisasian kode. Pada DecMed awal, beberapa bagian yang memiliki fungsi serupa ditulis berulang pada beberapa komponen. Pola tersebut membuat perubahan kode menjadi lebih sulit di-*maintain* karena perubahan pada satu bagian perlu direplikasi pada komponen lain. Pada implementasi ini, bagian yang digunakan bersama dipisahkan menjadi *crate* Rust agar dapat digunakan ulang oleh beberapa komponen.

Crate yang diimplementasikan dan digunakan adalah *decmed-rme-segment* dan *decmed-macaroon-auth*. *decmed-rme-segment* berisi definisi struktur segmentasi RME, definisi *dataset category* dan *function category*, serta fungsi pendukungnya. Sementara itu, *decmed-macaroon-auth* berisi implementasi token otorisasi berbasis Macaroons di DecMed, termasuk pembuatan token, atenuasi token, dan proses verifikasi. *crate* *decmed-macaroon-auth* memiliki dependensi utama yaitu *crate* *macaroon* yang merupakan *crate open source* implementasi Macaroons dengan pembaruan *dependency*. *decmed-rme-segment* digunakan oleh *Patient Client*, *Hospital Client*, dan *Server PRE*, sedangkan *decmed-macaroon-auth* digunakan oleh *Hospital Client* dan *Server PRE*. *Ministry Client* tidak memiliki dependensi langsung terhadap kedua *crate* tersebut karena perannya tetap terbatas pada kebutuhan registrasi dan aktivasi rumah sakit. Hubungan antar komponen implementasi dan dependensi *crate* bersama ditunjukkan pada *package diagram* pada Gambar IV.1 dengan legenda dari *package diagram* tersebut dapat dilihat pada Lampiran J.

Dengan pemisahan tersebut, implementasi segmentasi data dan otorisasi tidak tersebar sebagai kode duplikatif pada setiap komponen.

IV.2.3 Implementasi Segmentasi Data RME

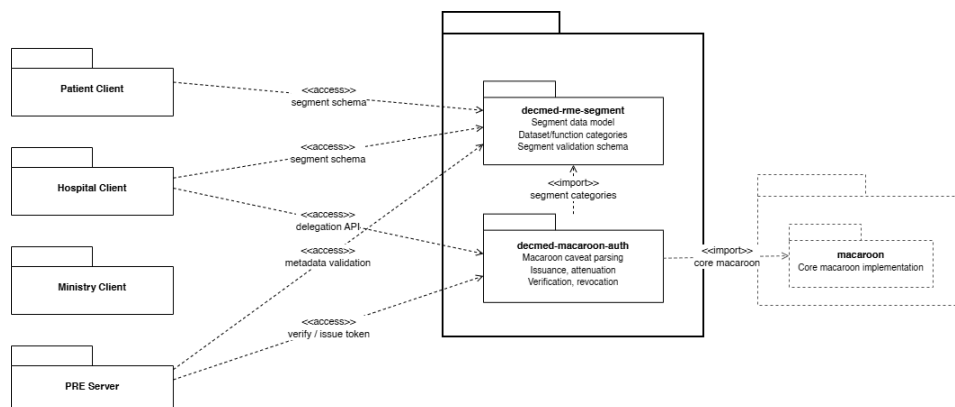
Segmentasi data RME diimplementasikan dengan memecah satu data RME menjadi beberapa objek data terenkripsi berdasarkan kombinasi *dataset category* dan *function category*. Kombinasi kategori tersebut mengikuti rancangan pada Subbab III.3.2. Setiap segmen data RME disimpan sebagai satu entri metadata pada penyimpanan *on-chain* dan satu objek data pada penyimpanan *off-chain*.

Segmentasi data RME diimplementasikan pada *crate* `decmed-rme-segment`. *Crate* ini mendefinisikan `DatasetCategory` dan `FunctionCategory` yang digunakan untuk kontrol akses granular data RME yang diakses. Implementasi `DatasetCategory` ditunjukkan pada Kode IV.1, implementasi `FunctionCategory` ditunjukkan pada Kode IV.2.

```
1  pub enum DatasetCategory {  
2      RAWAT_JALAN,  
3      RAWAT_INAP,  
4      LABORATORIUM,  
5      APOTEK,  
6  }
```

Kode IV.1: Implementasi `DatasetCategory` pada *crate* `decmed-rme-segment`

```
1  pub enum FunctionCategory {  
2      // ADMINISTRATIVE  
3      ADMINISTRATIVE_GENERAL,  
4  
5      // RAWAT_JALAN & RAWAT_INAP
```



Gambar IV.1: Package Diagram Implementasi

```

6      ANAMNESIS,
7      PEMERIKSAAN_FISIK,
8      PEMERIKSAAN_PSIKOLOGIS,
9      RIWAYAT_PENGUNAAN_OBAT,
10     RENCANA_RAWAT,
11     INSTRUKSI_MEDIK_DAN_KEPERAWATAN,
12     PEMERIKSAAN_PENUNJANG,
13     DIAGNOSIS,
14     INFORMED_CONSENT,
15     TERAPI,
16
17     // RAWAT_INAP
18     PERENCANAAN_PEMULANGAN,
19
20     // LABORATORIUM
21     LABORATORIUM,
22
23     // APOTEK
24     PERESEPAN,
25     DISPENSING,
26 }

```

Kode IV.2: Implementasi FunctionCategory pada *crate* decmed-rme-segment

Perbedaan antara *dataset category* rawat jalan dan rawat inap terdapat pada *function category* PERENCANAAN_PEMULANGAN. Segmen data ini hanya ada pada *dataset category* RAWAT_INAP, sedangkan pada *dataset category* RAWAT_JALAN tidak ada. Selain itu, *function category* ADMINISTRATIVE_GENERAL ada pada setiap *dataset category*.

IV.2.3.1 Implementasi Segmentasi Metadata RME pada Penyimpanan On-Chain

Definisi struktur metadata segmentasi ditulis pada *crate* decmed-rme-segment. *Crate* ini mendefinisikan RmeSegmentMetadata yang merupakan representasi metadata segmen sebelum diserialisasi menjadi JSON base64 dan disimpan pada *field* metadata milik PatientMedicalMetadata yang sudah ditunjukkan pada

Kode III.2. Implementasi struktur RmeSegmentMetadata terdapat pada Kode IV.3. Struktur tersebut sesuai dengan rancangan skema metadata segmen RME pada Lampiran Tabel G.1.

```
1 pub struct RmeSegmentMetadata {
2     pub segment_id: String,
3     pub related_rme_id: String,
4     pub patient_address: String,
5     pub author_address: String,
6     pub hospital_cid: String,
7     pub dataset_category: DatasetCategory,
8     pub function_category: FunctionCategory,
9     pub ipfs_cid: String,
10    pub integrity_hash: String,
11    pub capsule: String,
12    pub enc_key_and_nonce: String,
13    pub created_at: String,
14 }
```

Kode IV.3: Implementasi struktur RmeSegmentMetadata pada *crate* *decmed-rme-segment*

Sebelum metadata segmen disimpan pada IOTA, struktur metadata divalidasi terlebih dahulu. Validasi tersebut mencakup pemeriksaan setiap *field* wajib terisi, validitas kombinasi *dataset_category* dan *function_category*, serta kesesuaian *integrity_hash* terhadap data terenkripsi. *dataset_category* dan *function_category* memiliki nilai sesuai dengan Kode IV.1 dan IV.2. *patient_address* dan *author_address* memuat *address wallet* dari pasien dan aktor yang menulis isi segmen. *hospital_cid* merupakan *identifier* fasyankes, menunjukkan bahwa segmen data tersebut dihasilkan oleh tenaga medis dari fasyankes tersebut. *ipfs_cid* merupakan *pointer* ke data segmen terenkripsi yang disimpan pada penyimpanan *off-chain*. *integrity_hash* merupakan hash SHA-256 dari data segmen terenkripsi yang digunakan untuk memverifikasi integritas data. *capsule* merupakan kapsul PRE yang diperlukan untuk proses *re-encryption*. *enc_key_and_nonce* merupakan kunci simetris dan *nonce* yang sudah terenkripsi untuk membuka segmen data. *created_at* memuat informasi

tanggal dan waktu ketika segmen tercatat dengan format YYYY-MM-DD HH:MM:SS serta informasi *timezone*.

IV.2.3.2 Implementasi Segmentasi Data RME pada Penyimpanan *Off-Chain*

Definisi struktur segmentasi data penyimpanan *off-chain* didefinisikan pada *crate* `decmed-rme-segment`. *Crate* ini mendefinisikan `RmeSegmentData`, struktur data segmen RME yang disimpan pada penyimpanan *off-chain*. Implementasi struktur `RmeSegmentData` terdapat pada Kode IV.4. Struktur tersebut sesuai dengan rancangan skema data segmen RME pada Lampiran Tabel G.2.

```
1 pub struct RmeSegmentData {  
2     pub segment_id: String,  
3     pub related_rme_id: String,  
4     pub dataset_category: DatasetCategory,  
5     pub function_category: FunctionCategory,  
6     pub patient_address: String,  
7     pub service_date: String,  
8     pub author_address: String,  
9     pub payload: Value,  
10    pub payload_hash: String,  
11 }
```

Kode IV.4: Implementasi struktur `RmeSegmentData` pada *crate* `decmed-rme-segment`

`segment_id` merupakan ID unik untuk satu segmen data RME. `dataset_category` dan `function_category` digunakan untuk kontrol akses granular data RME. atribut tersebut memiliki nilai sesuai dengan Kode IV.1 dan IV.2. `patient_address` dan `author_address` memuat *address wallet* dari pasien dan aktor yang menulis isi segmen. `service_date` merupakan tanggal dan waktu ketika segmen tercatat dengan format YYYY-MM-DD HH:MM:SS beserta informasi *timezone*. `payload` berisi konten utama segmen dengan tipe data `Value`. Pada implementasi ini, `payload` berisi data dalam bentuk `text`. `payload_hash` merupakan hash integritas dari konten `payload`.

Pada saat data RME disimpan, *Hospital Client* membentuk objek untuk setiap segmen. Setiap segmen berisi data klinis atau administratif sesuai kategorinya,

kemudian dienkrpsi sebelum diunggah ke IPFS. Hasil unggahan IPFS berupa CID disimpan bersama metadata segmen pada IOTA sebagai *ipfs_cid*. Metadata tersebut disimpan dalam bentuk serialisasi JSON dan dikodekan dalam base64 pada *field* metadata milik *PatientMedicalMetadata*. Keputusan ini menjaga kompatibilitas dengan struktur penyimpanan DecMed yang masih berbentuk daftar metadata *flat*, tetapi tetap memungkinkan *Server* PRE membaca kategori segmentasi dari isi metadata.

IV.2.4 Implementasi Token Otorisasi Berbasis Macaroons

IV.2.4.1 Komponen Implementasi Macaroons

Seperti yang sudah dijelaskan pada subbab IV.2.2, terdapat dua *crate* yang berkaitan dengan implementasi Macaroons pada implementasi Tugas Akhir ini, yaitu *macaroon* dan *decmed-macaroon-auth*.

Crate *macaroon* merupakan hasil penyesuaian dari implementasi Macaroons berbasis Rust yang tersedia secara *open source* pada URL <https://github.com/macaroon-rs/macaroon>. Penyesuaian dilakukan karena *crate* asli memerlukan beberapa penyesuaian agar lebih sesuai dengan kebutuhan implementasi sistem DecMed, terutama dari sisi kompatibilitas *toolchain* dan pengelolaan *dependency*. Perubahan utama pada *crate* *macaroon* adalah penggantian *dependency* kriptografi berbasis *sodiumoxide* dengan *crate* pure Rust, yaitu *hmac*, *sha2*, *chacha20poly1305*, dan *rand*.

Setelah perubahan tersebut dilakukan, *unit testing* pada *crate* dijalankan untuk memastikan bahwa fungsi utama Macaroons tetap berjalan dengan benar. Pengujian tersebut mencakup pembuatan Macaroon, penambahan *caveat*, serialisasi, deserialisasi, serta verifikasi token. Dengan demikian, *crate* *macaroon* digunakan sebagai fondasi implementasi Macaroons pada sistem DecMed.

decmed-macaroon-auth merupakan *wrapper* yang dibangun untuk mempermudah penggunaan *crate* *macaroon* pada implementasi sistem DecMed. *Wrapper* ini menyediakan fungsi-fungsi utilitas untuk kebutuhan otorisasi DecMed, seperti proses penerbitan token Macaroon, melakukan atenuasi dengan penambahan *caveat* baru, definisi struktur *caveats* Macaroon, verifikasi Macaroon, dan fungsi pendukung lainnya. Rincian modul dan implementasi

Tabel IV.2: *Caveats* yang Diimplementasikan pada Token Macaroon

<i>Caveat</i>	Tipe Data
patient_address	String (alamat IOTA format heksadesimal)
related_rme_id	String (ID episode RME)
root_subject	String (alamat IOTA format heksadesimal)
delegated_by	String (alamat IOTA format heksadesimal)
delegated_to	String (alamat IOTA format heksadesimal)
read_dataset_in	HashSet<DatasetCategory>
write_dataset_in	HashSet<DatasetCategory>
read_function_in	HashSet<FunctionCategory>
write_function_in	HashSet<FunctionCategory>
expires_before	DateTime<Utc> (YYYY-MM-DD HH:MM:SS)
max_delegation_depth	u32
hospital_cid	String (CID IPFS)
parent_token_hash	String (hash SHA-256 dari token induk)
purpose	String (nilai: Read atau Update)

yang diimplementasikan pada *crate* *decmed-macaroon-auth* ditunjukkan pada Lampiran L.

IV.2.4.2 Struktur *Caveats* Macaroons

Struktur *caveats* pada token Macaroon pada implementasi ini ditunjukkan pada Tabel IV.2. *caveats* terdefinisi pada *crate* *decmed-macaroon-auth*, sehingga *hospital client* membentuk token dengan menambahkan *caveats* sesuai dengan daftar *caveats* yang terdefinisi pada *crate*, dan PRE akan menolak *request* jika terdapat *caveats* yang tidak termasuk ke dalam daftar *caveats*.

IV.2.4.3 Penerbitan Token Macaroon

Token otorisasi diimplementasikan menggunakan Macaroons sesuai rancangan pada Subbab III.3.3. Pada implementasi ini, penerbitan token dilakukan oleh *Server* PRE setelah permintaan akses dari pasien dinyatakan valid.

Pada proses penerbitan akses awal, *Server* PRE menerbitkan dua jenis token untuk penerima akses, yaitu token Read dan token Update. Token Read memuat *caveat* utama berupa *patient_address*, *root_subject*, *read_dataset_in*, *read_function_in*, *expires_before*, *max_delegation_depth*, *hospital_cid*, dan *purpose* = Read. Token Update memuat *caveat* berupa *patient_address*, *related_rme_id*, *root_subject*, *write_dataset_in*, *write_function_in*, *expires_before*, *max_delegation_depth*, *hospital_cid*, dan *purpose* =

Update.

Ketika melakukan penerbitan untuk petugas rekam medis, fungsi yang dipanggil adalah `issue_admin_personnel_token` yang termasuk implementasi pada `crate decmed-macaroon-auth`. Alur lengkap pemberian akses awal dari pasien kepada petugas rekam medis, termasuk pemindaian QR code, verifikasi identitas, pembentukan material PRE, penyimpanan ke Redis, dan pencatatan grant akses ke *smart contract*, dijelaskan pada subbab IV.2.5.

IV.2.4.4 Atenuasi Token Macaroon

Pada implementasi ini, pemegang token utama adalah para tenaga kesehatan, dengan role `AdministrativePersonnel` atau `MedicalPersonnel`. Pemegang token melakukan atenuasi token melalui *hospital client* dengan melakukan *request* atenuasi token ke *server* PRE. Implementasi dari fungsi atenuasi ini terdapat pada `crate decmed-macaroon-auth` dengan nama fungsi `attenuate_macaroon`. Fungsi ini bekerja dengan tahapan melakukan deserialisasi token Macaroon, lalu menambahkan *caveat* baru sesuai dengan *caveats* yang ingin ditambahkan, kemudian melakukan serialisasi kembali terhadap token Macaroon yang telah diatenuasi sebelum di-*return* sebagai *response*. Perhitungan *signature* baru dari Macaroon pada proses *chained-HMAC* dilakukan pada setiap kali *caveat* baru ditambahkan.

IV.2.4.5 Revokasi Token Macaroon

Revokasi token pada implementasi ini sesuai dengan rancangan pada Subbab III.3.4. Implementasi revokasi akses melibatkan pencatatan di IOTA *on-chain*, Redis PRE, dan *server* PRE.

Setiap token turunan menyimpan keterkaitan terhadap token induknya melalui *caveat parent_token_hash*. Informasi ini digunakan untuk mendukung pencabutan berantai, sehingga pencabutan terhadap token pada tingkat tertentu dapat berdampak pada token-token turunan yang bergantung kepadanya.

Pemeriksaan pencabutan dilakukan oleh *Server* PRE sebelum akses terhadap data terenkripsi diberikan. Saat *request* diterima oleh PRE, maka PRE akan memeriksa *hash* dari token Macaroon yang digunakan, dan mengecek apakah

ada di daftar *revoked keys* yang tersimpan pada Redis PRE dengan pola *revoked:token:<hash>*. Hal ini juga dilakukan dengan semua *hash* token yang ada pada *caveat parent_token_hash*. Jika ada salah satu *hash* yang ditemukan, maka *request* akan ditolak. Selain itu, proses ini juga akan menandai hak akses yang tercatat pada *on-chain* sebagai *revoked*.

IV.2.5 Delegasi Akses dan Atenuasi Offline

Mekanisme ini merupakan realisasi dari rancangan pada Subbab III.3.5. Implementasi dibagi menjadi dua bagian. Bagian pertama adalah proses pemberian akses awal dari pasien kepada petugas rekam medis sebagai penerima akses awal. Bagian kedua adalah proses delegasi akses antar tenaga kesehatan.

IV.2.5.1 Delegasi Akses dari Pasien kepada Petugas Rekam Medis

Implementasi delegasi akses dari pasien kepada petugas rekam medis merupakan realisasi dari rancangan pada Subbab III.3.5.2 dan III.3.5.3.

Proses diawali ketika pasien melakukan *scan* QR code milik AdministrativePersonnel. QR code tersebut memuat sebuah *string* dengan format *{hospitalCid}@{personnelIota}@{personnelPrePK}*. Informasi tersebut terdiri dari *hospital_cid* sebagai *identifier* fasyankes, *personnel_iota_address* sebagai alamat IOTA milik AdministrativePersonnel, dan *personnel_pre_public_key* sebagai PRE *public key* milik AdministrativePersonnel dalam format base64. PRE *public key* tersebut digunakan untuk mengenkripsi *data_pre_secret_key_seed* yang nantinya hanya dapat dibuka oleh petugas penerima akses.

Setelah QR code berhasil di-*scan*, *Patient Client* mengambil informasi publik AdministrativePersonnel melalui *smart contract* *get_hospital_personnel_info*. Langkah ini dilakukan untuk memastikan identitas petugas yang akan menerima akses. Pasien kemudian memilih jenis layanan, yaitu RAWAT_JALAN atau RAWAT_INAP, serta memilih mode akses yang akan diberikan. Implementasi mendukung tiga mode akses, yaitu *read*, *update*, dan *read.update*. Mode *read* hanya menerbitkan token baca, mode *update* hanya menerbitkan token pembaruan, sedangkan mode *read.update*

menerbitkan kedua token tersebut. Pada implementasi, batas waktu akses bawaan untuk RAWAT_JALAN adalah 1 hari, sedangkan batas waktu akses bawaan untuk RAWAT_INAP adalah 3 hari. Namun, implementasi juga mendukung nilai *expires_before* khusus sehingga batas waktu akses dapat disesuaikan dengan kebutuhan.

Selanjutnya, *Patient Client* meminta *nonce* kepada *Server PRE* melalui *endpoint /nonce*. Sebelum *nonce* diterbitkan, *Server PRE* memeriksa terlebih dahulu apakah alamat pasien telah terdaftar pada *smart contract is_patient_registered*. *Nonce* yang diterima kemudian di-*sign* menggunakan kunci IOTA pasien. Tanda tangan ini digunakan sebagai bukti bahwa permintaan penerbitan akses benar-benar berasal dari pasien yang sah.

Bersamaan dengan proses tersebut, *Patient Client* membuat *data_pre_secret_key_seed* baru dan menurunkan *data_pre_public_key* dari *seed* tersebut. *Seed* kemudian dienkripsi menggunakan *personnel_pre_public_key*. Selain itu, *Patient Client* juga membuat *key fragment k_frag* dari *PRE secret key* pasien menuju *data_pre_public_key*. Material *PRE* ini disimpan pada *Redis PRE* agar *Server PRE* dapat melakukan *re-encryption* ketika petugas mengakses data pasien. *TTL* untuk material *PRE* dihitung berdasarkan waktu kedaluwarsa aktual dari token yang diterbitkan.

Permintaan penerbitan akses kemudian dikirim ke *endpoint /keys*. *Payload* yang dikirim mencakup alamat IOTA pasien, alamat IOTA petugas, *hospital.cid*, *encounter_dataset*, mode akses, *signature* pasien, *k_frag*, *data_pre_public_key*, *enc_data_pre_secret_key_seed*, *data_pre_secret_key_seed.capsule*, *patient_pre_public key*, dan *signer_pre_public_key*. *Server PRE* kemudian memverifikasi *signature* pasien terhadap *nonce* dan memeriksa apakah *role* petugas penerima akses adalah *AdministrativePersonnel*.

Jika verifikasi berhasil, *Server PRE* membentuk *MacaroonKey* dari *RootKey* dan menerbitkan token *Macaroon* sesuai mode akses yang diminta. Jika mode yang dipilih adalah *read*, *Server PRE* hanya menerbitkan token *Read*. Jika mode yang dipilih adalah *update*, *Server PRE* hanya menerbitkan token *Update*. Jika mode yang dipilih adalah *read_update*, *Server PRE* menerbitkan token *Read* dan token

Update.

Token Read memiliki *caveat* `patient_address`, `root_subject`, `read_dataset_in`, `read_function_in`, `expires_before`, `max_delegation_depth`, `hospital_cid`, dan `purpose = Read`. Nilai `read_dataset_in` dan `read_function_in` pada token ini mencakup seluruh *dataset category* dan seluruh *function category*. Dengan demikian, token Read memberikan kemampuan baca terhadap data RME pasien.

Token Update memiliki *caveat* `patient_address`, `related_rme_id`, `root_subject`, `write_dataset_in`, `write_function_in`, `expires_before`, `max_delegation_depth`, `hospital_cid`, dan `purpose = Update`. Berbeda dengan token Read, cakupan `write_dataset_in` pada token Update tidak mencakup seluruh *dataset category*. Nilainya dibatasi pada `encounter_dataset` yang dipilih pasien, ditambah LABORATORIUM dan APOTEK. Nilai `write_function_in` diturunkan dari *function category* yang termasuk ke dalam cakupan *dataset category* tersebut.

`related_rme_id` hanya dibuat apabila akses yang diterbitkan mencakup kemampuan Update. Pada kondisi tersebut, *Server PRE* membuat `related_rme_id` baru sebagai identitas episode rekam medis yang sedang dibuat atau diperbarui. ID tersebut menggunakan format RME-`{sequence_6_digit}`, misalnya RME-000001. Perhitungan *sequence* dilakukan pada *Server PRE* dengan memanfaatkan Redis untuk menyimpan nilai *sequence*. Apabila pasien hanya memberikan akses read, *Server PRE* tidak membuat `related_rme_id`.

Setelah token diterbitkan, *Server PRE* menyimpan material akses PRE ke Redis PRE. Entri PRE menggunakan *key* `{personnel_iota_address}@{patient_iota_address}`. *Server PRE* kemudian menghitung *hash* dari token yang diterbitkan. PRE lalu mengembalikan `access_token_read`, `access_token_update`, *hash* masing-masing token, serta `related_rme_id` kepada *Patient Client*.

Pada tahap akhir, *Patient Client* mengenkripsi token dan metadata akses menggunakan `personnel_pre_public_key`. Metadata akses kemudian dicatat ke IOTA melalui *smart contract* `create_access`. Dengan demikian, token Macaroon awal telah dirilis kepada `AdministrativePersonnel` secara terenkripsi, *hash*

token telah tersedia untuk keperluan audit dan revokasi, dan material PRE telah tersimpan di Redis PRE untuk proses akses data berikutnya.

IV.2.5.2 Delegasi Akses antar Tenaga Kesehatan

Implementasi delegasi akses antar tenaga kesehatan merupakan realisasi dari rancangan pada Subbab III.3.5.2 dan III.3.5.3. Tenaga medis memiliki *role* `MedicalPersonnel` dengan *subrole* masing-masing sesuai dengan tabel III.11.

Proses diawali ketika *delegator* memilih *delegatee* yang akan menerima akses. Informasi kandidat *delegatee* diperoleh melalui *smart contract* `get_delegatee_candidates`. Data yang digunakan dalam proses delegasi mencakup alamat IOTA *delegatee* dan PRE *public key* milik *delegatee*. PRE *public key* tersebut digunakan untuk mengenkripsi metadata delegasi dan material akses agar hanya dapat dibuka oleh *delegatee*.

Sebelum membentuk token turunan, *Hospital Client* memuat material kriptografis milik *delegator*, yaitu PRE *secret key*, alamat IOTA, dan pasangan kunci IOTA. *Delegator* kemudian mendekripsi `data_pre_secret_key_seed` yang terenkripsi, lalu mengenkripsi ulang *seed* tersebut menggunakan PRE *public key* milik *delegatee*. Proses ini memungkinkan *delegatee* memperoleh material yang diperlukan untuk membuka data yang didelegasikan.

Hospital client kemudian mengirim *request* atenuasi token ke *server* PRE. *request* memuat token induk, *requested scope*, *expiry*, alamat *delegator/delegatee/patient*, *parent delegation signature* bila token induk merupakan token hasil delegasi, serta *delegation_request_signature*. Signature terakhir merupakan tanda tangan IOTA *delegator* atas `DelegationRequestProofContext` dan digunakan PRE untuk memastikan bahwa permintaan atenuasi benar-benar disetujui oleh *delegator*.

Pertama PRE memverifikasi autentikasi token induk melalui mekanisme *chained-HMAC*, mengecek apakah token ada di daftar yang sudah direvokasi, dan dilanjutkan dengan pemeriksaan `EffectiveCapability` `EffectiveCapability` dari token induk divalidasi terhadap *requested scope*. Validasi ini memastikan bahwa waktu kedaluwarsa token turunan tidak melebihi token induk, cakupan *dataset* dan *function* merupakan subset dari token induk, serta nilai `max_delegation_depth` tidak bertambah. Definisi `EffectiveCapability` dapat

dilihat pada IV.6.

Pemeriksaan kemudian dilanjutkan dengan pemeriksaan metadata akses pada IOTA melalui `get_delegation_access_snapshot`. Pemeriksaan ini memeriksa `expiry` dari hak akses yang dimiliki oleh *delegator*, fasyankes sama, *delegation depth* saat ini, dan memeriksa apakah terdapat delegasi aktif terhadap sebuah *subrole* yang telah didelegasikan oleh *delegator*. Jika sudah ada, maka *request* ditolak.

Apabila verifikasi berhasil, *server* PRE melakukan atenuasi pada token induk dengan menambahkan *caveat* melalui fungsi `attenuate_macaroon` dari *crate* `decmed-macaroon-auth`. Fungsi ini menerima token Macaroon induk dan parameter atenuasi berupa alamat *delegator*, alamat *delegatee*, cakupan *dataset category*, cakupan *function category*, waktu kedaluwarsa, dan batas kedalaman delegasi. *Server* PRE kemudian mengembalikan token turunan untuk diproses lebih lanjut oleh *client delegator* untuk dicatat pada penyimpanan *on-chain*. *Response* ke *client delegator* memuat token turunan, *hash* token turunan, dan konteks bukti delegasi yang akan menjadi bukti delegasi.

Caveat yang mungkin ditambahkan pada token turunan adalah `parent_token_hash`, `delegated_by`, `delegated_to`, `read_dataset_in`, `write_dataset_in`, `read_function_in`, `write_function_in`, `expires_before`, dan `max_delegation_depth`. Jika mengikuti *preset* pada *hospital client*, maka *caveats* yang ditambahkan dapat dilihat pada Lampiran H sesuai dengan alur delegasinya.

Setelah token turunan diterima oleh *client delegator*, *client* membuat bukti delegasi menggunakan `DelegationProofContext`. Struktur `DelegationProofContext` ditunjukkan pada Kode IV.5. Nilai `scope_fingerprint` merupakan *hash* dengan struktur `r:{ds_read}|w:{ds_write}|rf:{fn_read}|wf:{fn_write}`, dengan `ds_read`, `ds_write`, `fn_read`, dan `fn_write` merupakan *dataset category* dan *function category* yang didelegasikan. `DelegationProofContext` kemudian diserialisasi dan ditandatangani menggunakan kunci IOTA milik *delegator*.

```
1 pub struct DelegationProofContext {
2     pub token_hash: String,
3     pub delegated_by: String,
4     pub delegated_to: String,
```

```

5     pub related_rme_id: String,
6     pub expires_before: String,
7     pub scope_fingerprint: String,
8 }

```

Kode IV.5: Definisi DelegationProofContext

Token turunan, *hash* token, tanda tangan delegasi, metadata akses, dan *data_pre_secret_key_seed* yang telah dienkripsi ulang kemudian dikemas menjadi metadata delegasi. Metadata tersebut dienkripsi menggunakan PRE *public key* milik *delegatee*. Dengan demikian, token dan material akses hanya dapat dibuka oleh *delegatee*.

Tahap berikutnya adalah pencatatan delegasi ke IOTA melalui *smart contract* fungsi *create_delegated_access*. Fungsi ini menerima metadata delegasi, data audit delegasi, alamat pasien, dan alamat *delegatee*. Pada sisi *smart contract*, dilakukan validasi bahwa *delegatee* terdaftar dan aktif, keduanya berada pada fasyankes yang sama, serta *delegator* masih memiliki akses yang valid terhadap pasien terkait. Jika validasi berhasil, akses baru untuk *delegatee* dimasukkan ke tabel akses Read dan Update.

Selain menyimpan akses untuk *delegatee*, fungsi *create_delegated_access* juga mencatat audit delegasi pada data pasien. Entri audit tersebut memuat alamat *delegator*, *root subject* dari rantai delegasi, alamat *delegatee*, jenis akses, *related_rme_id*, kedalaman delegasi, *hash* token turunan, *hash* token induk, waktu kedaluwarsa. Dengan demikian, perpindahan akses antar tenaga kesehatan tetap dapat ditelusuri tanpa membuka isi token Macaroon dan metadata akses yang terenkripsi.

Khusus untuk delegasi dari *AdministrativePersonnel* kepada *MedicalPersonnel*, delegasi dari *AdministrativePersonnel* pertama akan menambahkan *function category* ADMINISTRATIVE_GENERAL pada setiap *dataset category*. Hal ini mengasumsikan bahwa *AdministrativePersonnel* sudah memeriksa data ADMINISTRATIVE_GENERAL pasien, sehingga siap dicatat pada setiap *dataset category*. Hal ini dilakukan agar sesuai dengan Lampiran A yang memuat dokumentasi administratif pasien pada masing-masing *dataset category*.

Ketika *delegatee* menggunakan akses yang telah didelegasikan, *request* melewati *middleware* otorisasi pada *Server PRE*. *Server PRE* memverifikasi sesuai dengan implementasi yang dijelaskan pada subbab IV.2.6. Untuk memperoleh material *PRE*, *Server PRE* mencari entri Redis berdasarkan subjek aktif, aktor pada rantai delegasi, hingga *root_subject*. Mekanisme ini memungkinkan token turunan menggunakan material *PRE* dari akses awal, sementara pembatasan akses tetap ditegakkan dengan memeriksa *Macaroon*.

Dengan mekanisme tersebut, delegasi antar tenaga kesehatan dapat dilakukan oleh *delegator* melalui atenuasi *Macaroon* di *server PRE*. Hak akses *delegatee* tetap tidak dapat melebihi hak akses *delegator*, proses delegasi tercatat pada *smart contract*, dan penggunaan akses berikutnya tetap diverifikasi oleh *Server PRE*.

IV.2.6 Implementasi Penegakan Otoritas

Penegakan otoritas pada *Server PRE* diimplementasikan sebagai tahap validasi berlapis sebelum operasi terhadap data RME dijalankan. Secara umum, mekanisme ini memastikan bahwa setiap *request* yang membutuhkan otorisasi membawa *Macaroon* yang valid, belum kedaluwarsa, belum dicabut, sesuai dengan cakupan akses yang diminta, dan digunakan oleh aktor aktif yang sah.

Alur implementasi penegakan otoritas pada *Server PRE* disajikan sebagai referensi pada Lampiran M. Bagian awal alur pada Gambar M.2 mencakup autentikasi token, perhitungan *effective capability*, pembentukan *delegation chain*, dan pemeriksaan revokasi. Kelanjutan alur pada Gambar M.3 mencakup validasi *role* dan *subrole*, validasi konteks segmen, verifikasi bukti kepemilikan *wallet*, serta eksekusi operasi *PRE*, *IPFS*, dan *IOTA*. Status implementasi penegakan otoritas pada *Server PRE* disajikan pada Tabel IV.3.

IV.2.6.1 Autentikasi Token Macaroon

Tahap awal validasi dilakukan pada *middleware PRE*. *Middleware* membaca token *Macaroon* dari *header Authorization* dengan skema *Bearer*, melakukan deserialisasi token, mem-parsing *caveats DecMed*, serta memverifikasi keaslian token menggunakan mekanisme *chained-HMAC* dengan *RootKey PRE*. Apabila token tidak dapat dideserialisasi, bukan merupakan token *DecMed*, atau nilai

Tabel IV.3: Status Implementasi Penegakan Otoritas pada *Server* PRE

Bagian	Status
Validasi <i>Bearer Macaroon</i>	Terimplementasi.
Validasi <i>chained-HMAC</i> pada token Macaroon	Terimplementasi.
<i>Parsing caveats</i> pada token Macaroon	Terimplementasi.
Perhitungan <i>EffectiveCapability</i>	Terimplementasi.
Penentuan aktor aktif melalui <i>DelegationChain</i>	Terimplementasi.
Pemeriksaan revokasi token melalui Redis	Terimplementasi.
Pemeriksaan <i>role</i> dan <i>subrole</i> aktor melalui <i>Move Smart Contract</i>	Terimplementasi.
Validasi aksi baca dan tulis terhadap cakupan <i>dataset</i> dan <i>function</i>	Terimplementasi.
Verifikasi <i>wallet signature</i> terhadap aktor aktif dan konteks akses segmen	Terimplementasi.

HMAC tidak valid, *request* langsung ditolak.

Setelah token dinyatakan autentik, *Server* PRE membentuk *EffectiveCapability* dari seluruh *caveats* yang terdapat pada token. Struktur ini merepresentasikan cakupan akses efektif yang berlaku setelah proses pembatasan dan penggabungan *caveats*. Untuk *caveat* berbentuk *set*, seperti *read_dataset_in*, *write_dataset_in*, *read_function_in*, dan *write_function_in*, cakupan efektif diperoleh melalui irisan antar-*caveats*. Untuk *expires_before*, waktu kedaluwarsa efektif diambil dari nilai paling awal. Struktur *EffectiveCapability* ditunjukkan pada Kode IV.6.

```

1 pub struct EffectiveCapability {
2     pub read_datasets: HashSet<DatasetCategory>,
3     pub write_datasets: HashSet<DatasetCategory>,
4     pub read_functions: HashSet<FunctionCategory>,
5     pub write_functions: HashSet<FunctionCategory>,
6     pub expires_before: Option<DateTime<Utc>>,
7     pub root_max_delegation_depth: Option<u32>,
8     pub remaining_max_delegation_depth: Option<u32>,
9     pub patient_address: Option<String>,
10    pub related_rme_id: Option<String>,
11    pub hospital_cid: Option<String>,
12 }

```

Kode IV.6: Definisi *EffectiveCapability*

Selain menghitung *EffectiveCapability*, *Server* PRE membentuk

DelegationChain untuk menentukan aktor aktif dari token. Jika token belum didelegasikan, aktor aktif adalah `root_subject`. Jika token telah didelegasikan, aktor aktif adalah nilai `delegated_to` pada langkah delegasi terakhir. Rantai delegasi juga divalidasi agar setiap langkah delegasi tersambung secara konsisten, yaitu penerima delegasi pada satu langkah harus menjadi pemberi delegasi pada langkah berikutnya. Struktur DelegationChain ditunjukkan pada Kode IV.7.

```
1 pub struct DelegationStep {
2     pub delegated_by: String,
3     pub delegated_to: String,
4 }
5
6 pub struct DelegationChain {
7     pub root_subject: String,
8     pub steps: Vec<DelegationStep>,
9     pub active_subject: String,
10 }
```

Kode IV.7: Definisi DelegationStep dan DelegationChain

Setelah `EffectiveCapability` dan `DelegationChain` terbentuk, *Server* PRE memeriksa waktu kedaluwarsa token. Token ditolak apabila nilai `expires_before` pada `EffectiveCapability` telah melewati waktu saat *request* diproses. *Server* PRE juga memeriksa batas kedalaman delegasi sudah melewati `max_delegation_depth` dengan menghitung jumlah `steps` pada `DelegationChain` untuk memastikan jumlah delegasi yang digunakan tidak melebihi batas yang ditentukan. Apabila ada yang tidak terpenuhi, *request* langsung ditolak.

IV.2.6.2 Pemeriksaan Status Revokasi Token

Pemeriksaan revokasi token dilakukan melalui Redis. *Server* PRE menghitung *hash* dari token yang sedang digunakan, kemudian membentuk kunci revokasi dengan format `revoked:token:<token_hash>`. Selain token yang sedang digunakan, *Server* PRE juga memeriksa *hash* token induk yang tersimpan dalam *caveat* `parent_token_hash`. Dengan demikian, pencabutan token induk juga dapat menyebabkan token hasil delegasi atau atenuasi ditolak.

IV.2.6.3 Verifikasi *Role* dan *Subrole* melalui *Move Smart Contract*

Setelah token dinyatakan valid dan tidak dicabut, *Server PRE* memeriksa identitas kelembagaan aktor aktif melalui *smart contract* `get_hospital_personnel_auth_info` menggunakan alamat IOTA aktor aktif. Fungsi ini mengembalikan *role* dan *subrole* personel yang tersimpan pada penyimpanan *on-chain*. Eksistensi aktor aktif dalam sistem juga tervalidasi melalui pemeriksaan ini.

IV.2.6.4 Verifikasi Konteks Segmen

Validasi konteks segmen dilakukan pada *handler* operasi baca dan tulis. Pada operasi baca, *handler* mengambil metadata segmen dari IOTA, kemudian mendekode metadata JSON yang tersimpan dalam format base64. Pada operasi tulis, *handler* membentuk metadata *preview* dari *payload* segmen sebelum data disimpan. Metadata ini digunakan untuk membangun `SegmentAccessContext` yang berisi `segment_id`, `patient_address`, `related_rme_id`, `dataset_category`, dan `function_category`.

Pemeriksaan akses segmen dilakukan dengan membandingkan konteks segmen terhadap `EffectiveCapability`. Pada operasi baca, *Server PRE* memastikan bahwa `patient_address` sesuai dengan token, serta `dataset_category` dan `function_category` berada dalam cakupan `read_dataset_in` dan `read_function_in`. Pada operasi tulis, *Server PRE* memastikan bahwa `patient_address` sesuai, `related_rme_id` sesuai, serta `dataset_category` dan `function_category` berada dalam cakupan `write_dataset_in` dan `write_function_in`.

Untuk operasi tulis segmen, *Server PRE* juga melakukan pemeriksaan tambahan berdasarkan *role* dan *subrole* aktor aktif. Aktor dengan *role* `MedicalPersonnel` hanya dapat menulis kategori dataset yang sesuai dengan *subrole*-nya. Dokter dan perawat dapat menulis dataset `RAWAT_JALAN` dan `RAWAT_INAP`, petugas laboratorium dapat menulis dataset `LABORATORIUM`, sedangkan apoteker dapat menulis dataset `APOTEK`. Aktor dengan *role* `AdministrativePersonnel` hanya dapat membuat segmen dengan *function category* `ADMINISTRATIVE_GENERAL`. Selain itu, pada operasi tulis, *Server PRE* memeriksa kesesuaian `hospital_cid`

pada token dengan `hospital_cid` pada segmen yang akan disimpan.

IV.2.6.5 Verifikasi Bukti Kepemilikan *Wallet*

Setelah konteks akses dinyatakan sesuai, *Server PRE* memverifikasi bukti kepemilikan *wallet* dari aktor aktif. Bukti ini dikirimkan melalui *header* `x-decmed-wallet-signature`, sedangkan waktu pembuktian dikirimkan melalui *header* `x-decmed-wallet-timestamp`. Tanda tangan dibuat terhadap representasi `WalletProofContext`. Struktur `WalletProofContext` ditunjukkan pada Kode IV.8. *Server PRE* kemudian memverifikasi *wallet signature* tersebut terhadap alamat IOTA aktor aktif yang diperoleh dari `DelegationChain`. Timestamp pada konteks pembuktian juga divalidasi agar berada dalam rentang waktu yang dapat diterima. Pada implementasi ini, waktu yang diterima adalah rentang waktu 5 menit.

```
1 pub struct WalletProofContext {  
2     pub token_id: String,          // ID Macaroon  
3     pub patient_address: String, // Alamat pasien  
4     pub related_rme_id: String, // ID episode RME  
5     pub operation: AccessMode,    // Read / Update  
6     pub segment_id: String,       // ID segmen yang diakses  
7     pub dataset_category: DatasetCategory,  
8     pub function_category: FunctionCategory,  
9     pub timestamp: String,        // YYYY-MM-DD HH:MM:SS  
10 }
```

Kode IV.8: Struktur `WalletProofContext` sebagai konteks verifikasi *wallet signature*

Pada jalur operasi baca, apabila *wallet signature* belum disertakan, *Server PRE* mengembalikan respons 401 UNAUTHORIZED yang berisi `proof_context`. *Client* kemudian dapat menandatangani konteks tersebut menggunakan kunci privat *wallet* aktor aktif dan mengirimkan ulang *request* dengan menyertakan `x-decmed-wallet-signature` serta `x-decmed-wallet-timestamp`.

IV.2.6.6 Eksekusi Operasi *Re-encryption*, IPFS, dan IOTA

Setelah seluruh pemeriksaan otorisasi selesai, *Server PRE* melanjutkan permintaan ke operasi utama sesuai jenis aksi yang diminta. Pada tahap ini, *Server PRE*

berperan sebagai penghubung antara data terenkripsi yang tersimpan di IPFS, metadata segmen yang tersimpan di IOTA, dan mekanisme *proxy re-encryption*.

Pada operasi baca, *Server PRE* terlebih dahulu mengambil metadata RME dari IOTA berdasarkan pasien dan indeks data yang diminta. Metadata tersebut digunakan untuk memperoleh informasi penyimpanan data, seperti *ipfs_cid*, *capsule* dan *enc_key_and_nonce*. Setelah metadata diperoleh, *Server PRE* mengambil data medis terenkripsi dari IPFS menggunakan *ipfs_cid* pada metadata. Selanjutnya, *Server PRE* memuat material PRE yang diperlukan, yaitu *kfrag*, kunci publik pasien, kunci publik penanda tangan, dan kunci publik data RME.

Proses *re-encryption* dilakukan dengan Umbral PRE terhadap *capsule* yang terkait dengan data medis. Hasil proses ini adalah *cfrag*, yaitu fragmen kapsul yang memungkinkan *client* penerima membuka data terenkripsi.

Response yang dikembalikan ke *client* untuk operasi baca berisi data medis terenkripsi, *cfrag* hasil *re-encryption*, informasi kunci publik yang diperlukan oleh *client*, *capsule*, *encrypted key and nonce*, dan waktu pembuatan data.

Pada operasi tulis, *client* mengirimkan segmen RME yang sudah terenkripsi. *Server PRE* menyimpan *payload* terenkripsi ke IPFS. Setelah penyimpanan berhasil, IPFS mengembalikan CID sebagai penunjuk terhadap data terenkripsi tersebut.

CID dari IPFS kemudian dimasukkan ke dalam metadata segmen final. Metadata ini memuat informasi sesuai dengan skema pada Lampiran G. Setelah metadata final terbentuk, *Server PRE* menyimpannya ke IOTA melalui *smart contract create_medical_record_segment*.

Response yang dikembalikan ke *client* untuk operasi tulis berisi ringkasan metadata segmen yang baru dibuat, yaitu *segment_id*, *related_rme_id*, *dataset_category*, *function_category*, *ipfs_cid*, *integrity_hash*, dan *created_at*. *Response* ini menjadi konfirmasi bahwa data terenkripsi telah tersimpan di IPFS dan metadata referensinya telah dicatat di IOTA.

Pada operasi *list*, *Server PRE* mengambil kumpulan metadata dari IOTA secara bertahap menggunakan mekanisme *pagination*. Setiap metadata yang berhasil di-*decode* RME kemudian diubah menjadi *list item* yang berisi informasi ringkas,

yaitu `index`, `segment_id`, `related_rme_id`, alamat pasien, `dataset category`, `function category`, `ipfs_cid`, `created_at` dan `author_address`. Daftar metadata tersebut kemudian diproses agar hanya menampilkan metadata aktif sesuai dengan `EffectiveCapability`. Setelah itu, hasilnya diurutkan dan dipotong sesuai parameter `cursor` dan `limit` dari *request*. *Response* yang dikembalikan berisi daftar item metadata aktif dan `next_cursor` apabila masih terdapat data lanjutan yang dapat diambil oleh *client*.

Dengan alur ini, IPFS digunakan untuk menyimpan isi data RME terenkripsi, sedangkan IOTA digunakan untuk menyimpan metadata referensial yang diperlukan untuk pencarian, validasi konteks, dan navigasi data. *Server PRE* menghubungkan keduanya dengan mekanisme *Umbral PRE* pada operasi baca, pencatatan metadata pada operasi tulis, dan penyusunan daftar metadata aktif pada operasi *list*.

IV.2.7 Implementasi *Smart Contract* IOTA Move

Smart contract IOTA Move diimplementasikan untuk menyimpan metadata yang diperlukan oleh mekanisme segmentasi dan kontrol akses. Data RME tetap disimpan secara *off-chain* pada IPFS, sedangkan IOTA menyimpan informasi yang dibutuhkan untuk menemukan, memverifikasi, dan menghubungkan segmen data yang diakses dengan pemiliknya.

Pada implementasi Tugas Akhir ini, struktur penyimpanan metadata klinis di Move masih bersifat flat melalui `medical.metadata: TableVec<PatientMedicalMetadata>`. Struktur `PatientMedicalMetadata` menyimpan `index` dan metadata. Informasi segmentasi seperti `dataset_category`, `function_category`, `related_rme_id`, `CID IPFS`, `integrity_hash`, dan `capsule` disimpan di dalam metadata sebagai JSON base64. Dengan cara ini, *Server PRE* dapat mengambil metadata segmen dari IOTA sebelum memutuskan apakah token *Macaroon* berhak mengakses segmen tersebut.

Fungsi pada *smart contract* disesuaikan untuk mendukung pembuatan metadata RME, penambahan segmen baru, pembaruan referensi segmen, pembacaan metadata, penyimpanan metadata akses personel, serta pencatatan audit delegasi.

Operasi yang mengubah metadata hanya dapat dilakukan oleh aktor yang memiliki akses tulis yang sah pada lapisan aplikasi dan *smart contract*. Rincian *module* dan *method* pada implementasi *smart contract* IOTA Move disajikan pada Lampiran O.

IV.2.8 Implementasi *Patient Client*

Patient Client disesuaikan untuk mendukung skenario penerbitan token awal oleh pasien, pembacaan RME, peninjauan audit akses, dan pencabutan akses. *Patient Client* tidak hanya berfungsi sebagai tampilan pengguna, tetapi juga sebagai titik pembentukan konteks permintaan pasien yang dikirimkan ke *Server PRE*. Tangkapan layar implementasi disajikan pada Lampiran P.

Terdapat beberapa bagian dari *Patient Client* yang sudah ada dari implementasi DecMed sebelumnya, seperti halaman register, halaman login, halaman *Home*, halaman Profil, Halaman log akses, halaman *scan QR*, dan halaman detail RME. Bagian-bagian tersebut diadaptasi dan disesuaikan untuk mendukung implementasi Tugas Akhir ini. *Patient Client* mengalami penyesuaian dan tambahan halaman baru pada implementasi Tugas Akhir ini untuk beberapa halaman berikut,

IV.2.8.1 Halaman *Home* (*List RME*)

Halaman *home* merupakan halaman utama yang pertama kali dilihat pasien setelah berhasil masuk ke dalam aplikasi. Halaman ini menampilkan seluruh RME pasien dalam bentuk daftar *card* yang tersusun secara vertikal. Setiap *card* merepresentasikan satu RME unik dan menampilkan tanggal pembuatan RME dan ID RME. Daftar RME diurutkan berdasarkan tanggal terbaru, sehingga RME yang paling baru selalu muncul di bagian atas. Setiap *card* dapat diklik, di mana klik tersebut akan mengarahkan pasien menuju halaman detail RME yang bersangkutan.

Tampilan dari halaman *home* dapat dilihat pada Gambar P.1.

IV.2.8.2 Halaman Detail RME

Halaman detail RME ditampilkan setelah pasien memilih salah satu *card* dari daftar RME pada halaman *home*. Halaman ini menampilkan detail lengkap dari sebuah RME per dataset. Bagian atas halaman menampilkan ID dari RME. Di bawahnya, terdapat *tab* daftar *dataset category* RME tersebut. *Tab* yang terpilih menentukan

entri data RME yang ditampilkan dalam bentuk *card* yang memuat nama *function category*. Pasien juga dapat kembali ke halaman *home* melalui tautan navigasi balik yang tersedia di bagian atas halaman.

Tampilan dari halaman detail RME dapat dilihat pada Gambar P.2.

IV.2.8.3 Halaman Audit Delegasi

Halaman audit delegasi menampilkan informasi mengenai seluruh rantai delegasi akses yang terbentuk dari token Macaroon yang telah diterbitkan oleh pasien. Setiap entri pada halaman ini merepresentasikan satu rantai delegasi dan menampilkan identitas personel rumah sakit yang menjadi penerima akses awal, jenis akses yang diberikan, ID RME yang terkait, serta tanggal kedaluwarsa dari akses awal tersebut. Status keseluruhan rantai delegasi juga ditampilkan dalam bentuk label berwarna, yakni *Active* berwarna hijau, *Expired* berwarna kuning, atau *Revoked* berwarna merah. Di dalam setiap entri rantai delegasi, terdapat subdaftar yang merinci setiap delegasi, yaitu pasangan delegasi dari satu personel ke personel lainnya, beserta informasi kedalaman delegasi, tanggal kedaluwarsa, dan status aktif atau dicabutnya delegasi tersebut.

Tampilan dari halaman detail audit delegasi RME dapat dilihat pada Gambar P.3.

IV.2.8.4 Halaman Detail Scan QR

Halaman *scan QR* merupakan halaman yang digunakan pasien untuk memulai proses pemberian akses kepada personel rumah sakit. Halaman ini menyediakan antarmuka unggah berkas gambar QR yang diperoleh dari rumah sakit, di mana pasien dapat memilih berkas gambar dalam format PNG atau JPEG. Setelah berkas QR diunggah dan diproses, sistem akan memvalidasi isi QR dan mengidentifikasi identitas personel rumah sakit yang terkandung di dalamnya. Apabila QR valid, sebuah dialog konfirmasi akan muncul yang menampilkan nama personel dan nama rumah sakit tujuan pemberian akses. Pada dialog konfirmasi ini, pasien dapat mengonfigurasi cakupan akses yang akan diberikan, meliputi jenis layanan medis berupa rawat jalan atau rawat inap, mode akses berupa *Read*, *WRITE*, atau keduanya, serta tanggal dan waktu kedaluwarsa akses. Setelah mengonfirmasi pilihan tersebut, pasien akan diarahkan ke dialog masukan PIN enam digit sebagai verifikasi identitas sebelum token Macaroon awal diterbitkan dan akses resmi

diberikan kepada personel rumah sakit yang dituju.

Tampilan dari halaman *scan* QR dan halaman konfirmasi akses awal dapat dilihat pada Gambar P.4 dan Gambar P.5.

IV.2.9 Implementasi *Hospital Client*

Hospital Client disesuaikan untuk mendukung skenario penggunaan token oleh tenaga medis, pembuatan token turunan, dan akses terhadap segmen data RME. Tangkapan layar implementasi disajikan pada Lampiran Q.

IV.2.9.1 Halaman *Dashboard* (*List RME*)

Halaman *dashboard* merupakan halaman utama yang ditampilkan setelah aktor rumah sakit berhasil masuk ke sistem. Konten halaman ini bersifat *role-based*, sehingga tampilan yang disajikan berbeda-beda bergantung pada peran aktor yang sedang aktif. Untuk peran `MedicalPersonnel`, halaman ini menampilkan dua tab yang dapat dipilih secara bergantian, yaitu tab *Read* dan tab *Write*.

Tab *Read* menampilkan daftar pasien yang token aksesnya memiliki kapabilitas baca, sedangkan tab *Write* menampilkan daftar pasien yang token aksesnya memiliki kapabilitas tulis. Setiap entri dalam kedua tab tersebut menampilkan nama pasien dan dapat diklik untuk menavigasi ke halaman berikutnya. Untuk entri pada tab *Read*, klik akan membawa aktor menuju halaman daftar RME pasien yang terkait dengan token baca. Untuk entri pada tab *Write*, klik akan membawa aktor langsung menuju halaman formulir penulisan segmen RME, dengan konteks token tulis yang sesuai.

Tampilan dari halaman *dashboard* untuk peran `MedicalPersonnel` dapat dilihat pada Gambar Q.1.

IV.2.9.2 Halaman Delegasi Akses

Halaman delegasi akses digunakan oleh aktor yang memiliki hak untuk mendelegasikan token aksesnya kepada personel lain di fasyankes. Pada atas halaman ini terdapat ringkasan hak akses yang saat ini dimiliki oleh aktor, berupa daftar pasien. Di bawahnya adalah *form* delegasi yang aktif apabila terdapat setidaknya satu akses pasien yang tersedia. Pada formulir ini, aktor dapat memilih

pasien sebagai hak akses yang akan didelegasikan, menentukan mode delegasi berupa *Read*, *Write*, atau keduanya, memilih *preset role* penerima delegasi seperti dokter atau perawat, serta menetapkan tanggal dan waktu kedaluwarsa delegasi. Sistem secara otomatis membatasi waktu kedaluwarsa maksimum token induk yang sedang digunakan. Formulir juga menyediakan pratinjau cakupan baca dan tulis berupa daftar *dataset category* dan *function category* yang dapat dipilih atau dihilangkan secara individual sesuai kapabilitas token induk. Terakhir, aktor memilih penerima delegasi dari daftar kandidat personel yang tersedia di fasyankes, kemudian menekan tombol *Delegate* untuk memulai proses delegasi.

Tampilan halaman delegasi akses dapat dilihat pada Gambar Q.2 dan Gambar Q.3.

IV.2.9.3 Halaman Daftar RME Pasien

Halaman daftar RME pasien menampilkan seluruh RME yang dimiliki oleh pasien tertentu dan dapat diakses oleh aktor sesuai token yang dibawa. Bagian atas halaman menampilkan nama pasien sebagai judul halaman. Di bawahnya, terdapat daftar RME dalam bentuk *card* yang masing-masing memuat ID RME, tanggal pembuatannya, serta kumpulan label *badge* yang merepresentasikan kategori dataset yang terdapat dalam RME tersebut. Setiap *card* RME dapat diklik dan akan mengarahkan aktor menuju halaman detail RME dari RME yang dipilih, dengan konteks token akses yang diteruskan melalui *query string*. Apabila tidak terdapat RME yang dapat diakses, halaman menampilkan pesan kosong.

Tampilan halaman daftar RME pasien dapat dilihat pada Gambar Q.4.

IV.2.9.4 Halaman Baca Detail RME

Halaman detail RME menampilkan isi lengkap dari sebuah RME yang dipilih oleh aktor. Bagian atas halaman memuat ID RME sebagai identifikasi, diikuti oleh tab navigasi yang merepresentasikan masing-masing *dataset category* yang terdapat dalam RME tersebut. Aktor dapat berpindah antarklasifikasi dataset dengan mengklik tab yang sesuai. Pada setiap tab dataset, ditampilkan daftar kartu segmen yang memuat nama *function category* dari segmen tersebut. Setiap kartu segmen dilengkapi dengan tombol ikon informasi yang dapat diklik untuk menampilkan atau menyembunyikan metadata. Konten payload segmen dimuat secara *on-demand* dan ditampilkan dalam bentuk *textarea* hanya-baca, atau dalam format tabel data

khusus apabila segmen termasuk *function category* ADMINISTRATIVE_GENERAL.

Tampilan halaman detail RME dapat dilihat pada Gambar Q.5.

IV.2.9.5 Halaman Tulis Segmen RME

Halaman penulisan segmen RME digunakan ketika aktor memiliki kapabilitas tulis terhadap RME pasien tertentu. Halaman ini dapat diakses dari daftar pasien pada tab *Write* di halaman *dashboard*.

Bagian atas halaman menampilkan nama pasien sebagai konteks identifikasi. Di bawahnya, terdapat menu pilihan *dataset category* yang hanya memuat kategori yang diizinkan oleh token tulis yang aktif. Halaman secara otomatis menampilkan daftar formulir *textarea* yang bersesuaian dengan *function category* yang diizinkan dalam dataset tersebut, di mana setiap *textarea* merepresentasikan satu jenis data klinis yang dapat diisikan. Aktor hanya perlu mengisi *textarea* untuk *function category* yang ingin dituliskan, karena hanya entri yang terisi yang akan dikirimkan. Setelah mengisi data yang diperlukan, aktor menekan tombol *Write* untuk menyimpan segmen-segmen baru ke dalam RME pasien.

Tampilan halaman penulisan segmen RME dapat dilihat pada Gambar Q.6.

IV.3 Pengujian Sistem

Pengujian sistem dilakukan untuk memastikan implementasi memenuhi kebutuhan yang sudah didefinisikan pada Subbab III.1.3.

IV.3.1 Strategi Pengujian

Strategi pengujian disusun berdasarkan kebutuhan sistem pada Tabel III.3 dan III.4. Pengujian dibagi menjadi pengujian kebutuhan fungsional dan pengujian kebutuhan nonfungsional.

Pengujian kebutuhan fungsional diberi ID PS-TA dan dilakukan melalui skenario manual untuk memeriksa perilaku sistem dari sudut pandang aktor layanan kesehatan. Pengujian kebutuhan nonfungsional diberi ID PU-TA dan dilakukan melalui pengujian unit atau integrasi terhadap komponen token, segmentasi data,

validasi akses, dan *Server* PRE. Pengujian otomatis dilakukan dengan *cargo test* pada komponen yang relevan.

Kriteria keberhasilan pengujian ditentukan berdasarkan kesesuaian hasil skenario manual terhadap kebutuhan fungsional dan kelulusan pengujian otomatis terhadap kebutuhan nonfungsional.

IV.3.2 Pengujian Kebutuhan Fungsional

Pengujian kebutuhan fungsional dilakukan secara manual untuk memastikan perilaku sistem sesuai dengan rancangan alur akses dan delegasi pada Subbab III.3.5.

Ringkasan pengujian kebutuhan fungsional ditunjukkan pada Tabel IV.4. Terdapat 12 skenario pengujian yang dilakukan untuk alur Rawat Jalan dan Rawat Inap. Perbedaan kedua alur tersebut terdapat pada keberadaan segmen data *PERENCANAAN_PEMULANGAN* yang hanya muncul pada alur Rawat Inap. Delegasi akses yang dilakukan mengikuti *preset* yang sudah terdefinisi, menghasilkan token turunan dengan *caveats* sesuai dengan Lampiran H.

Tabel IV.4: Pengujian Kebutuhan Fungsional pada Alur Rawat Jalan dan Rawat Inap

ID	Skenario	Ekspektasi	KF
PS-TA-01	Pasien memberikan akses awal kepada petugas RM sesuai konteks episode pelayanan.	Token awal episode pelayanan berhasil diterbitkan.	KF-TA-01
PS-TA-02	Petugas RM mendelegasikan akses pemeriksaan awal kepada perawat.	Token turunan perawat berhasil dibuat.	KF-TA-04
PS-TA-03	Petugas RM mendelegasikan akses klinis kepada dokter.	Token turunan dokter berhasil dibuat.	KF-TA-04
PS-TA-04	Perawat mengisi hasil pemeriksaan awal.	Data pemeriksaan awal berhasil disimpan.	KF-TA-07

Tabel IV.4: Pengujian Kebutuhan Fungsional pada Alur Rawat Jalan dan Rawat Inap (lanjutan)

ID	Skenario	Ekspektasi	KF
PS-TA-05	Dokter membaca pemeriksaan awal dan mengisi data klinis.	Data klinis berhasil dibaca dan disimpan.	KF-TA-05
PS-TA-06	Dokter mendelegasikan akses laboratorium.	Token turunan laboratorium berhasil dibuat.	KF-TA-06
PS-TA-07	Petugas laboratorium membaca pemeriksaan penunjang dan mengisi hasil laboratorium.	Data pemeriksaan penunjang berhasil dibaca dan hasil laboratorium disimpan.	KF-TA-08
PS-TA-08	Dokter membaca hasil laboratorium.	Hasil laboratorium berhasil dibaca oleh dokter.	KF-TA-05
PS-TA-09	Dokter mendelegasikan akses apotek.	Token turunan apoteker berhasil dibuat.	KF-TA-06
PS-TA-10	Apoteker membaca resep dan mengisi status pelayanan obat.	Data resep dan pelayanan obat berhasil diproses.	KF-TA-09
PS-TA-11	Pasien membuka riwayat delegasi episode pelayanan.	Riwayat delegasi episode pelayanan ditampilkan.	KF-TA-03
PS-TA-12	Pasien merevokasi salah satu akses delegasi episode pelayanan.	Aktor <i>delegatee</i> tidak bisa mengakses data RME pasien lagi.	KF-TA-02

IV.3.3 Pengujian Kebutuhan Nonfungsional

Pengujian kebutuhan nonfungsional dilakukan untuk mengevaluasi pemenuhan aspek keamanan dan keterlacakan pada sistem, khususnya pada komponen yang menangani token Macaroon, segmentasi data RME, validasi akses, *wallet proof*, revokasi, pencatatan delegasi, dan *Server PRE*. Setiap pengujian dikaitkan dengan

kebutuhan nonfungsional yang telah didefinisikan pada Tabel III.4. Untuk memudahkan ketertelusuran evaluasi, pengujian unit dan integrasi yang mendukung kebutuhan nonfungsional diberi ID PU-TA.

IV.3.3.1 Pengujian KNF-TA-01

Pengujian KNF-TA-01 memastikan bahwa penerima akses hanya dapat memperoleh data RME dalam bentuk *plaintext* apabila segmen yang diminta berada dalam cakupan otorisasi token. Dengan demikian, permintaan terhadap segmen di luar cakupan harus ditolak sebelum proses *re-encryption* menghasilkan data yang dapat didekripsi oleh penerima akses.

Pengujian pertama, yaitu PU-TA-01, dilakukan melalui *unit testing* pada komponen PRE. Tes ini menyimulasikan kondisi ketika aktor memiliki token baca, tetapi token tersebut tidak mencakup akses baca terhadap segmen data laboratorium. Kemudian fungsi yang diuji, yaitu `verify_segment_access`, dipanggil dengan parameter Macaroon tanpa akses ke laboratorium dan segmen data laboratorium. Tes ini mengharapkan akses ditolak. Kode pengujian dapat dilihat pada Kode IV.9.

```
1  #[test]
2  fn pre_rejects_segment_outside_caveats() {
3      let (_token, mac, verified) = verified_read_token();
4      let outside_scope = segment(
5          DatasetCategory::LABORATORIUM,
6          FunctionCategory::LABORATORIUM,
7      );
8
9      let err = verify_segment_for_token(
10         ...,
11         &outside_scope,
12         &mac,
13     )
14     .unwrap_err();
15     // assert error DatasetCategoryNotAllowed
16 }
```

Kode IV.9: Pengujian PU-TA-01 untuk KNF-TA-01

Pengujian kedua dilakukan melalui *client hospital* untuk memastikan aktor apoteker tidak dapat membuka atau memperoleh segmen data laboratorium. Pada skenario ini, akun apoteker tidak memiliki caveat akses terhadap kategori laboratorium, sehingga segmen laboratorium tidak ditampilkan sebagai data yang dapat diakses.

IV.3.3.2 Pengujian KNF-TA-02

Pengujian KNF-TA-02 memastikan integritas token otorisasi dan batasan akses tetap terjaga selama proses delegasi. Secara khusus, pengujian diarahkan untuk memastikan bahwa token turunan tidak dapat memperluas cakupan akses, menaikkan kedalaman delegasi, atau memperpanjang masa berlaku melebihi token induk.

Pengujian pertama, yaitu PU-TA-02, dilakukan melalui *unit testing* pada Kode IV.10. Tes ini memastikan token delegasi tidak dapat menambahkan hak akses `write_function` yang tidak dimiliki oleh *parent token*. Ketika *child token* mencoba meminta `write_functions` di luar scope *parent token*, proses `attenuate_macaroon()` ditolak dengan error `DelegationExpandsAccess`.

```
1  #[test]
2  fn cannot_delegate_write_function_from_parent_read_only_scope() {
3      let write_functions: Vec<_> = ... // tidak mencakup PERESEPAN
4      let parent = scoped_token(
5          ...
6          &write_functions,
7          Some("Read"),
8      );
9      let params = DelegationAttenuationParams {
10         write_functions: vec![FunctionCategory::PERESEPAN],
11         ...
12     };
13     let err = attenuate_macaroon(&parent, &params).unwrap_err();
14     // assert error DelegationExpandsAccess
15 }
```

Kode IV.10: Pengujian PU-TA-02 untuk KNF-TA-02

Pengujian kedua, yaitu PU-TA-03, dilakukan melalui *unit testing* pada

Kode IV.11. Tes ini memastikan batas kedalaman delegasi tidak dapat dinaikkan oleh token turunan. Jika *child token* mencoba menetapkan `max_delegation_depth` lebih besar dari batas *parent token*, delegasi ditolak dengan error `DelegationDepthNotMonotonic`.

```
1  #[test]
2  fn cannot_increase_max_delegation_depth_on_delegate() {
3      let mut params = DelegationAttenuationParams {
4          ...,
5          max_delegation_depth: 99
6      };
7      let err = attenuate_macaroon(
8          &doctor_token(), // max depth 1
9          &params
10     )
11     // assert error DelegationDepthNotMonotonic
12 }
```

Kode IV.11: Pengujian PU-TA-03 untuk KNF-TA-02

Pengujian ketiga, yaitu PU-TA-04, dilakukan melalui *unit testing* pada Kode IV.12. Tes ini memastikan delegasi dari token admin write tetap tidak dapat memperluas cakupan dataset. Ketika *child token* mencoba meminta `write_datasets` di luar cakupan *parent token*, sistem menolak dengan error `DelegationExpandsAccess`.

```
1  #[test]
2  fn admin_write_true_expansion_still_fails() {
3      let params = DelegationAttenuationParams {
4          ...,
5          write_datasets: vec![DatasetCategory::RAWAT_INAP],
6      };
7      let err = attenuate_macaroon(
8          &admin_write_token(), // write token RAWAT_JALAN
9          &params
10     )
11     // assert error DelegationExpandsAccess
12 }
```

Kode IV.12: Pengujian PU-TA-04 untuk KNF-TA-02

Pengujian terakhir dilakukan melalui *client hospital* untuk memastikan sistem menolak atenuasi hak akses dengan batas waktu yang melebihi masa berlaku token induk. Pada skenario ini, halaman delegasi menampilkan notifikasi *error* ketika pengguna mencoba membuat token turunan dengan waktu kedaluwarsa yang lebih panjang daripada token induk.

IV.3.3.3 Pengujian KNF-TA-03

Pengujian KNF-TA-03 memastikan permintaan akses ditolak apabila token, rantai delegasi, atau bukti identitas aktor aktif tidak valid.

Pengujian pertama, yaitu PU-TA-05, dilakukan melalui *unit testing* pada Kode IV.13. Tes ini memastikan token yang sudah melewati batas waktu berlaku tidak dapat digunakan untuk mengakses segmen RME. Pada skenario ini, waktu verifikasi dibuat berada setelah waktu kedaluwarsa token. Sistem menolak permintaan akses dengan error `ExpiredToken`.

```
1  #[test]
2  fn expired_token_rejected() {
3      let mac = ... // token dokter, expired 2026-07-02
4      let future = ... // 2031-01-01T00:00:00+00:00
5      let err = verify_decmed_token(
6          ...
7          &mac,
8          &ctx({
9              ...,
10             now: future,
11         })),
12     )
13     // assert error ExpiredToken
14 }
```

Kode IV.13: Pengujian PU-TA-05 untuk KNF-TA-03

Pengujian kedua, yaitu PU-TA-06, dilakukan melalui *unit testing* pada Kode IV.14.

Tes ini memastikan rantai delegasi yang tidak valid akan ditolak. Pada pengujian ini, token dimodifikasi dengan caveat `delegated_by` yang tidak sesuai dengan rantai delegasi semestinya. Ketika token diverifikasi, sistem menolak permintaan dengan error `InvalidDelegationChain`.

```
1  #[test]
2  fn broken_delegation_chain_rejected() {
3      let mut mac = ... // token dokter
4      add_caveat_to_macaroon(&mut mac, "delegated_by", "0xAPOTEK");
5      add_caveat_to_macaroon(&mut mac, "delegated_to", LAB);
6      let err = verify_ctx(
7          &mac,
8          ...
9      )
10     // assert error InvalidDelegationChain
11 }
```

Kode IV.14: Pengujian PU-TA-06 untuk KNF-TA-03

Pengujian ketiga, yaitu PU-TA-07, dilakukan melalui *unit testing* pada Kode IV.15. Tes ini memastikan bukti identitas aktor aktif harus sesuai dengan subjek aktif pada token hasil delegasi. Pada skenario ini, token diberikan kepada aktor LAB, tetapi permintaan pertama menggunakan tanda tangan `doctor-sig`. Sistem menolak permintaan dengan error `InvalidWalletSignature`.

```
1  #[test]
2  fn wallet_proof_must_match_active_actor() {
3      let mac = ... // token LAB
4      let verifier = ActiveActorVerifier {
5          active_actor: LAB,
6          valid_sig: "lab-sig",
7      };
8      let ctx = TokenVerificationContext {
9          ...,
10         wallet_signature: Some("doctor-sig"),
11     };
12     let err = verify_decmed_token(
13         &mac,
```

```

14         &ctx,
15         Some(&verifier),
16     )
17     // assert error InvalidWalletSignature
18 }

```

Kode IV.15: Pengujian PU-TA-07 untuk KNF-TA-03 dan KNF-TA-06

IV.3.3.4 Pengujian KNF-TA-04

Pengujian KNF-TA-04 memastikan sistem menyediakan keterlacakan akses dan delegasi melalui pencatatan metadata yang dapat ditinjau kembali oleh pasien.

Pengujian dilakukan melalui *client* pasien. Skenario dimulai dari pemberian akses awal oleh pasien kepada petugas rekam medis, kemudian dilanjutkan dengan delegasi dari petugas rekam medis kepada dokter. Setelah proses tersebut dilakukan, pasien membuka halaman riwayat delegasi untuk memastikan informasi delegator, delegatee, status, waktu, dan relasi delegasi dapat ditelusuri. Tangkapan layar halaman ini dapat dilihat pada Gambar P.3.

IV.3.3.5 Pengujian KNF-TA-05

Pengujian KNF-TA-05 memastikan sistem menolak akses untuk hak akses yang sudah dicabut.

Pengujian pertama, yaitu PU-TA-08, dilakukan melalui tes pada Kode IV.16. Tes ini memastikan token yang sudah masuk daftar revokasi tidak dapat digunakan kembali. Pada skenario ini, sistem terlebih dahulu memverifikasi token yang belum dicabut dan hasilnya berhasil. Setelah itu, revocation key token dibuat bernilai cocok dengan token yang sedang diverifikasi. Sistem kemudian menolak akses dengan status UNAUTHORIZED.

```

1  #[test]
2  fn revocation_key_rejects_revoked_token() {
3      let token = verified_read_token(); // token valid
4      let revoked_key = ... // hash token valid
5
6      ensure_token_not_revoked(

```

```

7      ...,
8      &token,
9      |_| Ok(false), // simulasi Redis kosong
10     ) // success
11
12     let err = ensure_token_not_revoked(
13         ...,
14         &token,
15         |key| Ok(key == revoked_key), // simulasi revoked key
16         // terdapat pada Redis
17     ) // error UNAUTHORIZED
18
19     // assert status UNAUTHORIZED
20 }

```

Kode IV.16: Pengujian PU-TA-08 untuk KNF-TA-05

Pengujian kedua dilakukan melalui *client* pasien dan *client hospital*. Pengujian dimulai dengan pasien mencabut hak akses yang sebelumnya diberikan kepada petugas rekam medis. Setelah revokasi dilakukan, akses dicoba kembali dari akun petugas rekam medis dan dokter yang memperoleh delegasi. Sistem dinyatakan memenuhi kebutuhan apabila hak akses yang telah dicabut tidak lagi dapat digunakan dan tidak lagi muncul sebagai akses aktif pada halaman *Home* maupun halaman delegasi.

IV.3.3.6 Pengujian KNF-TA-06

Pengujian KNF-TA-06 memastikan bukti kepemilikan identitas terikat terhadap aktor aktif, token akses, dan konteks permintaan akses.

Pengujian pertama, yaitu PU-TA-09, dilakukan melalui *unit testing* pada Kode IV.17. Tes ini memastikan setiap permintaan akses menggunakan token DecMed harus menyertakan bukti kepemilikan identitas berupa tanda tangan wallet. Jika *wallet_signature* tidak diberikan, sistem menolak permintaan dengan error *WalletSignatureRequired*. Hal ini menunjukkan bahwa token akses saja tidak cukup untuk memperoleh akses tanpa bukti identitas dari aktor aktif.

```

1  #[test]

```

```

2  fn wallet_signature_required_for_every_decmed_token() {
3      let mac = ... // token dokter
4      let err = verify_decmed_token(
5          &mac,
6          ...,
7          &TokenVerificationContext {
8              ...,
9              wallet_signature: None, // tidak ada
10         },
11     )
12     // assert error WalletSignatureRequired
13 }

```

Kode IV.17: Pengujian PU-TA-09 untuk KNF-TA-06

Pengujian kedua dilakukan melalui PU-TA-07 pada Kode IV.15, sama dengan yang digunakan sebelumnya untuk menguji KNF-TA-03. Tes ini memastikan bukti identitas harus sesuai dengan aktor aktif pada rantai delegasi. Token pada skenario ini merupakan token untuk aktor LAB. Ketika permintaan menggunakan tanda tangan doctor-sig, sistem menolak permintaan dengan error `InvalidWalletSignature`. Setelah tanda tangan diganti menjadi lab-sig, verifikasi token berhasil.

Pada implementasinya, fungsi `verify_decmed_token()` membentuk `WalletProofContext` dari token dan konteks akses yang sedang diminta. Konteks tersebut memuat `token_id`, alamat pasien, `related_rme_id`, mode operasi, `segment_id`, kategori dataset, kategori fungsi, dan timestamp. Konteks ini kemudian diverifikasi terhadap `active_subject` pada rantai delegasi, sehingga bukti kepemilikan identitas terikat pada aktor aktif.

IV.4 Evaluasi

Evaluasi dilakukan dengan membandingkan hasil implementasi dan pengujian terhadap kebutuhan sistem. Evaluasi ini digunakan untuk menilai apakah pendekatan Macaroons, segmentasi data RME, dan penegakan akses pada *Server PRE* mampu menjawab keterbatasan kontrol akses pada DecMed awal.

Berdasarkan hasil pengujian kebutuhan fungsional dan nonfungsional, seluruh kebutuhan sistem yang dirumuskan pada Tugas Akhir ini dinyatakan terpenuhi.

IV.4.1 Evaluasi terhadap Kebutuhan Sistem

Berdasarkan kebutuhan sistem pada Subbab III.1.3, implementasi dievaluasi terhadap kebutuhan fungsional dan nonfungsional yang menjadi dasar pengembangan DecMed. Evaluasi menghubungkan kebutuhan sistem pada Subbab III.1.3 dengan implementasi dan pengujian yang telah dilakukan. Setiap kebutuhan diperiksa berdasarkan komponen yang merealisasikannya dan skenario pengujian yang memverifikasi perilakunya. Pemetaan ini menunjukkan keterhubungan antara kebutuhan, implementasi, dan pengujian tanpa memisahkan ketiganya sebagai daftar yang berdiri sendiri.

Ringkasan pemetaan kebutuhan sistem terhadap implementasi dan pengujian ditunjukkan pada Tabel IV.5. Hasil evaluasi menunjukkan bahwa seluruh kebutuhan fungsional dan nonfungsional telah terpenuhi berdasarkan skenario pengujian manual serta pengujian unit atau integrasi yang telah dilakukan.

Tabel IV.5: Evaluasi Kebutuhan Sistem terhadap Implementasi dan Pengujian

Kebutuhan	Implementasi Terkait	Pengujian Terkait	Status
KF-TA-01	I-TA-02, I-TA-03, I-TA-05	PS-TA-01	Terpenuhi.
KF-TA-02	I-TA-03, I-TA-04, I-TA-05	PS-TA-12, PU-TA-08	Terpenuhi.
KF-TA-03	I-TA-04, I-TA-05	PS-TA-11	Terpenuhi.
KF-TA-04	I-TA-02, I-TA-04, I-TA-06	PS-TA-02, PS-TA-03	Terpenuhi.
KF-TA-05	I-TA-01, I-TA-03, I-TA-06	PS-TA-05, PS-TA-08	Terpenuhi.
KF-TA-06	I-TA-02, I-TA-04, I-TA-06	PS-TA-06, PS-TA-09	Terpenuhi.
KF-TA-07	I-TA-01, I-TA-03, I-TA-06	PS-TA-04	Terpenuhi.
KF-TA-08	I-TA-01, I-TA-03, I-TA-06	PS-TA-07	Terpenuhi.

Tabel IV.5: Evaluasi Kebutuhan Sistem terhadap Implementasi dan Pengujian (lanjutan)

Kebutuhan	Implementasi Terkait	Pengujian Terkait	Status
KF-TA-09	I-TA-01, I-TA-03, I-TA-06	PS-TA-10	Terpenuhi.
KNF-TA-01	I-TA-01, I-TA-02, I-TA-03	PU-TA-01 dan pengujian akses segmen pada <i>client hospital</i>	Terpenuhi.
KNF-TA-02	I-TA-02, I-TA-03, I-TA-06	PU-TA-02, PU-TA-03, PU-TA-04 dan pengujian atenuasi masa berlaku pada <i>client hospital</i>	Terpenuhi.
KNF-TA-03	I-TA-02, I-TA-03, I-TA-06	PU-TA-05, PU-TA-06, PU-TA-07	Terpenuhi.
KNF-TA-04	I-TA-04, I-TA-05, I-TA-06	PS-TA-11 dan pengujian riwayat delegasi pada <i>client</i> pasien	Terpenuhi.
KNF-TA-05	I-TA-03, I-TA-05	PU-TA-08, PS-TA-12	Terpenuhi.
KNF-TA-06	I-TA-03, I-TA-06	PU-TA-07, PU-TA-09	Terpenuhi.

Dengan demikian, evaluasi menunjukkan bahwa implementasi Macaroons, segmentasi data RME, atenuasi delegasi, revokasi, audit delegasi, dan verifikasi bukti identitas aktor aktif telah memenuhi kebutuhan yang ditetapkan. Kebutuhan fungsional terpenuhi melalui skenario PS-TA-01 sampai PS-TA-12, sedangkan kebutuhan nonfungsional terpenuhi melalui pengujian KNF-TA-01 sampai KNF-TA-06 yang didukung oleh pengujian unit PU-TA-01 sampai PU-TA-09.

BAB V

PENUTUP

V.1 Kesimpulan

Berdasarkan hasil analisis, perancangan, implementasi, dan evaluasi yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

1. Merancang dan mengimplementasikan kontrol akses granular pada DecMed dengan Macaroons sebagai token otorisasi. Batasan akses direpresentasikan melalui *caveats*, sedangkan penegakan otoritas dilakukan pada *Server PRE* sebelum data diakses.
2. Segmentasi data RME diterapkan dengan memecah data *off-chain* berdasarkan *dataset category* dan *function category*. Dengan struktur ini, data RME tidak lagi diperlakukan sebagai satu *encrypted blob* besar, tetapi sebagai kumpulan segmen klinis. Metadata setiap segmen digunakan untuk menentukan segmen yang dapat diakses oleh aktor sesuai cakupan token.
3. Atenuasi *offline* diterapkan untuk mendukung pendelegasian akses antar tenaga kesehatan tanpa penerbitan token baru oleh pasien pada setiap perpindahan akses. Pemegang token membentuk token turunan secara *offline* dengan menambahkan *caveats* yang mempersempit cakupan token induk. Implementasi ini juga mencakup pencatatan delegasi, pembatasan kedalaman delegasi, validasi agar token turunan tidak memperluas hak akses token induk, serta revokasi berantai.

V.2 Saran

Berdasarkan batasan dan hasil evaluasi Tugas Akhir ini, beberapa saran untuk pengembangan berikutnya adalah sebagai berikut:

1. Cakupan skenario layanan klinis hingga pelayanan antar fasyankes, memungkinkan skenario nyata seperti rujukan antar fasyankes.

2. Cakupan skenario layanan klinis hingga pelayanan gawat darurat. Mengimplementasikan data IGD dan mekanisme akses ketika pasien tidak sadar, memungkinkan skenario nyata gawat darurat.
3. Integrasi dengan standar interoperabilitas data kesehatan dapat diteliti lebih lanjut. Segmentasi RME dapat dikembangkan untuk integrasi dengan SATUSEHAT atau pemetaan ke standar FHIR.
4. Integrasi dan Pengujian dengan Sistem Rumah Sakit untuk mendukung penggunaan sistem dalam konteks nyata. Disesuaikan dengan tata laksana atau dokumen Pedoman Nasional Pelayanan Kesehatan.

DAFTAR REFERENSI

- Amaracitra, D. S., & Alamsyah, A. (2025). Decentralized Medical Record Model Using Composable NFT: Achieving Data Security, Interoperability, and Privacy through Granular Data Access. *2025 15th International Conference on Electrical Engineering (ICEENG)*. <https://doi.org/10.1109/ICEENG64546.2025.11031352>
- Anagnostopoulou, A., Kehrioti, E., Mavridis, I., & Gritzalis, D. (2025). Bolstering IIoT Resilience: The Synergy of Blockchain and CapBAC. *Proceedings of the 22nd International Conference on Security and Cryptography (SECRYPT 2025)*, 120–131. <https://doi.org/10.5220/0013513800003979>
- Arciszewski, S., & Haussmann, S. (2022, Mei). PASETO: Platform-Agnostic Security Tokens [Work in Progress, expired Internet-Draft]. <https://datatracker.ietf.org/doc/html/draft-paragon-paseto-rfc-01>
- Benet, J. (2014, Juli). *IPFS — Content Addressed, Versioned, P2P File System*. arXiv: 1407.3561 [cs.NI]. <https://doi.org/10.48550/arXiv.1407.3561>
- Birgisson, A., Politz, J. G., Erlingsson, Ú., Taly, A., Vrabie, M., & Lentzner, M. (2014). Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. *Proceedings of the 2014 Network and Distributed System Security Symposium (NDSS)*. <https://doi.org/10.14722/ndss.2014.23251>
- Blaze, M., Bleumer, G., & Strauss, M. (1998). Divertible Protocols and Atomic Proxy Cryptography. *Advances in Cryptology — EUROCRYPT '98*, 1403, 127–144. <https://doi.org/10.1007/BFb0054122>
- dCache. (nodate). *Chapter 1. Understanding Macaroons: dCache UserGuide 8.2*. dCache. Diakses pada Juni 16, 2026, dari <https://dcache.org/manuals/UserGuide-8.2/macaroons.shtml>
- Eclipse Biscuit Project. (nodate). *Biscuit Tokens*. Diakses pada Juni 4, 2026, dari <https://www.biscuitsec.org/>
- for Standardization, I. O. (2014, Oktober). *Health informatics — Privilege management and access control — Part 2: Formal models* (techreport Nomor ISO 22600-2:2014) (Accessed: 2026-07-02). ISO. Geneva,

- Switzerland. Diakses pada Juli 2, 2026, dari <https://www.iso.org/standard/62654.html>
- Hu, V. C., Ferraiolo, D. F., Kuhn, R. D., Schnitzer, A., Sandlin, K., Miller, R., & Scarfone, K. (2014). *Guide to Attribute Based Access Control (ABAC) Definition and Considerations* (NIST Special Publication Nomor 800-162) (Accessed: 2026-06-03). National Institute of Standards dan Technology. Gaithersburg, MD. <https://doi.org/10.6028/NIST.SP.800-162>
- Peraturan Menteri Kesehatan Republik Indonesia Nomor 73 Tahun 2016 tentang Standar Pelayanan Kefarmasian di Apotek, Jakarta (2016, Desember 23). Diakses pada Juni 16, 2026, dari <https://farmalkes.kemkes.go.id/unduh/permenkes-73-2016/>
- Joint Task Force Transformation Initiative. (2020). *Security and Privacy Controls for Information Systems and Organizations* (NIST Special Publication Nomor 800-53, Revision 5). National Institute of Standards dan Technology. Gaithersburg, MD. <https://doi.org/10.6028/NIST.SP.800-53r5>
- Jones, L. A., Nelder, J. R., Fryer, J. M., Alsop, P. H., Geary, M. R., Prince, M., & Cardinal, R. N. (2022). Public opinion on sharing data from health services for clinical and research purposes without explicit consent: an anonymous online survey in the UK. *BMJ Open*, 12, Artikel jurnal e057579. <https://doi.org/10.1136/bmjopen-2021-057579>
- Jones, M. B., Bradley, J., & Sakimura, N. (2015, Mei). *JSON Web Token (JWT)* (RFC Nomor 7519). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7519>
- Kementerian Kesehatan Republik Indonesia. (2022a, April). Keputusan Menteri Kesehatan Republik Indonesia Nomor HK.01.07/MENKES/1128/2022 tentang Standar Akreditasi Rumah Sakit [Ditetapkan di Jakarta]. Diakses pada Juli 1, 2026, dari https://keslan.kemkes.go.id/unduh/fileunduhan_1654499045_682777.pdf
- Kementerian Kesehatan Republik Indonesia. (2022b, Agustus). Peraturan Menteri Kesehatan Republik Indonesia Nomor 24 Tahun 2022 tentang Rekam Medis.
- Leffew, K. (2019, Desember 4). *Secure Access Control in the Decentralized Cloud*. Storj. Diakses pada Juni 16, 2026, dari <https://www.storj.io/blog/secure-access-control-in-the-decentralized-cloud>

- Li, A. S., Safavi-Naini, R., & Fong, P. W. L. (2022). A Capability-based Distributed Authorization System to Enforce Context-aware Permission Sequences. *Proceedings of the 27th ACM Symposium on Access Control Models and Technologies*, 195–206. <https://doi.org/10.1145/3532105.3535014>
- Liu, R., & Chen, P. (2025). Research on Access Control Model of Electronic Medical Records Based on Blockchain Technology. *2025 5th Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS)*, 662–665. <https://doi.org/10.1109/ACCTCS66275.2025.00066>
- Keputusan Menteri Kesehatan Republik Indonesia Nomor HK.01.07/Menkes/1423/2022 tentang Pedoman Variabel dan Metadata Rekam Medis, Jakarta (2022).
- Microsoft. (2006). *Uncover Security Design Flaws Using the STRIDE Approach*. Diakses pada April 1, 2026, dari <https://learn.microsoft.com/en-us/archive/msdn-magazine/2006/november/uncover-security-design-flaws-using-the-stride-approach>
- NHS England. (2025, Maret 14). *Consent to using and sharing patient information* (1.1). Diakses pada April 29, 2026, dari <https://www.england.nhs.uk/long-read/consent-to-using-and-sharing-patient-information/>
- Núñez, D. (2017). Umbral: A Threshold Proxy Re-Encryption Scheme [Available at <https://github.com/nucypher/umbral-doc/blob/master/umbral-doc.pdf>].
- Odugu, N. K., & Rajesh, A. (2021). A Fine-Grained Access Control Survey For The Secure Big Data Access [Accessed: 2026-06-03]. *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, 12(10), 4180–4186. <http://www.turcomat.org/index.php/turkbilmat/article/view/5141>
- of Health, U. D., & Services, H. (2009). *Minimum Necessary: guidance for covered entities* (techreport) (Accessed: 2026-07-02). Washington, D.C. Diakses pada Juli 2, 2026, dari <https://www.hhs.gov/sites/default/files/o%09cr/privacy/hipaa/understanding/coveredentities/minimumnecessary.pdf>
- Patil, S. P. (2024, Desember). *Blockchain Technology for Secure Healthcare Data Management with Patient Empowerment* [MSc Research Project]. National College of Ireland [MSc Cyber Security. Supervisor: Prof Liam McCabe. Word count: 6744]. <https://norma.ncirl.ie/8241/1/sidhantprakashpatil.pdf>
- Peng, G., Zhang, A., & Lin, X. (2023). Patient-Centric Fine-Grained Access Control for Electronic Medical Record Sharing With Security via Dual-Blockchain.

- IEEE Transactions on Network Science and Engineering*, 10(6), 3908–3921.
<https://doi.org/10.1109/TNSE.2023.3276166>
- Ramaswamy, A., dkk. (2023). Optimizing outcomes through a multidisciplinary approach to clinical care. *Current Opinion in Supportive and Palliative Care*, 17(4), 245–251. <https://doi.org/10.1097/SPC.0000000000000742>
- Republik Indonesia. (2004, Oktober). Undang-Undang Nomor 29 Tahun 2004 tentang Praktik Kedokteran [LN. 2004/ No.116, TLN NO. 4431]. Diakses pada November 11, 2025, dari <https://peraturan.bpk.go.id/Details/40752/uu-no-29-tahun-2004>
- Sadewa, B. (2025). *Pengembangan Kontrol Akses dalam Kerangka Kerja Sistem Manajemen Rekam Medis Elektronik Berbasis IOTA* [Tugas Akhir]. Institut Teknologi Bandung. Diakses pada Maret 11, 2026, dari https://digilib.itb.ac.id/gdl/view_data/pengembangan-kontrol-akses-dalam-kerangka-kerja-sistem-manajemen-rekam-medis-elektronik-berbasis-iota/Bakta-Sadewa
- Sandhu, R., & Samarati, P. (1994). Access control: principle and practice. *IEEE Communications Magazine*, 32(9), 40–48. <https://doi.org/10.1109/35.312842>
- SATUSEHAT Platform. (nodate). *Rawat Inap*. Diakses pada Juni 30, 2026, dari <https://satusihat.kemkes.go.id/platform/docs/id/interoperability/rawat-inap-new/>
- SATUSEHAT Platform. (2023, Agustus 22). *Resume Medis - Rawat Jalan*. Diakses pada Juni 30, 2026, dari <https://satusihat.kemkes.go.id/platform/docs/id/interoperability/rme-rawat-jalan/>
- Savadatti, S. G., Krishnamoorthy, S., & Delhibabu, R. (2025). Survey of Distributed Ledger Technology (DLT) for Secure and Scalable Computing. *IEEE Access*, 13, 8393–8415. <https://doi.org/10.1109/ACCESS.2025.3528211>
- Sedi Nzakuna, P., Paciello, V., Lay-Ekuakille, A., Kuti Lusala, A., Dello Iacono, S., & Pietrosanto, A. (2025). From IOTA Tangle 2.0 to Rebased: A Comparative Analysis of Decentralization, Scalability, and Suitability for IoT Applications. *Sensors*, 25(11), 3408. <https://doi.org/10.3390/s25113408>
- Shostack, A. (2014). *Threat Modeling: Designing for Security*. Wiley.
- Singh, Y., Jaiswal, S., & Kumar, V. (2025). Granular Level Privacy Aware Access Control in Block Chain Based EHR Systems. *2025 12th International Conference on Reliability, Infocom Technologies and Optimization (Trends*

and Future Directions) (ICRITO). <https://doi.org/10.1109/ICRITO66076.2025.11241459>

Undang-Undang Republik Indonesia Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi, Jakarta (2022, Oktober 17). Diakses pada Desember 20, 2025, dari <https://peraturan.bpk.go.id/Details/229798/uu-no-27-tahun-2022>

World Health Organization. (2010, September 1). *Framework for Action on Interprofessional Education and Collaborative Practice*. World Health Organization. Geneva. Diakses pada Juni 16, 2026, dari <https://www.who.int/publications/i/item/framework-for-action-on-interprofessional-education-collaborative-practice>

Lampiran A. Metadata Rekam Medis Elektronik

Tabel berikut menjabarkan metadata variabel RME yang diadopsi dari KMK No. HK.01.07/MENKES/1423/2022 untuk dataset Rawat Jalan, Rawat Inap, Laboratorium, dan Apotek (Menteri Kesehatan Republik Indonesia, 2022). Klasifikasi dokumentasi pada tabel berikut diperoleh melalui interpretasi terhadap definisi operasional metadata yang tercantum pada dokumen KMK tersebut. Dokumentasi administratif tidak dibagi ke dalam segmentasi yang lebih rinci karena tidak menjadi fokus utama pada rancangan segmentasi data dalam Tugas Akhir ini.

Tabel A.1: Metadata Variabel RME Rawat Jalan

Kelompok Data	Subkelompok Metadata	Klasifikasi Dokumentasi
Lembar Identitas	Identitas pasien dan data dasar rekam medis	Administratif
Cara Pembayaran	Metode atau skema pembayaran pelayanan	Administratif
General Consent / Persetujuan Umum	Persetujuan umum pasien terhadap pelayanan	Administratif
Formulir Umum / Asesmen Awal Rawat Jalan	Anamnesis	Klinis
	Pemeriksaan Fisik	Klinis
	Pemeriksaan Psikologis, Sosial Ekonomi, Spiritual	Klinis
Pemeriksaan Spesialistik	Riwayat Penggunaan Obat	Klinis
	Rencana Rawat	Klinis
	Instruksi Medik dan Keperawatan	Klinis
	Pemeriksaan Penunjang	Klinis
	Diagnosis	Klinis
	Persetujuan Tindakan / Penolakan Tindakan (<i>Informed Consent</i>)	Klinis

Kelompok Data	Subkelompok Metadata	Klasifikasi Dokumentasi
	Terapi: Tindakan dan Obat / Peresepan	Klinis

Tabel A.2: Metadata Variabel RME Rawat Inap

Kelompok Data	Subkelompok Metadata	Klasifikasi Dokumentasi
Lembar Identitas	Identitas pasien dan data dasar rekam medis	Administratif
Cara Pembayaran	Metode atau skema pembayaran pelayanan	Administratif
General Consent / Persetujuan Umum	Persetujuan umum pasien terhadap pelayanan	Administratif
Formulir Rawat Inap	Anamnesis	Klinis
	Pemeriksaan Fisik	Klinis
	Pemeriksaan Psikologis, Sosial Ekonomi, Spiritual	Klinis
Pemeriksaan Spesialistik	Riwayat Penggunaan Obat	Klinis
	Perencanaan Pemulangan Pasien	Klinis
	Rencana Rawat	Klinis
	Instruksi Medik dan Keperawatan	Klinis
	Pemeriksaan Penunjang	Klinis
	Diagnosis	Klinis
	Persetujuan Tindakan / Penolakan Tindakan (<i>Informed Consent</i>)	Klinis
	Terapi	Klinis

Tabel A.3: Metadata Variabel RME Laboratorium

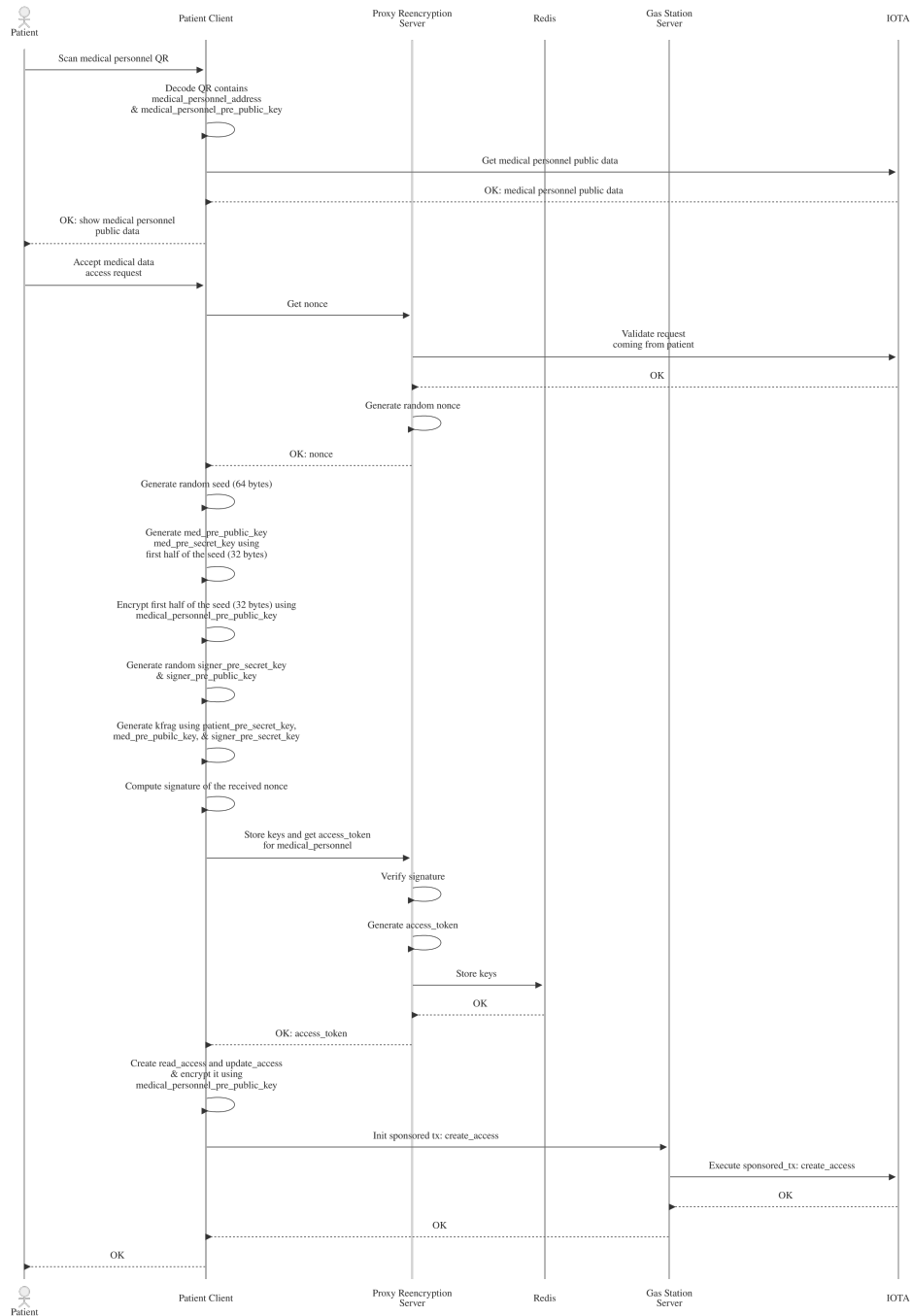
Kelompok Data	Subkelompok Metadata	Klasifikasi Dokumentasi
Laboratorium	Identitas Laboratorium dan Pasien	Administratif
	Permintaan Pemeriksaan	Klinis
	Spesimen Klinis	Klinis
	Fiksasi dan Pengolahan Spesimen	Klinis
	Hasil Pemeriksaan	Klinis
	Interpretasi dan Validasi Hasil	Klinis

Tabel A.4: Metadata Variabel RME Apotek

Kelompok Data	Subkelompok Metadata	Klasifikasi Dokumentasi
Peresepan	Identitas Pasien	Administratif
	Data Resep dan Obat	Klinis
	Riwayat Penggunaan Obat	Klinis
	Riwayat Alergi	Klinis
	Asal Resep	Klinis
	Dokter Penulis Resep	Klinis
	Status dan Pengkajian Resep	Klinis
Dispensing	Status Resep	Klinis
	Waktu Penyiapan Obat	Klinis
	Waktu Penyerahan Obat	Klinis
	Nama Petugas yang Melakukan Dispensing	Klinis
	Etiket	Klinis

Lampiran B. Alur Pemberian Akses Medis pada DecMed

Alur pemberian akses medis dari pasien kepada tenaga medis pada implementasi DecMed saat ini dapat dilihat pada Gambar B.1.



Gambar B.1: *Sequence Diagram* Alur Pemberian Akses Medis dari Pasien kepada Tenaga Medis (Sadewa, 2025)

Lampiran C. Kebutuhan Fungsional DecMed

Tabel C.1: Kebutuhan Fungsional DecMed (Sadewa, 2025)

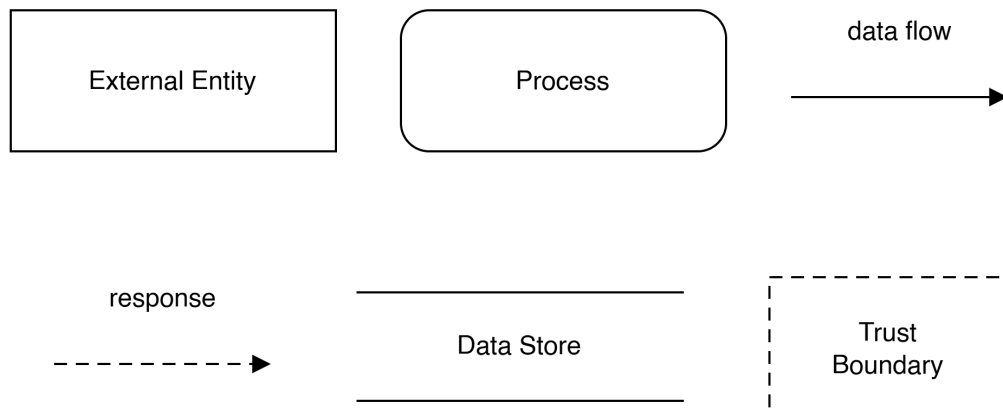
ID	Deskripsi
F01	Pasien dapat melakukan registrasi akun pada sistem
F02	Pasien dapat masuk ke akun menggunakan credentials yang dimiliki
F03	Pasien dapat keluar dari akun
F04	Pasien dapat melakukan scan kode QR yang dimiliki oleh masing-masing tenaga kesehatan dan petugas rekam medis
F05	Pasien dapat membaca seluruh RME yang ia miliki
F06	Pasien dapat memberikan akses baca bagian administratif dari data RME miliknya kepada petugas rekam medis untuk rentang waktu tertentu
F07	Pasien dapat memberikan akses baca bagian administratif dan klinis dari data RME miliknya kepada tenaga kesehatan untuk rentang waktu tertentu
F08	Pasien dapat melihat daftar log dari akses terhadap data RME yang dimiliki
F09	Pasien dapat mencabut akses yang telah diberikan. Akses tersebut berupa akses baca maupun tulis terhadap data RME.
F10	Tenaga kesehatan dapat melakukan aktivasi client
F11	Tenaga kesehatan dapat melakukan registrasi akun pada sistem
F12	Tenaga kesehatan dapat masuk ke akun menggunakan credentials yang dimiliki
F13	Tenaga kesehatan dapat keluar dari akun
F14	Tenaga kesehatan dapat membaca bagian administratif dan klinis dari data RME milik pasien setelah diberikan akses dalam waktu tertentu
F15	Tenaga kesehatan dapat menambahkan entri RME baru
F16	Tenaga kesehatan dapat mengedit bagian klinis dari entri RME baru yang dibuatnya pada rentang waktu yang telah ditentukan

Tabel C.1 – Lanjutan

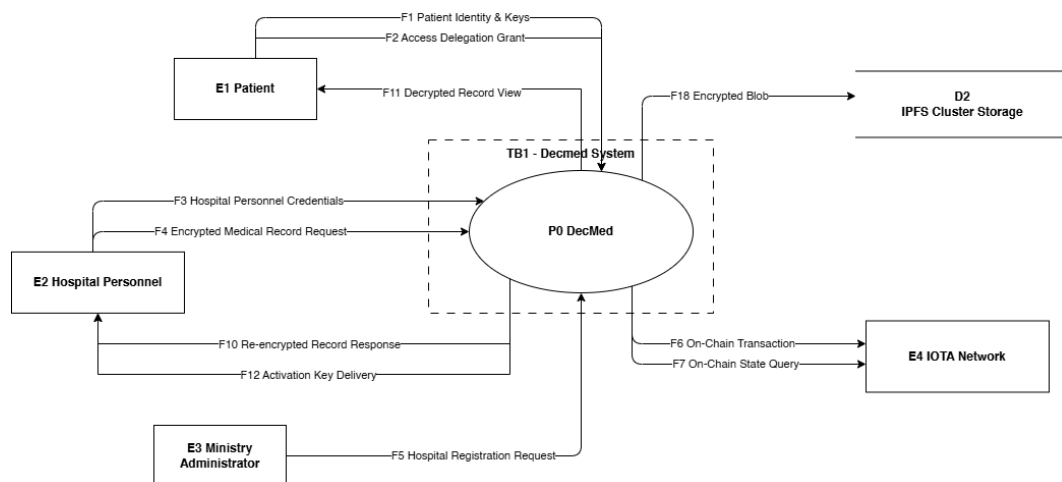
ID	Deskripsi
F17	Petugas rekam medis dapat melakukan aktivasi client
F18	Petugas rekam medis dapat melakukan registrasi akun pada sistem
F19	Petugas rekam medis dapat masuk ke akun menggunakan credentials yang dimiliki
F20	Petugas rekam medis dapat keluar dari akun
F21	Petugas rekam medis dapat membaca bagian administratif dari data RME milik pasien setelah diberikan akses dalam waktu tertentu
F22	Admin fasyankes dapat melakukan aktivasi client
F23	Admin fasyankes dapat melakukan registrasi akun pada sistem
F24	Admin fasyankes dapat masuk ke akun menggunakan credentials yang dimiliki
F25	Admin fasyankes dapat keluar dari akun
F26	Admin fasyankes dapat menambahkan personel fasyankes baru
F27	Admin fasyankes dapat membuat kunci aktivasi baru untuk personel fasyankes
F28	Kementerian kesehatan dapat mendaftarkan fasyankes dan admin untuk fasyankes tersebut
F29	Kementerian kesehatan dapat membuat kunci aktivasi baru untuk admin fasyankes

Lampiran D. DFD DecMed

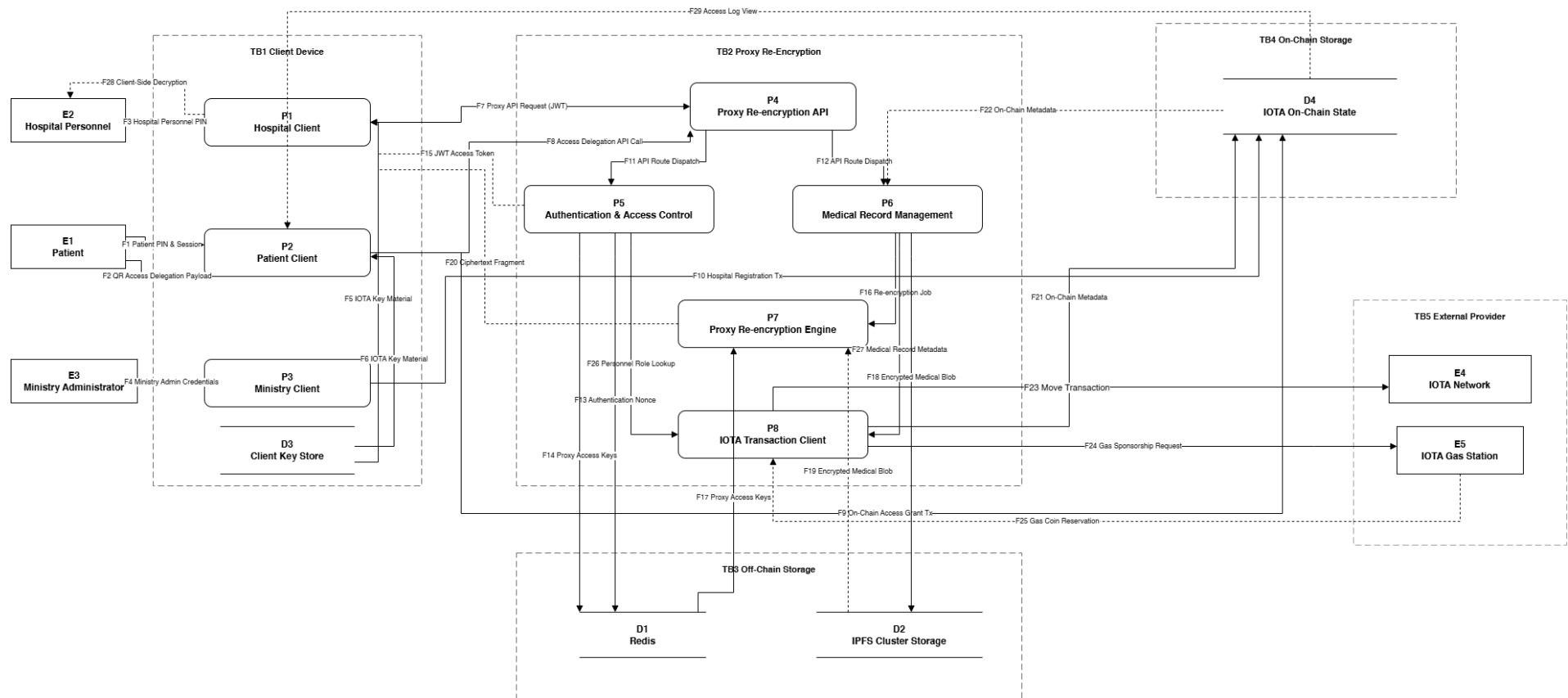
Hasil pemodelan DFD DecMed Level 0 dan Level 1 dapat dilihat pada Gambar D.2 dan D.3, sedangkan legenda pemodelan kedua DFD tersebut dapat dilihat pada Gambar D.1.



Gambar D.1: Legenda *Data Flow Diagram* (Dokumentasi Penulis)



Gambar D.2: DFD DecMed Level 0 (Dokumentasi Penulis)



Gambar D.3: DFD DecMed Level 1 (Dokumentasi Penulis)

Tabel D.1: Daftar *External Entities* pada DFD DecMed

ID	Nama	Interaksi utama
E1	Patient	PIN, QR delegasi akses, <i>revoke access</i> , lihat <i>access log</i>
E2	Hospital Personnel	PIN, baca/ <i>update</i> rekam medis, data administratif
E3	Ministry Administrator	Registrasi rumah sakit, <i>activation key</i>
E4	IOTA Network	RPC, <i>publish/execute transaction</i> , <i>read state</i>
E5	IOTA Gas Station	<i>reserve_gas</i> , <i>execute_tx</i>

Tabel D.2: Daftar *Process* DFD DecMed

ID	Nama
P0	Decmed (seluruh sistem)
P1	Hospital Client
P2	Patient Client
P3	Ministry Client
P4	Proxy Re-encryption API
P5	Authentication & Access Control
P6	Medical Record Management
P7	Proxy Re-encryption Engine
P8	IOTA Transaction Client

Tabel D.3: Daftar *Data Store* pada DFD DecMed

ID	Nama	Data sensitif
D1	Redis Cache	<i>Nonce challenge</i> , <i>AccessKeys</i> (k-frag, PRE public keys, seed terenkripsi)
D2	IPFS Cluster Storage	Blob rekam medis terenkripsi (CID direferensikan <i>on-chain</i>)
D3	Client OS Keyring	IOTA keypair, PRE keys, PIN-derived secrets, <i>activation key</i>
D4	IOTA On-Chain State	Metadata, <i>access grants</i> , <i>access log</i> , <i>hospital registry</i>

Tabel D.4: Daftar Data Flow pada DFD DecMed (Level 0)

ID	Label	Arah
F1	Patient Identity & Keys	E1 → P0
F2	Access Delegation Grant	E1 → P0
F3	Hospital Personnel Credentials	E2 → P0
F4	Encrypted Medical Record Request	E2 → P0
F5	Hospital Registration Request	E3 → P0
F6	On-Chain Transaction	P0 → E4
F7	On-Chain State Query	P0 → E4
F10	Re-encrypted Record Response	P0 → E2
F11	Decrypted Record View	P0 → E1
F12	Activation Key Delivery	P0 → E2
F18	Encrypted Blob	P0 → D2

Tabel D.5: Daftar Data Flow pada DFD DecMed (Level 1)

ID	Label	Arah Ringkas
F1	Patient PIN & Session	E1 → P2
F2	QR Access Delegation Payload	E1 → P2
F3	Hospital Personnel PIN	E2 → P1
F4	Ministry Admin Credentials	E3 → P3
F5	IOTA Key Material	D3 → P1
F6	IOTA Key Material	D3 → P2
F7	Proxy API Request (JWT)	P1 → P4
F8	Access Delegation API Call	P2 → P4
F9	On-Chain Access Grant Tx	P2 → D4
F10	Hospital Registration Tx	P3 → D4
F11	API Route Dispatch	P4 → P5
F12	API Route Dispatch	P4 → P6
F13	Authentication Nonce	P5 → D1
F14	Proxy Access Keys	P5 → D1
F15	JWT Access Token	P5 → P1
F16	Re-encryption Job	P6 → P7
F17	Proxy Access Keys	D1 → P7
F18	Encrypted Medical Blob	P6 → D2
F19	Encrypted Medical Blob	D2 → P7
F20	Ciphertext Fragment	P7 → P1
F21	On-Chain Metadata	P8 → D4
F22	On-Chain Metadata	D4 → P6
F23	Move Transaction	P8 → E4
F24	Gas Sponsorship Request	P8 → E5
F25	Gas Coin Reservation	E5 → P8
F26	Personnel Role Lookup	P5 → P8
F27	Medical Record Metadata	P6 → P8
F28	Client-Side Decryption	P1 → E2
F29	Access Log View	D4 → P2

Lampiran E. Analisis STRIDE pada Sistem DecMed

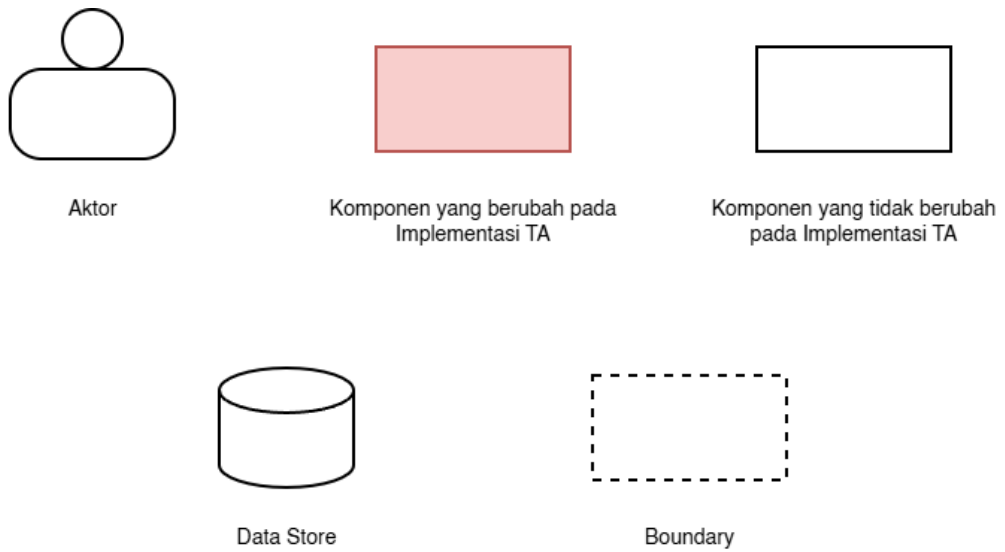
Tabel E.1: Analisis STRIDE pada Sistem DecMed

ID	Komponen / Flow	Kategori	Skenario Ancaman	Dampak
T01	P5, F7	<i>Spoofing</i>	Penyerang menggunakan token akses yang dicuri atau dipalsukan untuk data RME.	Akses tidak sah terhadap data rekam medis pasien.
T02	P8, D4, E4, dan F9	<i>Tampering</i>	Pengguna mengirim transaksi pemberian akses yang valid secara kriptografis, tetapi cakupan akses tidak dibatasi secara tepat oleh logika otorisasi <i>smart contract</i> .	Metadata akses atau metadata RME dapat menunjuk pada cakupan data yang tidak semestinya, sehingga integritas terganggu.
T03	P6, D2, dan F19	<i>Tampering</i>	Referensi CID yang mengarah ke objek <i>off-chain</i> dimanipulasi sehingga sistem mengambil <i>blob</i> terenkripsi yang tidak sesuai.	Data RME yang diambil tidak valid, tidak konsisten, atau gagal diverifikasi.
T04	P2, P5, P8, D4, F8, F9, F21, dan F29	<i>Repudiation</i>	Pihak tertentu menyangkal pernah memberikan atau menggunakan akses.	Kesulitan dalam audit dan investigasi penyalahgunaan akses.
T05	P5, P8, D4, F16, F19, F20, F21, dan F29	<i>Information Disclosure</i>	Delegasi akses membuka cakupan data pasien yang lebih luas dari kebutuhan.	Kebocoran data RME pasien.

Tabel E.1: Analisis STRIDE pada Sistem DecMed (lanjutan)

ID	Komponen / Flow	Kategori	Skenario Ancaman	Dampak
T06	P8, D4, dan F22	<i>Information Disclosure</i>	Metadata transaksi mengungkapkan relasi pasien, fasyankes, atau data pasien.	Privasi pasien dan informasi layanan klinis jadi terekspos.
T07	P4, P5, P6, P7, P8, D1, E4, dan F7	<i>Denial of Service</i>	Layanan PRE atau penyimpanan IPFS tidak dapat diakses akibat serangan atau gangguan layanan.	Proses akses rekam medis menjadi lambat atau tidak tersedia.
T08	P2, P5, P8, dan D4, F8, F9,	<i>Elevation of Privilege</i>	Pengguna memperoleh hak akses lebih tinggi melalui delegasi yang tidak dibatasi.	Terjadi eskalasi hak akses di luar kebutuhan klinis yang sah.

Lampiran F. Legenda Diagram Arsitektur Solusi



Gambar F.1: Legenda Diagram Arsitektur Solusi (Dokumentasi Penulis)

Lampiran G. Skema Penyimpanan Data RME

Lampiran ini memuat rincian skema penyimpanan data RME yang digunakan pada rancangan segmentasi data. Skema penyimpanan dibagi menjadi metadata segmen yang disimpan pada penyimpanan *on-chain* dan isi segmen terenkripsi yang disimpan pada penyimpanan *off-chain*.

Tabel G.1: Skema Metadata Segmen RME *On-Chain*

Komponen	Key JSON	Keterangan
Identitas Segmen	<code>segment_id</code>	UUID unik untuk satu segmen data RME.
	<code>related_rme_id</code>	ID RME atau episode pelayanan yang menjadi induk segmen, dengan format RME- <code>{6_digit}</code> .
	<code>patient_address</code>	Alamat <i>wallet</i> pasien pemilik RME.
	<code>hospital_id</code>	ID fasilitas pelayanan kesehatan ketika menerima pelayanan.
Kategori Data	<code>dataset_category</code>	Kategori dataset. Sesuai dengan tabel III.8.
	<code>function_category</code>	Kategori fungsi klinis atau administratif. Sesuai dengan tabel III.9.
Pointer dan Integritas	<code>ipfs_cid</code>	CID IPFS dari segmen data terenkripsi.
	<code>integrity_hash</code>	Hash SHA-256 dari segmen terenkripsi.
Kapsul Kunci	<code>capsule</code>	Kapsul PRE yang diperlukan untuk proses <i>re-encryption</i> .

Komponen	Key JSON	Keterangan
	enc_key_and_nonce	Kunci simetris dan <i>nonce</i> yang sudah terenkripsi untuk membuka segmen data.
Audit Metadata	created_at	Waktu pembuatan metadata segmen.
	author_address	Alamat <i>wallet</i> aktor yang membuat atau memperbarui segmen.

Tabel G.2: Skema Segmen Data RME *Off-Chain*

Komponen	Key JSON	Keterangan
Header Segmen	segment_id	UUID yang sama dengan segment_id pada metadata <i>on-chain</i> .
	related_rme_id	ID RME atau episode pelayanan yang menjadi induk segmen, dengan format RME-{6_digit}.
	dataset_category	Kategori dataset sesuai Tabel III.8.
	function_category	Kategori fungsi sesuai dengan Tabel III.9.
Konteks Medis	patient_address	Alamat IOTA pasien pemilik RME.
	service_date	Tanggal dan waktu ketika segmen tercatat dengan format YYYY-MM-DD HH:MM:SS beserta informasi <i>timezone</i> .
	author_address	Alamat IOTA aktor yang menulis isi segmen.

Komponen	Key JSON	Keterangan
Isi Segmen	payload	Objek segmen data RME dengan tipe data Value. Pada implementasi ini, isi segmen dalam bentuk text.
	payload_hash	<i>Hash</i> isi payload.

Lampiran H. Contoh *Caveat* Token Macaroon untuk Setiap Tahapan

Lampiran ini menyajikan contoh *caveat* token Macaroon pada setiap tahapan alur pemberian akses, mulai dari penerbitan token awal oleh pasien hingga rangkaian delegasi yang dilakukan oleh tenaga medis. Nilai alamat *wallet* (0xPASIEN, 0xPETUGAS_RM, 0xDOKTER, 0xPERAWAT, 0xLAB, 0xAPOTEKER), hospital-001, dan RME-001 hanya bersifat ilustratif. Pada implementasi, nilai-nilai tersebut akan berbeda tergantung pada alamat *wallet* pemegang token serta ID RME yang di-generate. Simbol titik tiga (...) adalah bentuk ilustrasi yang digunakan untuk menunjukkan bahwa token dapat memiliki lebih banyak nilai lainnya selain yang dituliskan.

Penerbitan Awal: Pasien → Petugas Rekam Medis

```
1  purpose = Read
2  patient_address = 0xPASIEN
3  root_subject = 0xPETUGAS_RM
4  hospital_cid = hospital-001
5
6  read_dataset_in = [RAWAT_JALAN, RAWAT_INAP, LABORATORIUM, APOTEK]
7
8  read_function_in = [
9      ADMINISTRATIVE_GENERAL,
10     ANAMNESIS,
11     PEMERIKSAAN_FISIK,
12     PEMERIKSAAN_PSIKOLOGIS,
13     ...
14 ] // seluruh 15 fungsi
15
16 expires_before = 2026-05-16T18:00:00
17 max_delegation_depth = 2
```

Kode H.1: Contoh *Caveat* Token Read Awal untuk Petugas Rekam Medis

```
1  purpose = Update
2  patient_address = 0xPASIEN
```

```

3  root_subject = 0xPETUGAS_RM
4  related_rme_id = RME-001
5  hospital_cid = hospital-001
6
7  write_dataset_in = [RAWAT_JALAN, LABORATORIUM, APOTEK]
8
9  write_function_in = [
10     ... // union fungsi dari ketiga dataset di atas
11 ]
12
13 expires_before = 2026-05-16T18:00:00
14 max_delegation_depth = 2

```

Kode H.2: Contoh *Caveat* Token Update Awal untuk Petugas Rekam Medis

Delegasi: Petugas Rekam Medis → Dokter

```

1  parent_token_hash = <sha256(parent)>
2  delegated_by = 0xPETUGAS_RM
3  delegated_to = 0xDOKTER
4
5  read_dataset_in = [RAWAT_JALAN, RAWAT_INAP, LABORATORIUM, APOTEK]
6  write_dataset_in = [RAWAT_JALAN, LABORATORIUM, APOTEK]
7
8  read_function_in = [
9     ... // semua 15 fungsi
10 ]
11 write_function_in = [
12     ... // 14 fungsi, ADMINISTRATIVE_GENERAL dihapus
13 ]
14
15 related_rme_id = RME-001
16 expires_before = 2026-05-16T18:00:00
17 max_delegation_depth = 1

```

Kode H.3: Contoh *Caveat* Delegasi Petugas Rekam Medis → Dokter

Delegasi: Petugas Rekam Medis → Perawat

```
1 parent_token_hash = <sha256(parent)>
2 delegated_by = 0xPETUGAS_RM
3 delegated_to = 0xPERAWAT
4
5 read_dataset_in = [RAWAT_JALAN]
6 write_dataset_in = [RAWAT_JALAN]
7
8 read_function_in = [
9     ADMINISTRATIVE_GENERAL,
10    ANAMNESIS,
11    PEMERIKSAAN_FISIK]
12 write_function_in = [ANAMNESIS, PEMERIKSAAN_FISIK]
13
14 expires_before = 2026-05-16T18:00:00
15 max_delegation_depth = 1
```

Kode H.4: Contoh *Caveat* Delegasi Petugas Rekam Medis → Perawat

Delegasi: Dokter → Petugas Laboratorium

```
1 parent_token_hash = <sha256(token_dokter)>
2 delegated_by = 0xDOKTER
3 delegated_to = 0xLAB
4
5 read_dataset_in = [LABORATORIUM]
6 write_dataset_in = [LABORATORIUM]
7
8 read_function_in = [
9     ADMINISTRATIVE_GENERAL,
10    PEMERIKSAAN_PENUNJANG,
11    LABORATORIUM]
12 write_function_in = [LABORATORIUM]
13
14 expires_before = 2026-05-16T18:00:00
15 max_delegation_depth = 0
```

Delegasi: Dokter → Apoteker

```
1 parent_token_hash = <sha256(token_dokter)>
2 delegated_by = 0xDOKTER
3 delegated_to = 0xAPOTEKER
4
5 read_dataset_in = [RAWAT_JALAN, APOTEK]
6 write_dataset_in = [APOTEK]
7
8 read_function_in = [
9     ADMINISTRATIVE_GENERAL,
10    RIWAYAT_PENGGUNAAN_OBAT,
11    TERAPI,
12    PERESEPAN,
13    DISPENSING
14 ]
15 write_function_in = [PERESEPAN, DISPENSING]
16
17 expires_before = 2026-05-16T18:00:00
18 max_delegation_depth = 0
```

Lampiran I. Daftar Dependensi DecMed

Daftar ini hanya mencakup dependensi langsung yang dideklarasikan pada masing-masing komponen, bukan seluruh dependensi transitif. Dependensi internal yang ditandai dengan path merupakan *crate* lokal yang dikembangkan sebagai bagian dari sistem.

Tabel I.1: Dependensi Umum yang Digunakan pada Beberapa Komponen

<i>Crate</i>	Versi/Sumber
serde	1
serde_json	1
sha2	0.10
uuid	1
bip39	2
iota-sdk	Git rev. 3d7c588
iota-keys	Git rev. 3d7c588
iota-config	Git rev. 3d7c588
iota-types	Git rev. 3d7c588
iota-json-rpc-types	Git rev. 3d7c588
shared_crypto	Git rev. 3d7c588
move-core-types	Git rev. 3d7c588
dotenv	0.15
bcs	0.1.4
schemars	0.8
rand	0.9
bincode	2
umbral-pre	0.11
aes-gcm	0.10
argon2	0.5.3
hex	0.4.3
regex	1.11.1
generic-array	1.2.0

<i>Crate</i>	Versi/Sumber
k256	0.13.4
elliptic-curve	0.13.8
chrono	0.4
base64	0.22
strum/strum_macros	0.27
anyhow	1.0
thiserror	2.0

Tabel I.2: Dependensi PRE

<i>Crate</i>	Versi/Sumber
axum	0.8.4
axum-extra	0.10.1
tower	0.5.2
tokio	1.44.2
reqwest	0.12
redis	0.32
r2d2	0.8
dotenvy	0.15.7
decmed-rme-segment	path
decmed-macaroon-auth	path

Tabel I.3: Dependensi *Client*

<i>Crate</i>	Versi/Sumber
tauri	2
tauri-plugin-opener	2
tauri-plugin-http	2
tauri-plugin-fs	2
keyring	3
rustls-platform-verifier	0.5
rqrr	0.9
image	0.25

<i>Crate</i>	Versi/Sumber
tokio-stream	0.1.17
decmed-rme-segment	path
decmed-macaroon-auth	path
tauri-build	2

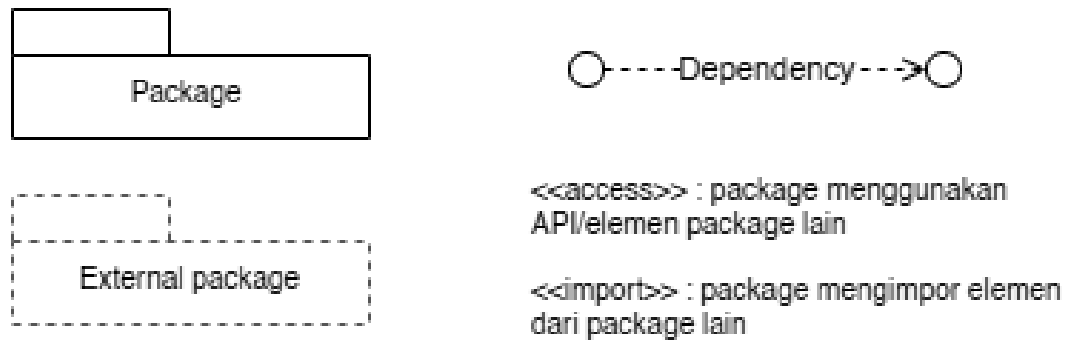
Tabel I.4: Dependensi decmed-rme-segment

<i>Crate</i>	Versi/Sumber
base64	0.22
serde	1
serde_json	1
sha2	0.10
uuid	1

Tabel I.5: Dependensi decmed-macaroon-auth

<i>Crate</i>	Versi/Sumber
chrono	0.4
serde	1
serde_json	1
thiserror	2.0
uuid	1
sha2	0.10
hex	0.4
macaroon	path
decmed-rme-segment	path

Lampiran J. Legenda Package Diagram



Gambar J.1: Legenda Package Diagram (Dokumentasi Penulis)

Lampiran K. Modul dan Implementasi *Crate* decmed-rme-segment

Tabel K.1: Daftar Modul dan Implementasi *Crate* decmed-rme-segment

Modul	Implementasi
types	CreateRmeSegmentRequest (<i>struct</i>), RmeSegmentData (<i>struct</i>) dengan new() dan validate(); ClientEncryptedRmeSegment (<i>struct</i>) dengan validate() dan into_metadata(); RmeSegmentMetadata (<i>struct</i>) dengan validate(); CreateRmeSegmentResponse (<i>struct</i>) dengan implementasi From<RmeSegmentMetadata>.
category	DatasetCategory (<i>enum</i>), FunctionCategory (<i>enum</i>), ALL_DATASET_CATEGORIES (<i>const</i>), ALL_FUNCTION_CATEGORIES (<i>const</i>).
crypto	canonical_json(), payload_hash(), sha256_hex(), ciphertext_integrity_hash_from_base64().
error	SegmentValidationError (<i>enum</i>).
administrative_payload	AdministrativeGeneralPayload (<i>struct</i>) dengan validate() dan to_payload_value(); administrative_general_payload_from_value(), administrative_general_payload_from_fields().
validation	assert_valid_correction_request(), assert_valid_correction_metadata(), get_allowed_function_categories(), is_valid_segment_category(), assert_valid_segment_category(), assert_segment_pair_consistent(), assert_no_plaintext_medical_fields().

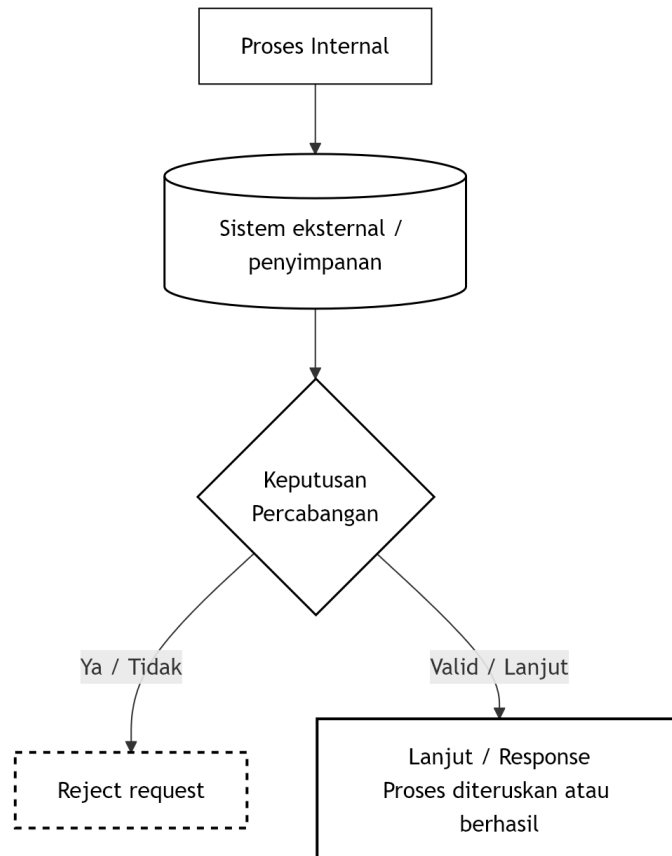
Lampiran L. Modul dan Implementasi *Crate* decmed-macaroon-auth

Tabel L.1: Daftar Modul dan Implementasi *Crate* decmed-macaroon-auth

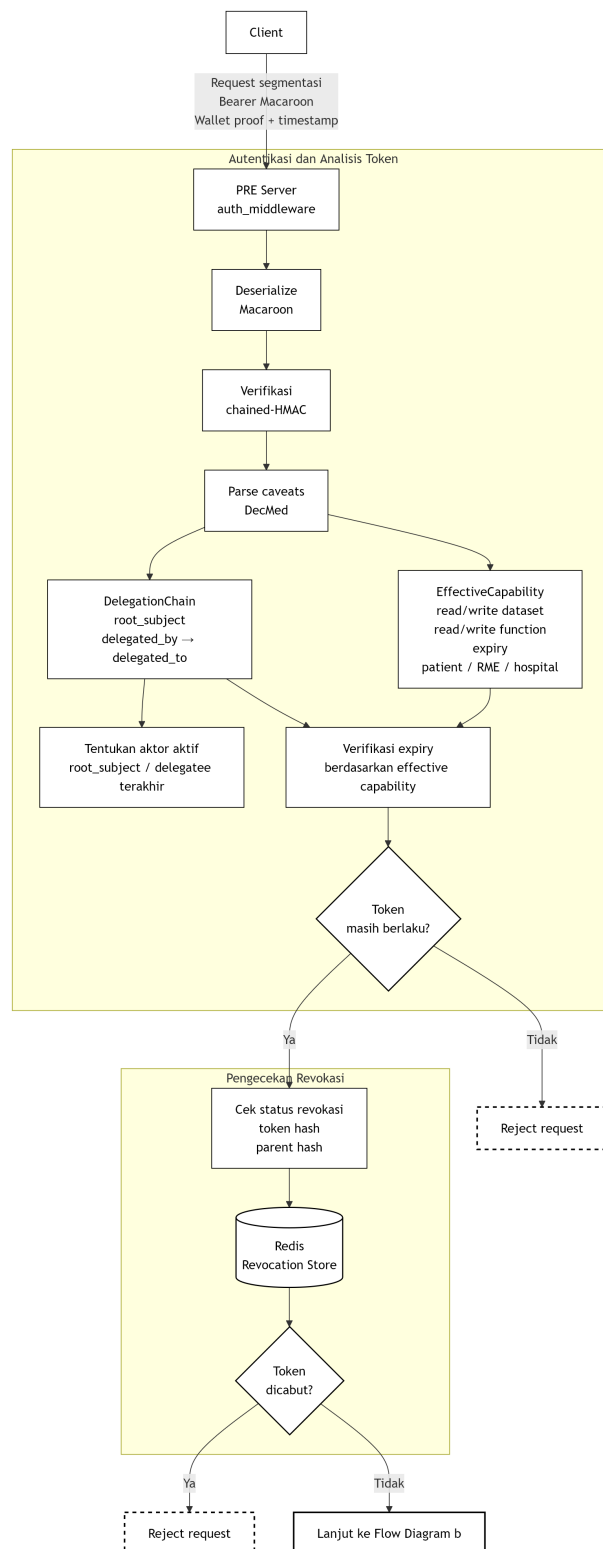
Modul	Implementasi
attenuation	DelegationAttenuationParams (<i>struct</i>), attenuate_macaroon().
caveats	CaveatKey (<i>enum</i>), CaveatValue (<i>enum</i>), DecmedCaveat (<i>struct</i>), ParsedCaveats (<i>struct</i>), parse_caveat_line(), format_dataset_list(), format_function_list(), caveat_line(), add_caveat_to_macaroon().
delegation	DelegationStep (<i>struct</i>), DelegationChain (<i>struct</i>) dengan from_parsed() dan delegation_depth().
delegation_ proof	DelegationProofContext (<i>struct</i>) dengan from_token(), from_last_delegation_step(), dan canonical_message(); hash_macaroon_token().
effective	AccessMode (<i>enum</i> : Read/Write), EffectiveCapability (<i>struct</i>) dengan from_parsed(), allows_dataset(), allows_function(), dan is_expired().
errors	CaveatVerificationError (<i>enum</i>).
issuance	AdminTokenKind (<i>enum</i> : Read/Update), InitialAdminPersonnelTokenParams (<i>struct</i>), admin_all_datasets(), admin_all_functions(), admin_write_datasets(), admin_write_functions(), issue_admin_personnel_token().
revocation	hash_token(), token_revocation_key(), compute_revocation_keys().
rme_id	format_related_rme_id().
verify	SegmentAccessContext (<i>struct</i>), TokenVerificationContext (<i>struct</i>), VerifiedDecmedToken (<i>struct</i>), verify_macaroon_signature(), verify_decmed_token(), verify_segment_access().
wallet_proof	WalletProofContext (<i>struct</i>) dengan canonical_message(), WalletSignatureVerifier (<i>trait</i>).

Lampiran M. Alur Implementasi Penegakan Otoritas pada Server PRE

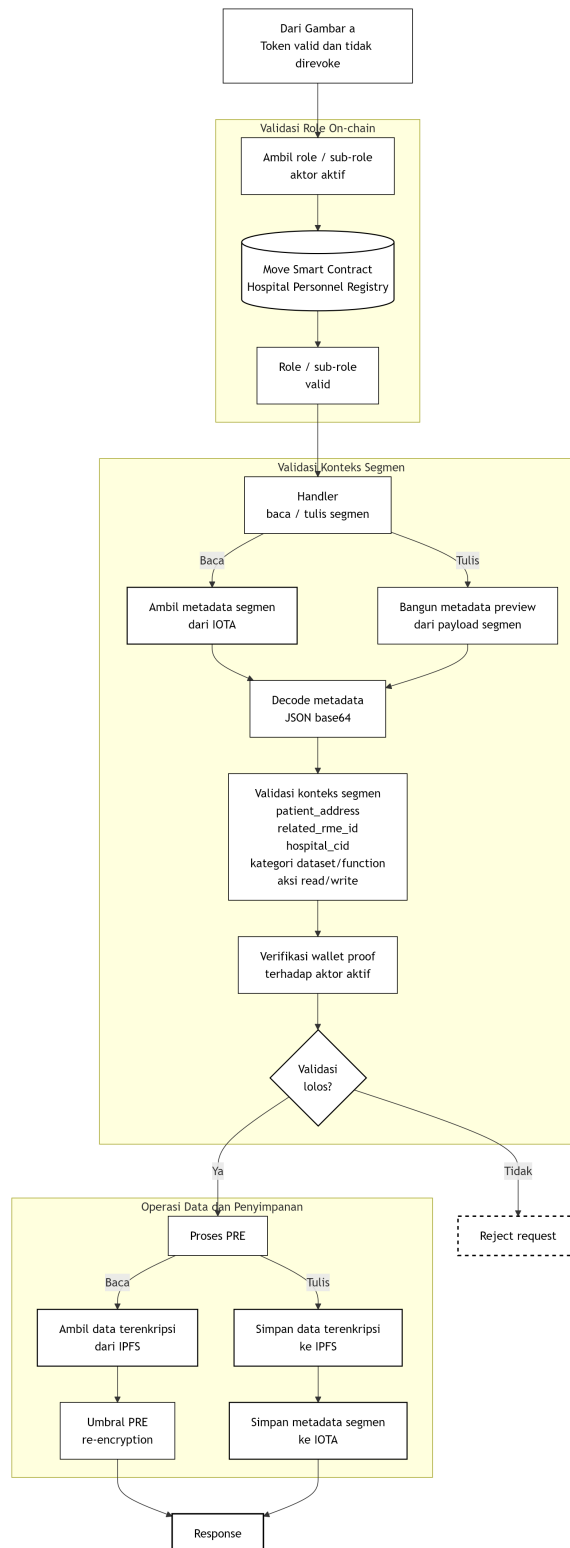
Alur penegakan otoritas pada *Server PRE* diilustrasikan dalam dua buah flow diagram yang saling berkelanjutan, sebagaimana ditunjukkan pada Gambar M.2 dan M.3. Gambar M.1 memuat legenda bentuk pada *Flow Diagram*.



Gambar M.1: Legenda bentuk pada *Flow Diagram*



Gambar M.2: Alur implementasi penegakan otoritas pada *Server PRE* bagian autentikasi token, *effective capability*, *delegation chain*, dan revokasi



Gambar M.3: Alur implementasi penegakan otoritas pada *Server PRE* bagian validasi *role* dan *subrole*, validasi segmen, *wallet proof*, serta eksekusi PRE, IPFS, dan IOTA

Lampiran N. Implementasi REST API Server PRE

Tabel N.1: Daftar Endpoint REST API pada Server PRE

Method	Endpoint	Fungsi	Status
GET	/	<i>Health check.</i>	Eksisting
GET	/medical-records	Mengambil daftar rekam medis.	Dimodifikasi
GET	/medical-record	Mengambil detail rekam medis.	Eksisting
POST	/medical-record	Membuat rekam medis.	Eksisting
PUT	/medical-record	Memperbarui rekam medis.	Eksisting
POST	/medical-record-segment	Membuat segmen rekam medis.	Baru
GET	/medical-record-update	Mengambil data untuk pembaruan RME.	Baru
GET	/administrative	Mengambil data administratif pasien.	Eksisting
GET	/gen/macaroon-key	Menghasilkan RootKey Macaroon.	Baru
GET	/gen/sig	Menghasilkan <i>signature</i> .	Eksisting
GET	/gen/proxy-address	Menghasilkan dan mendaftarkan alamat proxy.	Eksisting
POST	/nonce	Mengambil nonce.	Eksisting
POST	/keys	Menyimpan kunci.	Eksisting
POST	/revocations/patient	Mencabut hak akses dan turunannya.	Baru
POST	/delegations/attenuate	Melakukan atenuasi token Macaroon sesuai <i>request</i> .	Baru

Lampiran O. Implementasi Smart Contract Sistem DecMed

Tabel O.1: Daftar Module Smart Contract Sistem DecMed

Nama Module	Deskripsi
<code>decmed::admin</code>	Modul global admin untuk registrasi rumah sakit, aktivasi admin fasyankes, dan daftar rumah sakit.
<code>decmed::hospital_personnel</code>	Modul akun tenaga fasyankes, aktivasi, akses baca/tulis, <i>delegation</i> , dan <i>audit delegation</i> .
<code>decmed::patient</code>	Modul akun pasien, <i>grant</i> akses ke <code>AdministrativePersonnel</code> , log akses, <i>audit delegation</i> , dan pembacaan metadata RME milik pasien.
<code>decmed::proxy</code>	Modul yang dipakai <i>server</i> PRE untuk baca/tulis metadata RME.
<code>decmed::shared</code>	Modul penyedia <code>GlobalAdminCap</code> dan <code>ProxyCap</code>

Tabel O.2: Daftar Method pada Module `decmed::admin`

Nama Method	Deskripsi
<code>create_activation_key</code>	Membuat data rumah sakit dan activation key pertama untuk admin rumah sakit.
<code>update_activation_key</code>	Memperbarui activation key dan metadata admin rumah sakit.
<code>get_hospitals</code>	Mengambil daftar rumah sakit untuk client kementerian/global admin.

Tabel O.3: Daftar Method pada Module `decmed::hospital_personnel`

Nama Method	Deskripsi
<code>create_activation_key</code>	Admin fasyankes membuat activation key untuk tenaga administratif atau medis.
<code>use_activation_key</code>	Menandai activation key tenaga fasyankes sebagai sudah digunakan.
<code>signup</code>	Registrasi akun tenaga fasyankes ke address IOTA dan menyimpan metadata administratif.
<code>is_account_registered</code>	Mengecek apakah akun tenaga fasyankes sudah aktif/terdaftar.
<code>get_account_state</code>	Menentukan status <i>onboarding</i> tenaga fasyankes.
<code>get_account_info</code>	Mengambil metadata akun, <i>role</i> , <i>subrole</i> , dan metadata rumah sakit.
<code>update_account_activation_key</code>	Admin fasyankes mengganti activation key personel.
<code>update_administrative_metadata</code>	Memperbarui profil administratif tenaga fasyankes.
<code>get_hospital_personnels</code>	Mengambil daftar personel di bawah admin fasyankes.
<code>get_delegatee_candidates</code>	Mengambil kandidat personel yang bisa menerima delegasi akses.
<code>cleanup_read_access</code>	Membersihkan akses baca yang sudah kedaluwarsa.
<code>cleanup_update_access</code>	Membersihkan akses <i>update</i> yang sudah kedaluwarsa.
<code>get_read_access</code>	Mengambil daftar <i>capability</i> baca milik tenaga fasyankes.
<code>get_update_access</code>	Mengambil daftar <i>capability</i> update milik tenaga fasyankes.

Nama Method	Deskripsi
<code>create_delegated_access</code>	Membuat <i>delegated access</i> untuk personel lain.
<code>create_medical_record</code>	Membuat entri RME blob besar. (<i>legacy</i>)

Tabel O.4: Daftar Method pada Module `decmed::patient`

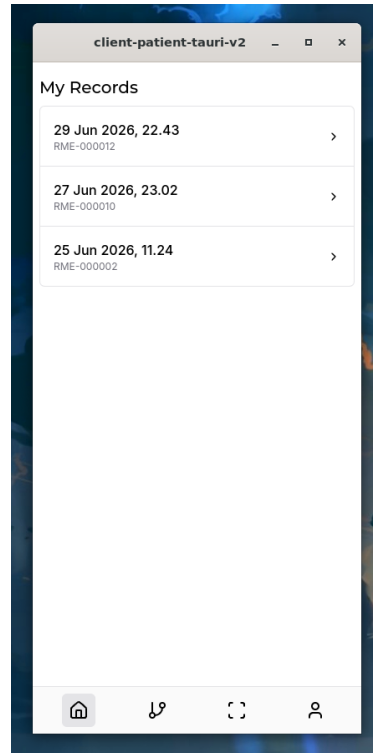
Nama Method	Deskripsi
<code>signup</code>	Registrasi akun pasien dan metadata administratif terenkripsi.
<code>is_account_registered</code>	Mengecek apakah <i>address</i> pasien terdaftar.
<code>get_account_state</code>	Menentukan status <i>onboarding</i> pasien.
<code>get_account_info</code>	Mengambil metadata administratif pasien.
<code>update_administrative_metadata</code>	Memperbarui metadata administratif pasien.
<code>get_hospital_personnel_info</code>	Mengambil info publik tenaga fasyankes saat pasien memproses QR.
<code>create_access</code>	Memberi akses baca/tulis kepada tenaga fasyankes.
<code>get_access_log</code>	Mengambil log <i>grant</i> akses pasien.
<code>get_delegation_audit_log</code>	Mengambil log <i>audit trail delegation</i> pasien.
<code>get_medical_records</code>	Mengambil daftar metadata RME pasien.
<code>get_medical_record</code>	Mengambil satu metadata RME berdasarkan indeks.
<code>revoke_access</code>	Mencabut akses tenaga fasyankes dan memicu <i>cascade revoke delegation</i> .

Tabel O.5: Daftar Method pada Module `decmed::proxy`

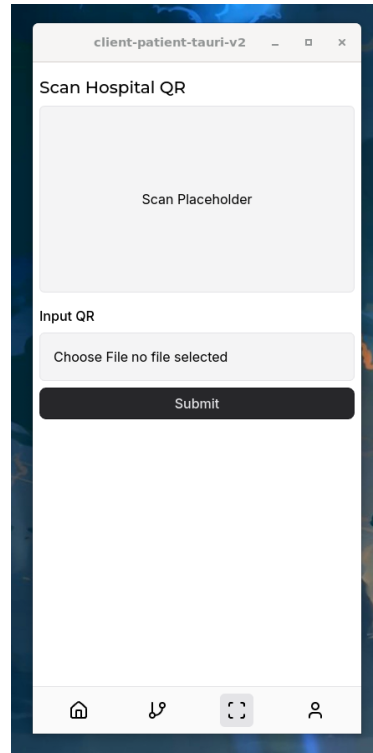
Nama Method	Deskripsi
<code>create_capability</code>	Membuat ProxyCap untuk address service proxy.
<code>create_medical_record_segment</code>	Menambahkan metadata RME tersegmentasi ke daftar medical metadata pasien.
<code>get_administrative_data</code>	Mengambil metadata administratif pasien untuk kebutuhan akses/dekripsi.
<code>get_hospital_personnel_role</code>	Mengecek role tenaga fasyankes saat setup token/key access.
<code>get_hospital_personnel_auth_info</code>	Mengecek <i>role</i> dan <i>subrole</i> .
<code>get_medical_records</code>	Mengambil daftar metadata RME pasien dan menyaringnya sebagai segmen berdasarkan cakupan akses.
<code>get_medical_record</code>	Mengambil satu metadata segmen dan data pendukung untuk pembacaan <i>payload</i> .
<code>is_patient_registered</code>	Memvalidasi bahwa pasien terdaftar
<code>create_medical_record</code>	Membuat entri RME blob besar. (<i>legacy</i>)
<code>update_medical_record</code>	Melakukan edit entri RME blob besar. (<i>legacy</i>)
<code>get_medical_record_update</code>	Mengambil data RME untuk proses edit entri blob besar. (<i>legacy</i>)
<code>get_delegation_access_snapshot</code>	Mengambil <i>snapshot</i> metadata akses delegasi.

Lampiran P. Tangkapan Layar Implementasi *Patient Client*

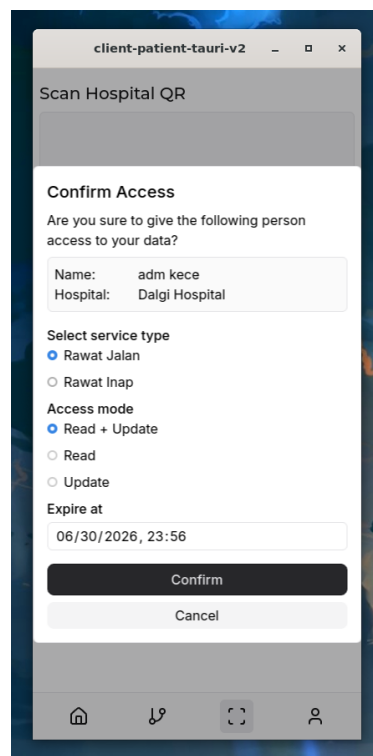
Semua tangkapan layar berikut merupakan dokumentasi penulis.



Gambar P.1: Halaman *Home* daftar RME.



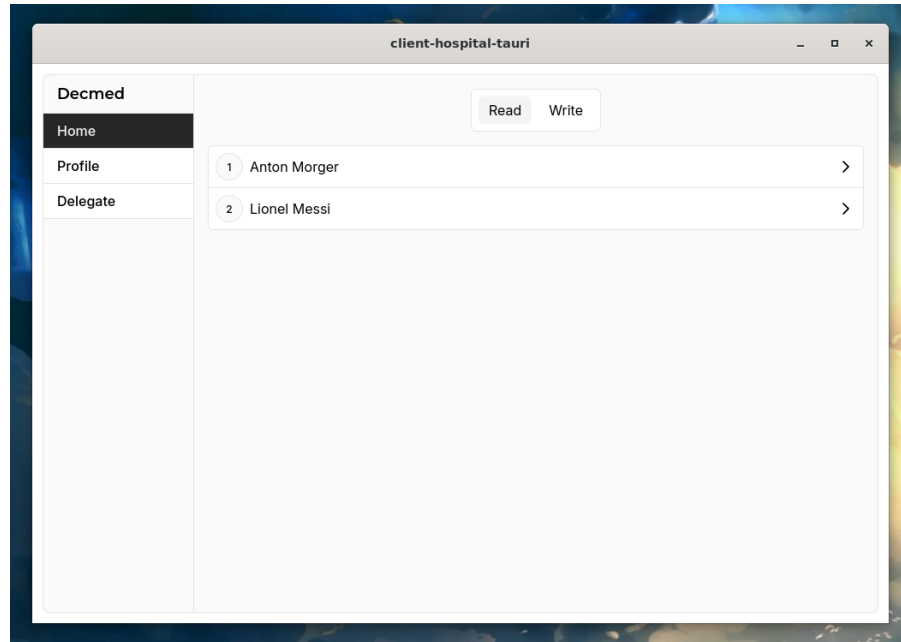
Gambar P.4: Halaman *scan* QR.



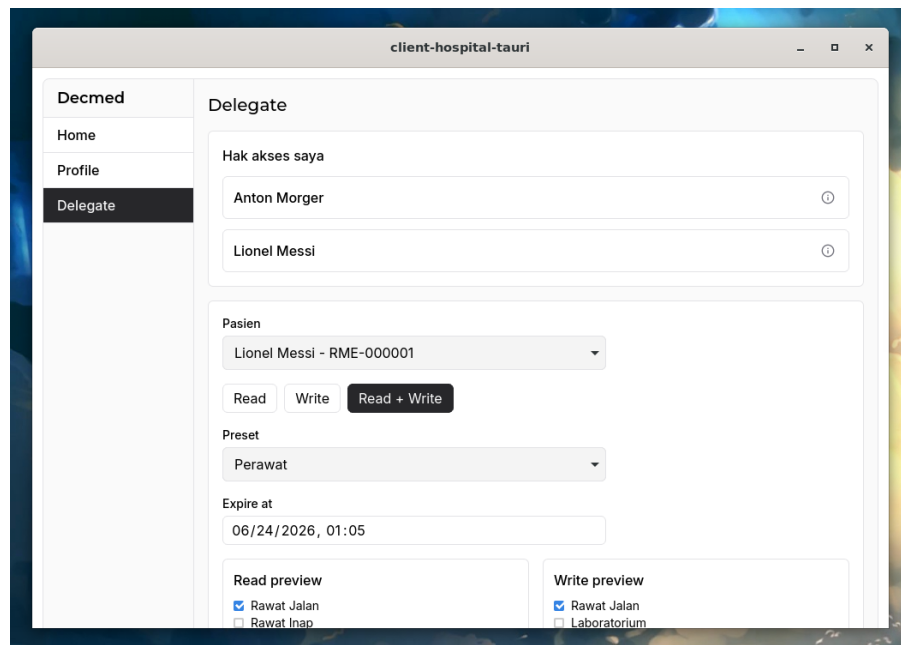
Gambar P.5: Halaman konfirmasi pemberian akses awal.

Lampiran Q. Tangkapan Layar Implementasi *Hospital Client*

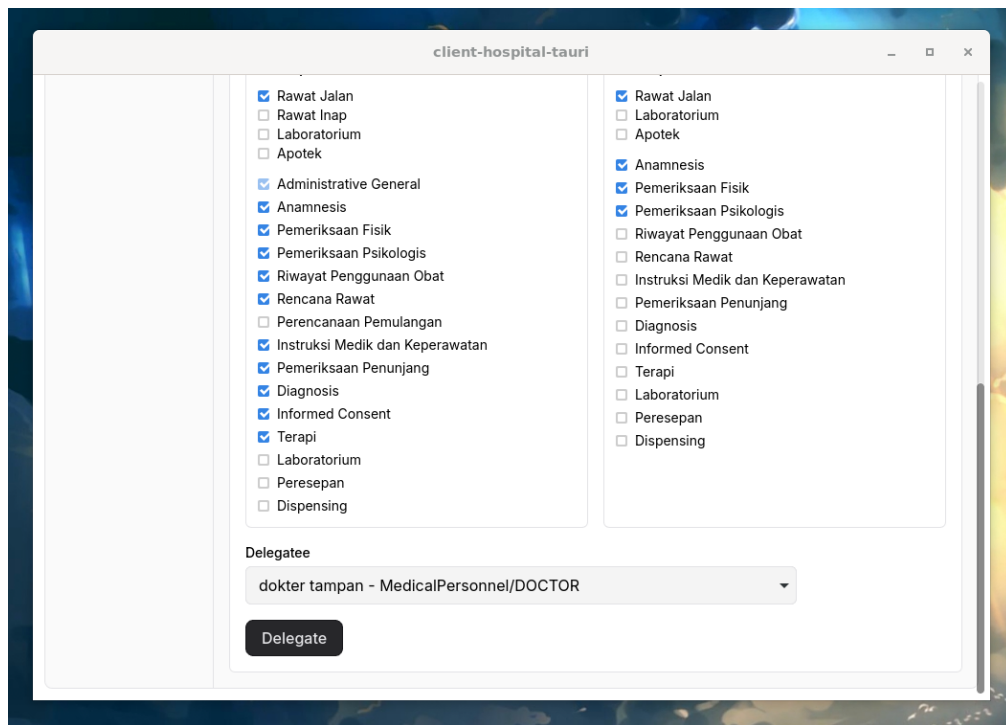
Semua tangkapan layar berikut merupakan dokumentasi penulis.



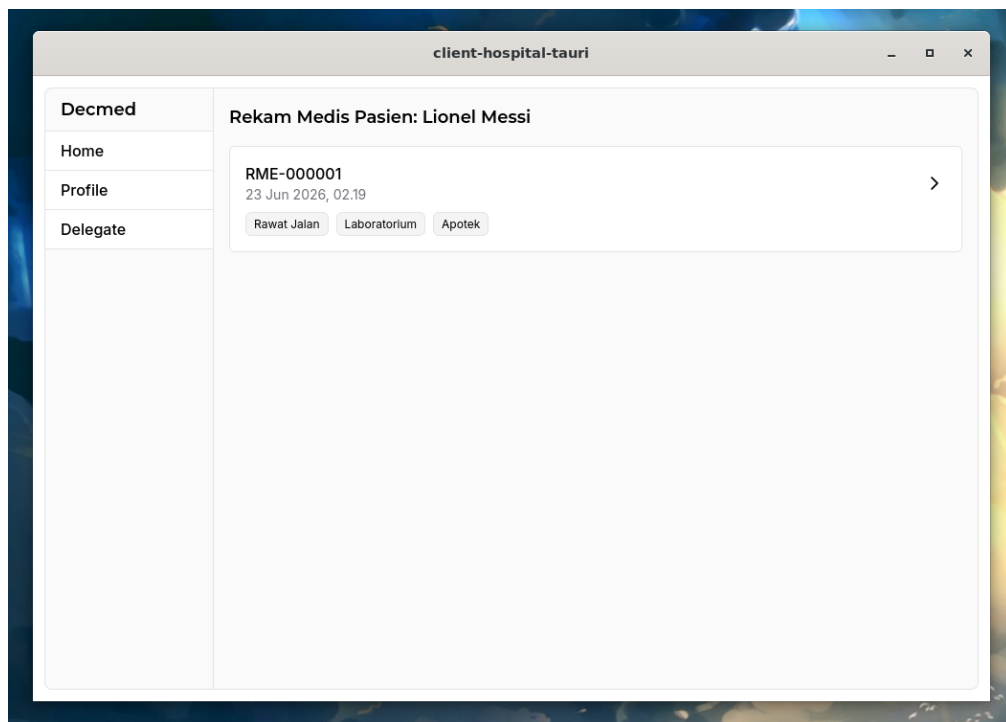
Gambar Q.1: Halaman *dashboard MedicalPersonnel*.



Gambar Q.2: Halaman delegasi akses bagian 1.



Gambar Q.3: Halaman delegasi akses bagian 2.



Gambar Q.4: Halaman daftar RME pasien pada *Hospital Client*.

client-hospital-tauri

Decmed

Home

Profile

Delegate

← Back

Detail RME

RME-000001

Rawat Jalan Laboratorium Apotek

Fungsi Administrative General

Administrative Data

NIK	3277021203040002
Name	Lionel Messi
Birth Place	Cimahi
Date of Birth	2000-03-03
Gender	Male
Religion	Katolik
Education	SMA
Occupation	Pemain
Marital Status	Married

Gambar Q.5: Halaman detail RME pada *Hospital Client*.

client-hospital-tauri

Decmed

Home

Profile

Delegate

Write RME

Lionel Messi

0x4858a58fb137a1a84336f6cd2cc42079a8216d9aee1f852f830e4d18185ce95c

Dataset

Rawat Jalan

Anamnesis

Pasien mengeluh demam tinggi sejak 3 hari, disertai nyeri kepala dan lemas. Nafsu makan menurun. Tidak batuk, tidak pilek. Pernah tifoid 2 tahun lalu.

Pemeriksaan Fisik

Pemeriksaan Psikologis

Gambar Q.6: Halaman alur akses tulis pada *Hospital Client*.